

复杂系统总体设计指标形式化表达与验证方法

陈婧琦

2022 年 6 月

中图分类号：TP391

UDC 分类号：621

复杂系统总体设计指标形式化表达与验证方法

作者姓名	陈婧琦
学院名称	机械与车辆学院
指导教师	王国新副教授
答辩委员会主席	胡耀光教授
申请学位	工学硕士
学科专业	机械工程
学位授予单位	北京理工大学
论文答辩日期	2022 年 6 月

Formal Specification and Verification for the Property Measures of Complex System Design

Candidate Name: Chen Jingqi

School or Department: School of Mechanical Engineering

Faculty Mentor: Prof. Wang Guoxin

Chair, Thesis Committee: Prof. Hu Yaoguang

Degree Applied: Master of Science in Engineering

Major: Mechanical Engineering

Degree by: Beijing Institute of Technology

The Date of Defence: June, 2022

复杂系统总体设计指标形式化表达与验证方法

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签 名： 日期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名： 日期：

导师签名： 日期：

摘要

采用基于模型的系统工程（Model-based System Engineering, MBSE）进行复杂系统设计时，指标的形式化验证是其中的一项挑战。具体体现如下：（1）指标定义完整性：缺少一种考虑多维度因素和多层次递进分解的指标定义方法。（2）指标描述精确性：现有图形化建模语言缺乏精确的语义刻画指标及其约束关系，难以形式化表达并验证指标。（3）指标验证通用性：在多种建模语言描述的复杂系统情景下，难以实现跨异构语言的指标验证。针对上述 3 类问题，本文提出一种支持跨建模语言、跨模型、自定义指标的指标验证方法。该方法突破异构语言限制，保持建模和指标验证的数据一致性，形式化描述和验证的理论一致性，降低因数据和方法的不一致造成的结果不确定性，从而提高设计效率和质量。本文主要研究内容如下：

（1）面向复杂系统设计的多维多层指标体系定义方法。首先，构建面向 MBSE 的霍尔系统工程方法，扩展系统指标定义的维度。其次，依据系统维的“需求定义-功能架构-逻辑架构-物理架构”的系统分析流程定义多层指标体系。最后，考虑“社会-实践-信息-技术”四维框架，细化每层指标的维度，形成面向复杂系统设计的多维多层指标体系。

（2）指标形式化表达与验证方法。首先，提出采用 GOPPRR-E（Graph, Object, Point, Property, Relationship, Role, Extension）元建模方法形式化描述异构建模语言的模型元素。其次，研究可满足性模理论描述和推理一阶逻辑表达式的能力。最后，构建可满足性模理论应用对象和 GOPPRR-E 元建模方法应用对象之间的映射关系。并结合上一研究内容定义的指标体系，明确面向元元模型的指标验证类型，从而支持形式化描述指标和系统模型之间的约束关系，验证约束关系的可满足性。

（3）指标验证语言设计与实现。该研究依据前文研究的指标验证理论方法，在基于 GOPPRR-E 理论构建的多架构建模语言 KARMA（Combination of Architecture Model Specification）的基础上，定义指标验证语言的语法和语义，实现模型建立、模型约束描述和验证的统一表达。首先，该研究分析多架构建模语言 KARMA 和基于可满足性模理论的 SMT-LIB（Satisfiability Modulo Theories Library）语言规范的语法和语义。其次，构建面向多架构模型的指标验证语法，突破异构建模语言的限制。最后，基于所提出的指标验证的语法和语义构建相应的语言编译器，实现指标验证功能。

（4）卫星姿态控制系统指标设计与验证案例。案例以卫星的姿态控制系统设计

为研究对象，定义卫星设计过程的建模方法，并根据系统分析流程完成指标体系的定义。最后，采用所提指标验证方法解决卫星设计过程中多设计方案确认和静态指标验证问题，验证该方法的可行性。

关键词：基于模型的系统工程；指标验证；形式化验证；方案确认；KARMA

Abstract

There exist 3 challenges of property validation in the design of complex systems using Model-based System Engineering (MBSE). It is embodied as follows: (1) Integrity of index definition: there is a lack of design index definition method that considers multi-dimensional factors and multi-level progressive decomposition. (2) Accuracy of indicator description: existing graphical modeling languages lack accurate semantic description of indicators and their constraint relations, making it difficult to formally express and verify indicators. (3) Versatility of property verification: the complex system scenario need be described by multiple modeling languages, and the current property verification is difficult to achieve indicator verification across heterogeneous languages. Aiming at the above three kinds of problems, this paper proposes an property verification method that supports cross-modeling language, cross-model and self-defined indexes. This method breaks through the limitation of heterogeneous language, which maintains the data consistency of modeling and index verification and the theoretical consistency of formal description and verification, and reduces the uncertainty of results caused by the inconsistency of data and methods to improve the design efficiency and quality. The specific research contents of this paper are as follows:

(1) Research on the definition method of multidimensional-multilayer measure system for complex system design. Firstly, the Hall system engineering method for MBSE is constructed to expand the dimension of system measure definition. Secondly, the multi-layer index system is defined according to the modeling process of "requirement definition function definition logical architecture physical architecture". Finally, consider the four-dimensional framework of "society practice information technology", refine the dimension of each layer of measures to form a measure system for complex system design.

(2) Research on formal expression and verification method of measures for system design. Firstly, GOPPRR-E meta modeling theory is used to formally describe the model elements of heterogeneous modeling languages. Secondly, the Satisfiability Modulo Theory is used to describe and infer first-order logical expressions. Finally, the mapping relationship between object of Satisfiability Modulo Theory and the object of GOPPRR-E meta modeling theory is constructed. Combined with the defined measure system, the property verification

type for meta model is clarified, so as to support the formal description of the constraint relationship between the measures and the system architecture, and verify the satisfiability of the constraint relationship.

(3) Design and implementation of property verification language. The syntax and semantics of property verification language is defined based on the multi architecture modeling language KARMA constructed by GOPPRR-E meta-meta models to realize the unified expression of model establishment, model constraint description and verification. Firstly, this study analyzes the syntax and semantics of KARMA and SMT-LIB language based on Satisfiability Modulo Theory. Secondly, the syntax for multi-architecture model is defined to break through the limitations of heterogeneous modeling languages. Finally, based on the defined property verification language, the corresponding language compiler is constructed to realize the property verification function.

(4) Case study based on attitude control system design of satellite. The case takes the design of attitude control system as the research object, defines the modeling method of satellite design process. And the definition of measure system according to the modeling process is defined. Finally, the proposed property verification method is used to solve the design space exploration and static property verification in the process of satellite design. Therefore the feasibility of this method is validated.

Key Words: Model-based System Engineering; Property verification; Formal verification; Space exploration; KARMA;

目录

第 1 章 绪论	1
1.1 研究背景	1
1.2 国内外研究现状	2
1.2.1 复杂系统设计	2
1.2.2 MBSE 技术	3
1.2.3 指标验证	6
1.2.4 存在的问题	9
1.3 研究意义	10
1.4 研究内容和组织架构	11
1.4.1 研究内容	11
1.4.2 组织架构	12
1.5 本章小结	14
第 2 章 多维多层系统设计指标体系定义方法	15
2.1 基于霍尔系统工程方法的多维设计指标定义	15
2.1.1 霍尔系统工程	15
2.1.2 面向 MBSE 的霍尔系统工程方法	17
2.2 基于 RFLP 系统分析流程的多层指标体系定义	20
2.2.1 RFLP 系统分析流程	20
2.2.2 多层指标体系定义方法	22
2.3 面向多层指标基于 SPIT 方法的指标维度细化	24
2.3.1 SPIT 方法	24
2.3.2 基于 SPIT 定义指标体系维度	25
2.4 面向复杂系统设计过程的多维多层指标体系定义方法	26
2.5 本章小结	29
第 3 章 系统设计指标形式化表达与验证方法	31
3.1 基于元元模型的元建模方法	31
3.1.1 GOPPRR-E 元建模方法	31
3.1.2 GOPPRR-E 元建模方法的优势	34

3.2 可满足性模理论	35
3.2.1 可满足性模理论的内涵	36
3.2.2 可满足性模理论的应用域	39
3.2.3 基于可满足性模理论的可满足性求解	46
3.2.4 可满足性模理论的优势	48
3.3 结合 GOPPRR-E 和可满足性模理论的指标验证方法	48
3.3.1 基于 GOPPRR-E 的指标验证形式化表达	49
3.3.2 指标验证的分类及形式化表达	52
3.3.3 指标验证方法研究	57
3.4 本章小结	57
第 4 章 指标验证语言设计与实现	59
4.1 基于 KARMA 的指标验证语言规划	59
4.1.1 语法和语义	59
4.1.2 KARMA 语言分析	60
4.1.3 SMT-LIB 语言分析	62
4.1.4 KARMA-指标验证语言规划	64
4.2 KARMA-指标验证语言研究	64
4.2.1 指标验证语言的抽象语法	65
4.2.2 指标验证语言的语法设计	67
4.2.3 指标验证语言支持指标的形式化表达和验证	74
4.3 指标验证语言的实现	76
4.3.1 基于 ANTLR 的构建指标验证语言编译器	76
4.3.2 指标验证功能的实现	78
4.4 本章小结	80
第 5 章 面向卫星姿态控制系统设计的案例研究	81
5.1 卫星设计问题陈述	81
5.2 卫星设计语言和工具	82
5.3 卫星总体架构模型构建	83
5.4 模型描述	85
5.5 指标体系构建	88

5.6 指标验证	89
5.6.1 面向卫星姿控系统逻辑单元的设计方案确认	89
5.6.2 姿态控制系统的可靠性验证	106
5.7 案例讨论	110
5.8 本章小结	112
结论与展望	113
参考文献	116
附录 A 语法规范（部分）	123
攻读学位期间发表论文与研究成果清单	139

本文图目录

图 1.1 研究框架	13
图 2.1 传统霍尔三维结构	15
图 2.2 霍尔三维结构逻辑维执行方法	17
图 2.3 基于 RFLP 流程的 V 字开发过程	18
图 2.4 面向 MBSE 的霍尔系统工程	19
图 2.5 应用于复杂系统设计过程的 V 模型研发流程	22
图 2.6 基于 RFLP 系统分析流程定义的多层指标体系	23
图 2.7 适用于复杂系统设计的 SPIT 方法	25
图 2.8 社会层指标定义方法	26
图 2.9 指标体系定义方法	27
图 2.10 面向卫星设计过程的多维多层指标体系	29
图 3.1 基于 GOPPRR-E 元建模方法的建模方法	33
图 3.2 可满足性模理论的应用域	40
图 3.3 差分不等式的加权有向图	44
图 3.4 基于可满足性模理论的求解过程	46
图 3.5 面向异构模型的指标验证方法	49
图 3.6 面向模型元素的指标验证方法	57
图 4.1 领域建模语言的语法和语义	59
图 4.2 KARMA 语言抽象语法	60
图 4.3 SMT-LIB 语言抽象语法	63
图 4.4 KARMA-指标验证语言抽象树（整体）	65
图 4.5 KARMA-指标验证语言抽象语法树（指标验证部分）	65
图 4.6 基础数据类型和表达式类型的抽象语法树	66
图 4.7 面向车辆底盘质量的建模	75
图 4.8 车辆底盘质量验证结果图	76
图 4.9 基于 ANTLR 语法分析器的编译器工作流程	77
图 4.10 指标验证实现方法	79

图 4.11 实现指标验证功能的工具设计方案	79
图 4.12 KARMA 指标验证文本输入界面	80
图 5.1 卫星示意图	81
图 5.2 卫星模型构建流程	84
图 5.3 卫星全生命周期用例图及细化飞行任务活动图	85
图 5.4 A.3 总体需求和指标图	86
图 5.5 卫星总体模块定义图	87
图 5.6 姿态控制系统的功能和非功能指标	89
图 5.7 姿态控制系统的用例图及分解的活动图	90
图 5.8 姿态控制模块定义图	91
图 5.9 逻辑单元选型-模块图	93
图 5.10 提取模型属性	97
图 5.11 指标验证应用于姿态稳定方式的选型过程	98
图 5.12 姿态稳定方式求解结果	100
图 5.13 Excel 形式的姿态稳定方式选型结构	101
图 5.14 姿态敏感器和调整机构结果	106
图 5.15 姿态控制系统的活动图	107
图 5.16 可靠性验证结果	110

本文表目录

表 1.1 指标体系构建方法.....	7
表 1.2 指标验证方法	9
表 3.1 元建模方法比较	35
表 3.2 一阶逻辑中的逻辑符号.....	39
表 3.3 一阶逻辑中的非逻辑符号	38
表 3.4 机器数符号	41
表 3.5 机器运算的枚举	43
表 3.6 字符串理论语法定义	45
表 3.7 GOPPRR 模型元素和数学类型的对应关系	50
表 4.1 KARMA 语言关键字	61
表 4.2 指标验证语法关键字	67
表 4.3 指标验证语言支持描述的模型元素	73
表 4.4 指标验证语言支持的可满足性模理论方法	74
表 5.1 根据顶层需求定义的效能指标及其细化.....	87
表 5.2 卫星常见姿态稳定方式的特性	91
表 5.3 姿态敏感器类型及相关特性.....	92
表 5.4 姿态控制系统执行机构特性.....	92
表 5.5 姿控系统选型后结果	106

论文术语

(1) 指标：评估系统状态的目标属性或特征。

(2) 设计指标：评估系统设计质量的目标属性或特征。

(3) 形式化表达：以精确的语法和语义描述。

(4) 语法：语法包含具体语法和抽象语法。具体语法定义了领域语言中元素的表示方法，可以为图形形式也可以为文本形式。抽象语法定义了领域语言中可用的语言元素的概念、元素之间的关系、元素之间的交互规则及约束。

(5) 语义：语义包含语义的定义域和映射关系。语义定义域通过数学逻辑定义了各元素的含义，语义映射关系定义了语法到语义定义域的映射。

(6) 指标验证：规范和评估指标和现有系统模型之间的关系，其验证对象是指标和目前系统模型之间的可满足性，用于判断目前的模型设计是否达到指标。

(7) MBSE 技术：MBSE (Model-based System Engineering) 称为基于模型的系统工程方法，MBSE 采用面向对象的图形化建模语言描述系统的组成元素，形成集成化、一致化、规范化的可视化系统模型，实现系统设计过程中需求、功能、架构的关系追溯、分析与验证。

(8) 设计指标体系：是能够反映系统设计质量的一系列相对独立但又相互关联的指标所形成的整体，包含指标维度和指标层次。指标维度是描述指标体系的角度，指标层次是描述指标体系的细化程度。

(9) 霍尔系统工程：是解决大型工程系统规划、组织和管理问题的一种方法，它主要从时间维、逻辑维和知识维 3 个维度构建问题解决方案

(10) RFLP 系统分析过程：RFLP 指的是一种服务于 V 型开发流程的系统分析方法：需求定义—功能架构—逻辑架构—物理架构 (Requirement - Function - Logic - Physics, RFLP)，对系统的整体设计、开发和测试提供支持。架构指的是解决方案。需求定义只从利益相关人出发，明确任务交付时利益相关人的评估要求。功能架构描述分系统或更小级别的结构为满足需求所提供的服务。逻辑架构设计是定义能够实现子功能的组件，描述功能是如何实现的以及系统的抽象动态变化。物理架构定义了物理设备及其接口的关系与参数，其目的是描述具体的、可实现的解决方案。

(11) SPIT 分析法：SPIT (Social-Practice-Information-Technical) 分析方法将相同的基本元素进行分组，并提供一致的抽象级别，以便理解整个超大复杂系统的共同

特征，包含社会层、实践层、信息层和技术层。

(12) **GOPRR-E 元建模方法**：是一种基于元元模型构建元模型的建模方法，用于描述异构模型元素和模型关系，支持模型元素的形式化。

(13) **元元模型**：六种元元模型：图、对象、属性、点、对象、角色 **GOPRR-E** 元建模方法的，以及它们之间的关系，这里所有的元素并非指某个具体的点或图，而是指一种抽象类。图表示一种模型元素的集合关系；对象表示图类中的一个实体；属性指其他五个元元模型中的属性，它的数据类型通常以数值、布尔、字符串、数组形式存在；点是指对象实例的一个端口；关系是指对象之间，或点之间，或对象和点之间的一种连接。

(14) **元模型**：元模型是基于元模型定义的，并且继承了元元模型之间的关系。常见的建模语言是基于元模型定义的，元模型定义了建模语言的概念及其关系，以及建模规则。

(15) **模型**：模型是由元模型的实例组合而成，用抽象的方式描述物理世界。

(16) **可满足性模理论**：可满足性模理论 (**Satisfiable Modulo Theory, SMT**) 是包含一系列公理的一组逻辑理论，可以检验包含数学理论的一阶逻辑表达式是否可满足。

(17) **指标约束**：指标约束指的是和指标和现有系统解决方案之间的约束关系。

(18) **KARMA 语言**：**KARMA (Combination of ARchitecture Model specificAtion)** 是一种基于 **GOPRR-E** 元建模方法构建的统一多架构建模语言，能够支持不同层次（模型层、元模型层、元元模型层）的模型描述。

(19) **SMT-LIB 语言**：是依据可满足性模理论构建的函数式语言。

第1章 绪论

1.1 研究背景

随着科学技术的不断进步，系统的复杂程度越来越高，许多工程领域内的系统被定义为复杂系统。这类复杂系统通常组件构成复杂^[1]、接口复杂、交互复杂^[2]。相应的，复杂系统的设计也给工程人员在带来了巨大的挑战^[3]。传统的系统设计方式在多任务管理、研发全流程协同设计、数据管理和复杂系统可靠性方面表现不佳，无法应对复杂系统设计中多方利益相关者对系统设计任务的整体管理挑战、多层次跨学科系统协同设计挑战、设计全生命周期的异构信息管理挑战和系统全生命周期中多层级的验证与确认挑战。为应对上述挑战，国际系统工程组织（INCOSE）于 2007 年提出基于模型的系统工程方法^[4]（Model-based System Engineering, MBSE）。MBSE 的核心是模型，它结合图形化建模语言的优势，采用形式化建模的手段支持系统需求定义、设计、分析、验证和确认等系统设计的全生命周期活动^[5]。该方法以形式化的模型为基础，构建系统设计中多维度、全生命周期的模型，实现信息的无歧义表达、各阶段模型可评估，并且能够增强信息捕获和重用的能力，大大降低了沟通成本、设计风险和设计周期。

采用 MBSE 方法进行系统设计时，验证和确认贯穿整个过程，是保证系统设计质量、提高开发效率的关键步骤。验证^[6]的目的是检验某一阶段的系统设计是否符合其设计规范，而设计规范一般以形式化的形式提供，比如电路之间的接口关系；确认^[6]的目的是系统的设计是否符合各利益相关者的需求，比如电路最终实现的功能是否符合设计者的需求。

系统的验证和确认是通过指标实现的，指标是评估系统状态的目标属性或特征^[32, 36]。对指标的验证评估系统状态、牵引系统设计、保证设计质量。目前，使用 MBSE 技术实现指标验证的手段主要包含模拟仿真，模拟仿真通常应用在设计后期，其执行阶段晚、对仿真人员专业要求高、仿真的时间成本高。因此，本文关注一种形式化的指标验证手段（下面简称为指标验证）。指标验证作为一种形式化的验证手段，可以弥补上述仿真手段的一些不足。指标验证能够规范和评估指标和现有系统模型之间的关系，其验证对象是指标和目前系统模型之间的可满足性^[77]，用于判断目前的模型设计是否达到指标^[34, 79]。

然而,目前 MBSE 领域内指标验证方法具有一定局限性:其一,指标体系定义不完整;现有指标验证方法支持的验证对象较为零散,没有形成和物理单元关联的多维多层系统设计指标体系;其二,指标描述不精确;目前常用于 MBSE 领域模型的建模语言(例如 UML(Unified Modeling Language)^[40]、SysML(System Modeling Language)^[39])为半形式化的语言,不具备精确语义,无法形式化描述模型元素,也不能量化和验证指标。其三,指标验证方法不具备通用性,现有指标验证方法只能针对某种建模语言,建模语言之间的数据是异构的,因此大部分指标验证方法只支持某一种模型的验证,如:SysML、UML,不具备面向跨建模语言使用指标验证的能力、重用性低。

1.2 国内外研究现状

1.2.1 复杂系统设计

复杂系统目前并没有一个明确的定义。Hobday 率先对复杂系统(Complex Product System, CoPS)进行了定义,认为这是一种涉及多方利益方的高成本、工程密集型、系统复杂、网络复杂、结构复杂的知识密集型系统^[7]。类似的概念还包括大型产品、复杂技术系统、大型技术系统等。Rechtin 将超大系统也定义为复杂系统,一种规模大、进化速度快和利益相关者多,具有高质量的可靠性、安全性、质量、信息和数据完整的体系^[8]。Northrop 则认为复杂系统是一种软件、制度、人、政策、经济相互依赖的网络系统^[9]。Prencipe 认为复杂产品是一个具有高成本、高工程量以及由众多子系统组成的产品系统^[10]。国内研究人员大多沿用了以上的复杂系统定义,徐路宁将复杂产品定义为用户需求复杂、产品结构复杂、产品技术复杂、加工制造复杂、项目管理复杂的一类产品^[11]。付振通则总结复杂系统的特点,将其定义为产品多层次结构、技术高度集成、研发生产成本高、设计集成、利益相关方复杂且参与度高的系统^[12]。

综上所述,复杂系统是一种需求复杂、组成结构复杂、交互复杂、制造管理复杂的系统。此类系统的设计过程面临着众多的挑战:1)多方利益相关者对系统设计任务的整体管理。复杂系统设计任务的任务难度高、周期长,往往需要不同领域的技术和管理人员共同协作。往年的研制经验为复杂系统的成功研制奠定了坚实的基础,然而,传统基于文档的设计和管理缺乏结构化的关联形式,不利于研制人员的重用。这种自然语言的表达为不同领域技术人员的沟通协作、知识的继承、管理与创新带来了挑战。因此,需用一种统一的、形象化的、包含设计知识和内容的表达方式,帮助跨

域设计人员实现更高效的统一管理^[58]。2) **多层级跨学科的协同设计**。复杂系统面临的运行环境往往也非常复杂,传统的集同方式通常使用文字汇报、统一开会、分别设计的方法。其协调设计的效率较低、缺乏系统完备性、模块设计的一致性低。因此,需要为不同领域、不同层级、不同组织的设计构建一种自主可控的集成设计环境^[13]。3) **设计全生命周期的异构数据管理**。复杂系统的组成复杂决定其难以只使用一种工具进行设计。众多数据往往存储在不同的工具或者文本中,为数据的传递和重用带来困难。因此,需要构建一种统一的异构数据管理方式,支持追溯、确认、使用、管理此类数据^[14]。4) **多层级的确认与验证**。复杂系统设计任务的顶层需求往往需要同时满足多方利益人的相关需求,如:可用性、可靠性、质量、经费、技术水平等。同时,复杂系统由于制造和运行环境的复杂,可能难以构造完整的实物验证^[13]。因此,复杂系统的验证迫切需要面向全局构建需求的分析追溯,定义相关的设计指标,验证前移、加强多层级验证和确认,减少设计工作的反复,提高设计质量和效率^[13]。

1.2.2 MBSE 技术

MBSE 是应对复杂系统设计上述挑战的数字化研制方法。MBSE 方法强调以模型为核心贯穿系统设计的全生命周期。MBSE 方法的关键在于 1) 构建系统的全生命周期架构模型^[18],架构即模型形式的方案。使用 MBSE 方法进行系统设计包含构建系统全生命周期的需求定义、概念设计、详细设计等模型,以及各视角模型的关联。2) 通过系统建模语言构建架构模型,提供全面、准确、一致的评估系统的基础^[19, 20]。MBSE 方法采用面向对象的图形化建模语言描述系统的组成元素,形成集成化、一致化、规范化的可视化系统模型,便于统一管理模型和数据,实现系统设计过程中需求、功能、架构的关系追溯、分析与验证^[21]。

本节主要研究 MBSE 技术的三大支柱^[22]: 建模方法、建模语言和建模工具,以及 MBSE 领域内的系统确认和验证。

1.2.2.1 MBSE 建模方法

建模方法指的一种融合模型创建、模型管理、模型使用的建模框架^[22]。MBSE 领域的典型方法论包括 HARMONY-SE 方法、面向对象的系统工程方法 (Object-oriented Systems Engineering Method, OOSEM)、Arcadia 方法、对象过程方法论 (Object Process Methodology, OPM)。

（1）HARMONY-SE 方法

2013 年，IBM 公司提出一套 MBSE 方法论 Harmony，Harmony 流程在映了经典的系统开发生命周期设计“v 模型”，其方法学的顶层内容分为 3 部分：需求分析、系统功能分析、结构综合设计。需求分析是指收集利益相关人的需求，并将其转化为功能性和非功能性的系统需求；功能分析是将需求转化为实现需求的系统操作的过程；设计综合过程是开发同时满足功能需求和非功能需求的体系架构。目前，Harmony 生态系统最初在软件开发和嵌入式应用^[27]较多，随着平台的完善与推广，在工业领域也有了较多的应用。

（2）面向对象的系统工程方法

面向对象系统工程方法（Object-Oriented Systems Engineering Method, OOSEM）^[28]集成了一种自上向下的、基于模型的方法。该方法使用 SysML 支持系统的规范、分析、设计和验证。OOSEM 包括以下开发活动：分析干系人需求、定义系统需求、定义逻辑架构、综合候选分配架构、优化和评估备选方案以及验证和验证系统^[29]。这些活动是以迭代的方式执行的，可以应用于系统的每个层次结构级别且具备相应的管理过程用于支持这些活动^[30]。

（3）Arcadia 方法

Arcadia 方法由法国泰勒斯 THALES 开发的基于 SysML 扩展的 MBSE 方法，用于软、硬件和系统架构设计。Arcadia 系统设计是在不同的工程视角上构建的一种建模方法，主要包括运行分析、系统分析、逻辑架构设计和物理架构设计四大流程，并支持终端产品结构分解功能以开展后续开发设计^[31, 33]。

1.2.2.2 MBSE 建模语言

建模语言提供了一种建模规范，是落地 MBSE 方法的关键驱动，包括图形化建模语言、文本式建模语言。一般而言，图像化建模语言在能够形象地表示对象及其关系，覆盖范围广，学习成本低，但是他们缺少形式化的基础，属于半形式化的建模语言，对于模型的描述不够精确，无法实现对模型的评估和管理^[43]。目前，最常用的建模语言是图像化建模语言：系统建模语言（SysML）^[39]和统一建模语言（UML）^[40]。SysML 和 UML 都是由图形符号和信息模型共同组成的，能够形象、直观地描述系统。

文本式建模语言具备一定的形式化基础，扩展性更高，能够支持模型的形式化描述，但是其学习成本较高，表达不够形象。架构分析和设计语言（The Architecture

Analysis & Design Language , AADL) 是一种文本语言^[15], 同时支持简单的图形化表示, 这是用来对嵌入式实时系统的软件和硬件进行标准化架构建模。EAST-ADL 是一种只应用于汽车硬软件系统的模型语言, 辅助于汽车嵌入式系统设计^[35]。KARMA 语言^[44]也是一种文本式语言, 它面向多架构模型, 从元模型的层次支持不同架构的统一描述, 即: 能够统一地形式化描述多种建模语言构建的异构模型。

1.2.2.3 MBSE 建模工具

建模工具是支撑模型建立、系统分析、仿真验证、集成设计的平台^[25]。国内外在 MBSE 领域内的全生命周期一体化建模开展了大量的研究工作, 各国专家构建了许多不同领域内的 MBSE 软件生态。

IBM 为支持 HARMONY-SE 理论, 开发了相应的集成平台。该平台包含^[26]需求管理软件、模型驱动开发集成环境、仿真平台、团队协作平台; 其中的衍生产品 *Designer for Systems Engineers* 可以对行业标准进行抽象并且通过基于模型的模拟和分析及早并持续检验规范及测试^[27]。

Capella 工具是支持 Arcadia 方法论的平台解决方案^[33]。Capella 工具提供的多视点功能能够实现对系统架构的早期分析验证, 实现了语言、工具、方法论的生态搭建。

MagicDraw 则是最常用的面向多种建模语言的建模平台。它是基于 MagicGrid 系统工程方法论的平台, 具有团队协作功能、支持可视化 UML 和 SysML, 并且支持工程师进行设计决策权衡、模型一致性检查及需求追溯分析^[47]。

1.2.2.4 使用 MBSE 方法进行系统验证和确认

MBSE 领域内确认和验证的目的是判断系统设计是否符合利益相关者的需求, 以及系统设计是否符合其设计规范^[6], 从而提高设计质量。**MBSE 领域内的验证和确认的手段主要有仿真模拟、状态机仿真、形式化验证。**仿真是指对真实世界的过程或系统随时间变化的动力学模拟^[52]。在航空领域, MBSE 是推进 Radio Aurora Explorer (RAX) CubeSat^[46, 47]任务获得成功的关键实践, 美国航空领域的学者通过扩展 MBSE 框架^[47], 集成多个仿真工具来执行基于 SysML 的行为模型, 从而验证子系统功能和航天器的内部状态, 证明了模拟环境在采集真实卫星运行状态方面的适用性, 评估设计参数与计划可行性。除此以外, MBSE 在 CubeSat 小型卫星项目上的应用 RAX CubeSat 模型描述了能够分析子系统功能和航天器状态的行为模型和参数模型。它由 SysML 模型, Simulink 模型和基于 STK 系统模拟的模型组成^[49]。状态机仿真也是一

种行为仿真, Aloui 等人结合 SysML 和状态机仿真实现群机器人的仿真^[53]。

国内 MBSE 领域内的验证和确认的主要手段为仿真方法、状态机仿真、可执行模型转换方法。MWorks 是由华中科技大学研究团队所推出的一种基于知识统一表达与互联标准的系统智能设计与验证平台^[57], 它采用基于模型的方法全面支撑系统设计, 通过不同层次、不同类型的仿真来验证系统设计。单卓为、郝斐等人主要研究结合 UML 和以 SPIN 为主的状态机仿真方法^[54, 55]。刘伟则面向航天器构建了物理多领域模型 Modelica 的仿真^[57]。的王丹丹等人则构建了 SysML 和 Petri 网之间的映射关系, 支持需求合理性和指标完整性的评价^[56]。

综上所述, MBSE 的建模方法、工具和语言, 以及验证和确认均有较广范围的应用, 以下问题仍然存在: 1) **特定域建模语言建模平台分散, 验证和确认方法重用难。**从文献综述可以看出, 建模语言和平台的特定性强, 往往针对某一领域, 并且底层数据分散、配套平台关联性强。此外, 验证和确认手段往往只应用于某种建模语言, 难以应用于不同建模语言。如果想要构建一个涵盖复杂系统全部子系统的全生命周期系统, 单一的建模语言和验证确认方法可能难以支撑复杂系统的设计与验证。2) **验证与确认方式以动态仿真为主, 验证和确认的对象集中在有限的动态过程。**验证和确认方法的主流为集成 simulink、有限状态机等动态仿真方法, 仿真方法只适用于动态过程, 而难以验证一些静态需求, 如: 成本、布局合理性等。另外, 仿真方法只能针对有限状态进行仿真, 对于多状态的情况, 仿真的时间成本较高。因此, 面向动态过程的仿真手段难以应对多级验证和确认挑战。

1.2.3 指标验证

指标验证是一种形式化验证方法, 它能够使用基于数学逻辑的方式规约系统, 并验证系统是否满足预期的属性或性质^[79], 评估系统设计的质量^[70], 从而尽早发现设计中的问题。设计指标是评估系统设计质量的目标属性或特征^[32, 36]。指标来源于需求, 是评估需求是否实现的关键特征描述^[48], 例如, 某需求是“各级部件满足可装配性”, 那么“可装配性”即是指标, 若可装配性指标细化为“各级部件之间满足装配间隙 0.1mm”, 则“装配间隙”是指标。可以看出, 满足指标是保证设计质量的必要条件。指标验证作为形式化验证方法弥补模拟仿真方法的不足。它以验证静态对象为主, 能够使用数学方法缩小状态空间, 对于多状态下的验证有一定的优势^[90]。

1.2.3.1 指标体系的构建

指标体系是衡量系统设计质量的系列标准，指标体系内的指标为指标验证提供了验证对象。设计指标体系指的是能够反映系统设计质量的一系列相对独立但又相互关联的指标所形成的整体^[32]。目前，定义设计指标体系的相关方法尚不完备，大部分指标较为分散，无法形成体系。从满足需求的角度出发，杨伯军等人提出结合产品不同阶段的生命周期，考量产品的环境性、经济性、社会性和技术性，从而定义产品的指标体系^[37]。贾启蒙、沈聘等人以汽车设计为研究对象，提出分层指标构建的方法，先从法律法规、客户需求等主客观因素出发，再层层细化需求，最终形成包含四层指标的指标体系^[38]。从系统本身角度出发，门星火、付广胜等人认为应该从系统的完整性、一致性、合理性出发，再依次对指标进行细化，构建具备层次结构的指标体系^[36, 51]。上述研究都提出应该构建层次化的指标体系，为指标体系的纵向维度提供了思路，并且在指标的横向范围内提出了部分考虑维度。但是缺乏定义指标层次的具体标准。

《NASA 系统工程手册》一书构建了效能测度指标（Measures of Effectiveness, MOE）、性能测度指标（Measures of Performance, MOP）、技术测度指标（Technical Performance Measures, TPM）三位一体的指标体系结构，提出明确的指标层次划分依据。效能指标指的是从利益相关者的角度定义产品的实现情况，性能指标是系统运行相关的物理或功能指标，技术指标是度量技术需求相关的指标^[50]。指标体系依据“MOE-MOP-TPM”的顺序依次分解，形成指标的纵向维度。该方法将每层指标映射至系统架构，符合正向设计的逻辑顺序。大部分研究将这一指标体系直接作为验证与确认的对象^[48]。Ryan 则以澳大利亚国防体系为例，在上述体系之外，增加了适用性指标（Measures of Suitability, MOS），形成 MOE-MOP, MOS-TPM 的指标体系。MOS 主要用于描述系统的作战适用程度，如：可用性、兼用型等。

如表 1.1 指标体系构建方法所示，现有指标体系定义方法的维度和分层方法较多。然而，仍然缺乏一种面向系统正向设计的指标体系，能够和系统架构形成映射，同时在具备明确的分层依据的同时，定义每层指标的维度。

表 1.1 指标体系构建方法

开发者	指标考虑维度	指标分层方法	是否细化至物理单元
NASA, 2013	性能	MOE-MOP-TPM 分层方法	是
Ryan, 2014	性能、可用性	MOE-MOP/MOS-TPM 分层方法	是

续表

门星火, 2016	系统的完整性、一致性、合理性	无	是
贾启蒙, 2020	主客观因素	基于整车参数性能—主客观评价—逐层细分	是
杨伯军, 2022	环境性、经济性、社会性、技术性	产品的生命周期	否

1.2.3.2 指标验证方法

指标验证属于 MBSE 方法内的验证与确认环节,是指通过形式化的方法规范和评估指标和系统架构之间的可满足关系^[70,71],从而明确需求是否得到实现。指标验证的主要优势有: **1) 无需依赖输入, 验证系统的预定义规则。**指标验证来源于形式化手段,而形式化验证方法是一种静态验证手段,不依赖于输入^[61],所以本文的研究对象均为静态指标,即:不随时间变化而变化的指标。**2) 不受设计阶段的限制。**指标验证对系统设计的级别没有要求,无论是概念设计还是详细设计,它都可以检验复杂系统不同阶段的设计缺陷,从而实现验证前置,大幅提升设计效率。目前指标验证主要应用于静态指标的确认,以及基于静态指标确认的多设计选项方案确认问题^[34]。

根据验证与确认的不同目的,实践指标验证的方法名有所不同,但其技术路径都是采用形式化验证的手段: 1) 描述预期实现的属性或特征; 2) 描述现有系统; 3) 验证现有系统和预期之间是否存在冲突。Brucker^[74]于 2008 年率先提出扩展统一建模语言(UML)功能,与对象约束语言(OCL)^[73]结合验证约束的不一致性。作为一种半形式化的元模型,UML 模型只有完整的语法结构,但其语义仍然沿用自然语言,计算机无法识别。半形式化建模语言缺乏对系统层次性和严格实时性的建模方法,难以对静态模型进行分析,无法对系统展开自动推理和证明。OCL 是一种制定约束和查询的文本语言,拥有良好的数学背景,具有一阶逻辑表达式的处理能力,可以用于构建约束、声明断言、属性建模。Martin Gogolla 于 2012 年对 UML 和 OCL 语言中的集合、包、有序集合和序列进行研究,构建两者的关系逻辑,将此类模型转化为布尔可满足问题(SAT)进行验证。这样模型关系就转化为以 OCL 语言体现的逻辑公式,实现检查模型的某些预期属性,这一集成方法已被运用在 UML 的规范环境中。Perez 于 2019 年提出了结合 UML 和 OCL 的改进办法^[75],采用基于约束逻辑编程的范式进行结合

OCL 的 UML 模型的推理和设计方案确认。该方案用于检查模型构建的正确性、模型的严谨性和领域内更符合设计需求的模型方案。

除了结合形式化求解器的方式, Kolovos 提出 EVL (Epsilon Validation Language) 语言^[77], 该语言是模型管理语言 Epsilon 的分支语言, 不附属于任意一种建模语言。它融合 OCL 的语法和语义, 构建自身的语言和语义, 完成对模型的评估, 主要应用于模型转换一致性的检验^[78]。国内的指标验证研究主要也采用建模语言和形式化语言结合的方法。罗奇鸣使用 OCL 语言结合 UML 类图的方式构建模型, 定义面向 UML 类图的一阶逻辑表达式, 采用 Kodkod 约束求解器验证类图的约束独立性和一致性^[42]。刘征等人构建 UML 和 NuSMV 约束语言的映射关系, 将 UML 转化为可执行的约束语言, 验证车站是否存在死锁^[41]。

除此以外, 指标验证还可以应用于基于静态指标验证的方案确认问题。当系统存在多个备选的设计方案时, 即: 存在设计空间时, 指标验证能够根据指标构建相应的约束, 再根据已知约束对每个设计方案进行评估, 从而排除无法达到指标要求的方案, 选出符合要求的方案。Martin Gogolla 教授和 Perez 在该领域同样有所研究^[34, 75], 他们均面向 UML 提出 OCL 能够实现模型的验证以及方案确认, 解决满足约束的方案补充和方案确认问题。如表 1.2 所示, 现有指标验证方法难以突破异构模型的验证, 存在模型建立与指标验证数据不一致和理论不一致的问题。

表 1.2 指标验证方法

开发者	语言/工具	支持的模型	验证对象
Brucker, 2008	UML, OCL, Kleene 算法	UML	模型内约束的一致性
罗奇鸣, 2014	OCL, UML, kodkod 求解器	UML	约束的独立性和一致性
Martin Gogolla , 2017	UML, OCL, Alloy 求解器	UML	模型内约束的一致性, 设计方案的补全
刘征, 2018	UML, NuSMV, 求解器	UML	死锁特性
Perez, 2019	UML, OCL, FORMULA 和相应的 求解器	UML	模型正确性, 设计方案 确认
Kolovos, 2018	EVL	Ecore	模型内约束的一致性

1.2.4 存在的问题

根据上述国内外研究现状分析可知, MBSE 结合指标验证功能为应对复杂系统设

计中的多级验证和确认挑战提供了方法。MBSE 方法能够通过模型化的方法实现设计过程的设计、分析、验证和确认。指标验证作为一种形式化方法，能够弥补仿真手段的不足，主要用于解决检验现有方案是否符合指标，和设计方案确认问题。但是 MBSE 领域内指标验证的仍然有以下问题：

- 1) **指标定义不完整：现有指标体系构建方法不具备构建面向复杂系统设计的多维多层指标体系的能力。**目前的指标体系构建方法从主客观需求、性能、环境等方向定义指标体系的维度，从产品生命周期、“效能指标—性能指标—技术指标”等方向定义指标体系的层次。然而，目前缺乏统一方法能够在明确指标层次和维度定义方式的基础上，定义多层多维的耦合指标，同时将指标和复杂系统的各级单元形成映射，不利于面向多种复杂系统设计的方法重用和落地。
- 2) **指标描述精确性：现有图形化建模语言缺乏精确的语义刻画指标及其约束关系，难以形式化表达并验证指标。**MBSE 领域内常用的建模语言是半形式化的，缺乏指标形式化表达和评估的基础。常用的指标验证方法是结合其他约束描述语言与求解器进行求解。这一方法需要建模人员学习至少两种建模语言或求解工具，使用时需要进行数据格式的转换，导致求解结果的不确定性增大，不利于方法的使用与推广。
- 3) **指标验证通用性：在多种建模语言描述的复杂系统情景下，难以实现跨异构语言的指标验证。**大部分指标验证功能只能针对某一类模型进行验证，如：UML 或 SysML。验证方法有一定的特定性，无法在多个领域语言内重用，大大限制了指标验证的应用范围。然而，现如今的复杂系统设计过程集成了众多不同领域的模型，单一的建模语言未必能够支持复杂设备设计过程。所以，面向特定建模语言的指标验证方法无法满足现有系统设计的需求。

1.3 研究意义

针对缺乏面向复杂系统设计的指标体系定义方法、现有建模语言不具备指标形式化和验证的功能、以及现有指标验证方法难以突破跨建模语言验证的问题，本文从复杂系统设计过程出发，结合系统分析过程定义多维多层指标体系，提供指标验证的对象；再结合元建模方法和形式化验证方法，构建形式化描述和验证指标的统一方法；最后，基于理论方法，实现基于统一建模语言的指标验证功能，支持面向异构建模语

言的指标验证。其意义在于：

(1) **构建面向异构建模语言的指标形式化表达和验证的理论方法。**本文提出一种结合元建模方法和形式化验证方法的指标验证方法。该方法基于 GOPPRR-E 元建模方法，支持多种建模语言的形式化描述；采用可满足性模理论检验一阶逻辑表达式的可满足性。所提方法结合两种理论，支持面向异构模型的建模、指标描述和验证的一致性表达，保证了建模和验证的数据一致性，降低了验证结果的不确定性。

(2) **解决工程领域内面向复杂系统设计过程的指标设计和验证的问题。**复杂系统需求复杂、组合复杂、交互复杂。复杂系统从需求到物理方案的过程中集合多种设计和验证工具。设计指标对设计有牵引作用，既来源于指标，又指导系统的设计，并评估设计质量。本文针对目前工程领域内指标体系应用语言和工具缺乏的问题，构建指标验证语言规范和相应的语言编译器，解决工程领域的指标验证问题。

1.4 研究内容和组织架构

本文拟对复杂产品设计的指标形式化表达和验证方法进行研究，具体研究内容从多维多层系统设计指标体系定义方法、系统指标的形式化表达和验证方法、指标验证语言的设计这三个方面进行研究。研究内容和组织架构如下。

1.4.1 研究内容

(1) 多维多层设计指标体系定义方法

针对现有指标体系构建方法不具备构建多维多层设计指标体系的能力的问题，开展面向复杂系统设计的指标体系研究，形成多维多层设计指标体系定义方法。本研究将依据面向 MBSE 的霍尔系统工程方法和 SPIT 系统分析方法，定义指标体系的维度。同时，基于“需求定义—功能架构—逻辑架构—物理架构”的 RFLP 系统分析流程结合“效能指标—性能指标—技术指标”定义指标体系的层次。最后，关联指标的分层依据和维度定义方法，形成多维多层设计指标体系。该部分的研究内容包括：1) 基于霍尔系统工程方法的多维设计指标定义；2) 基于 RFLP 系统分析流程的多层指标定义；3) 面向多层指标基于 SPIT 方法的指标维度细化；4) 面向复杂系统设计过程的多维多层指标体系。

(2) 系统设计指标形式化表达和验证方法

针对目前 MBSE 领域内现有建模语言无法形式化描述并验证指标和系统架构之

间约束关系的问题，研究如何使建模语言支持指标的形式化表达和验证。本研究内容首先研究 GOPPRR-E 元建模方法和可满足性模理论。其次，提出面向模型元素的形式化表达和验证方法。最后，定义指标验证的类型，实现面向元元模型的指标验证。研究内容包括以下几个部分：1) 基于元元模型的元建模方法；2) 可满足性模理论；3) 结合 GOPPRR-E 和可满足性模理论的指标验证方法。

(3) 指标验证语言设计和实现

针对现有指标验证方法难以突破异构模型验证的问题，研究一种支持模型建立、指标形式化表达和验证的统一建模语言。首先，研究 KARMA 语言和 SMT-LIB 语言的语法和语义。其次，扩展 KARMA 语言的语法和语义定义指标验证语言。最后，构建指标验证语言编译器，支持功能实现。本部分内容包括 1) 基于 KARMA 的指标验证语言规划 2) KARMA-指标验证语言研究；3) 指标验证语言的实现。

(4) 案例验证

针对提出的定义指标体系方法和指标形式化与验证的方法，选用典型复杂系统卫星的设计过程为验证对象，明确其设计的建模流程，在此基础上，使用指标验证方法解决面向姿态控制系统设计的逻辑单元方案确认问题和可靠性的验证，从而验证该方法的可行性。

1.4.2 组织架构

本文的论文结构如图 1.1 所示。本文共包含五个章节，各章节安排如下：

第 1 章 绪论，本章从 MBSE 中复杂系统设计指标验证角度，深入分析复杂系统设计特点，系统综述 MBSE、指标验证等方法的相关国内外研究进展，总结模型驱动设计情景中指标验证面临的挑战与问题，以此给出研究思路并建立论文结构。

第 2 章 多维多层系统设计指标体系定义方法，本章从指标体系的纵横两个维度进行研究。指标体系的纵向维度分层根据系统的“需求定义—功能架构—逻辑架构—物理架构”系统分析流程进行研究，形成指标层次划分依据；根据面向 MBSE 的霍尔系统工程中的学科领域和生命周期初步定义指标体系维度，再结合 SPIT 系统分析模型，定义不同层次指标的维度，形成多维多层的指标体系定义方法。

第 3 章 系统设计指标形式化表达和验证，本章首先研究统一建模方法 GOPPRR-E 元建模方法，GOPPRR-E 为跨建模语言的模型元素描述提供了统一形式化表达的理论基础。再研究可满足性模理论，将其作为构建面向模型元素构建一阶逻辑表达式的

理论基础。最后在两种理论的基础上，本章提出结合两种理论的指标形式化表达和验证方法。

第 4 章 指标验证的语言设计和实现，本章首先分析基于 GOPPRR-E 元建模方法的 KARMA 语言，和基于可满足性模理论的 SMT-LIB 语言，明确两种语言的语法和语义。基于 KARMA 语言，扩展语法和语义，构建能实现指标验证功能的 KARMA-指标验证语言。最后，在统一建模平台上构建相应的语言编译器和前端界面，实现指标验证功能。

第 5 章 面向卫星姿态控制系统设计的案例研究，本章以卫星的设计过程为例，构建卫星的建模过程，使用所提指标体系定义方法定义相应的指标体系，使用所提指标验证方法实现系统的逻辑单元设计方案确认，验证姿控系统逻辑架构的可靠性。



图 1.1 研究框架

1.5 本章小结

本章从复杂系统设计过程出发, 总结分析了国内外关于 MBSE 方法和指标验证方法的研究现状, 凝练该领域面临的问题, 确定本文的研究内容和论文结构。

第2章 多维多层系统设计指标体系定义方法

设计指标体系是指面向复杂系统的能够反映设计质量的一套指标。本章针对复杂系统总体设计过程中指标体系构建方法层次不清、考虑局限的问题，研究设计指标体系定义方法。首先根据霍尔系统工程方法，明确复杂系统设计指标需要考虑的因素，定义指标体系的初始维度。在此基础上，依据“需求定义—功能架构—逻辑架构—物理架构”的系统分析流程，分析每层架构的需求和对应指标。最后，结合系统的分类要素：社会层、系统层、信息层、技术层，细化各层指标考虑的维度因素，形成多维多层设计指标体系。

2.1 基于霍尔系统工程方法初步定义多维多层指标体系

霍尔系统工程^[80]方法是解决大型工程系统规划、组织和管理问题的一种方法，它主要从时间维、逻辑维和知识维 3 个维度构建问题解决方案。本章将该理论作为定义设计指标的横向维度的出发点，从霍尔系统工程的角度定义指标体系的总体横向维度，即：指标体系构建需要考虑哪些方面的因素。

2.1.1 霍尔系统工程

霍尔系统工程^[80]是将复杂工程问题的研究划分为三个维度：时间维、逻辑维和知识维。如图 2.1 传统霍尔三维结构所示。三维的具体含义如下：

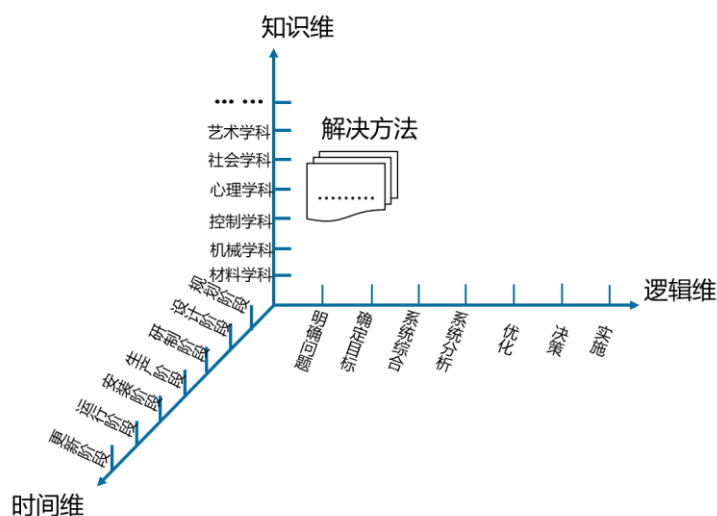


图 2.1 传统霍尔三维结构

- 知识维：完成时间维和逻辑维方案所涉及的学科以及所需要的领域知识和技能的统称。知识维被划为工程、建筑、经济商业、社会科学、艺术、管理、法律和医学八个方面。随着工程的逐步细化，这些学科知识也被细化。
- 时间维：从系统工程活动的全生命周期出发的，划分工程从开始到结束各阶段的任务。传统霍尔系统工程方法的时间维包含以下七个阶段：
 - a) 规划阶段：规划阶段主要提出解决工程问题的基本规划。首先对工程问题进行调研，并在此基础上制定工程活动的总体方针和战略计划。
 - b) 设计阶段：设计阶段是根据规划搭建设计方案的。设计阶段综合多方面知识，根据基本规划提出不同阶段具体的设计方案。
 - c) 研发阶段：研发阶段即根据设计方案提出详细具体的研制方案。该阶段进一步细化规划阶段和设计阶段的结果，完善各级方案的指标，制定相应的生产计划。
 - d) 生产阶段：该阶段实现生成制造。该阶段完成系统零部件的制造生产，完成安装系统的基本条件，并定制安装计划。
 - e) 安装阶段：将系统进行组装。
 - f) 运行阶段：实现系统运行，让系统服务于既定目标。
 - g) 更新阶段：对系统进行分析、评价，根据评价改进现有系统，运行进入下一个周期。
- 逻辑维：逻辑维是解决工程问题时所照的思考方法和工作程序。这种思考方法时应用系统工程方法解决问题的普适性方法，被划分为 7 个步骤。
 - a) 问题定义：明确待解决的问题，或者说定义和阐明该工程所要解决的问题的范围、内容。
 - b) 目标选择（构建指标体系）：确定工程项目解决过程中的分目标以及总体目标，定义相应的评价指标。
 - c) 系统设计和综合：根据需求和目标，构建多种能完成预定任务的方案。系统综合的过程就是研制方案的过程，即找出满足约束条件和预期目标的所有可工选择的系统架构。系统综合的质量决定了优化后方案的质量。
 - d) 分析：系统分析即分析上述所有可选方案的性能，对各个方案在目标的实现程度和目标体系中进行评价，可以通过数学计算、计算仿真或者其他定性定量的方法，为下阶段奠定数据基础。

- e) 系统优化：优化是指根据评价指标，从各个方案中选择一个或多个最优方案或次优方案。选择的方式一般依靠工程预定的约束和目标，或者借助多目标规划方法、运筹学方法等辅助方法帮助决策者评估，明确方案的排序（后者一般用于指标繁多、备选方案构成复杂的状况）。
- f) 决策：决策是在综合、分析、优化的基础上选择一个待实行方案。
- g) 实施：实施即实行最后决策的方案。实施计划不是单一地动作，在遇到问题时需要重新回到相应地上上述步骤，并进一步完善。

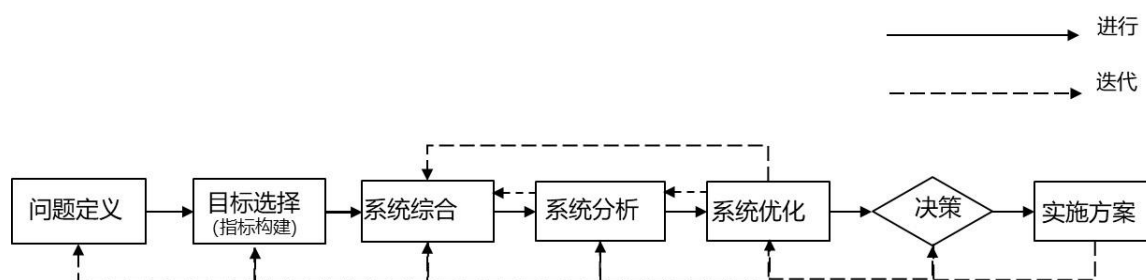


图 2.2 霍尔三维结构逻辑维执行方法

逻辑维的实施过程并非是单一的串行结构，若“分析”过程无法满足需求时就会返回上一步，相似的步骤还有“优化”和“实施”，其逻辑步骤如图 2.2 霍尔三维结构逻辑维执行方法所示^[80]。

然而，提出于 1969 年的霍尔系统工程方法囿于时代的局限性，无法适用于如今的复杂系统设计问题。首先，不同领域的利益相关方之间的知识无法完全保留、传递、使用。由于领域知识的专业性，各自领域的利益相关者无法完全理解其他领域的信息。信息的传递可能会出现不完整或者歧义的情况。那么，利益相关者不能对项目进行统一的认识和管理。其次，异构数据难以重用。工程问题涉及多个学科，在工程问题的探究中还需要借助各类仿真软件、验证方法和规划与运筹方法进行系统的分析、优化和决策。这个过程涉及众多软件，产生大量的异构数据。众多的异构数据则会导致信息理解和重用的障碍，从而影响迭代的效率和质量，无法追溯的方案会大大降低设计效率。最后，传统的霍尔工程针对的是普适性问题，而我们所面对的复杂系统设计问题主要着眼于“设计”过程，所以需要对逻辑维度进行一定的修改。

2.1.2 面向 MBSE 的霍尔系统工程方法

针对上述问题，我们将霍尔系统工程方法进行改进，以适应目前的复杂系统设计。核心的一点是将设计方案定义为模型化的设计架构，设计活动均由模型来体现。

2.1.2.1 基于 V 字研发流程的系统维

系统维是原霍尔系统工程中逻辑维的模型化。将逻辑维度的“问题阐述—目标选择—系统综合—分析—优化—决策—实施计划”更改为适用于 MBSE 开发流程中的“需求定义—功能架构—逻辑架构—物理架构—验证与确认”。其中，架构指的是模型化的解决方案，因为各个阶段的需求和方案在 MBSE 中被模型表示，如：逻辑架构指的是由逻辑单元组成的方案。

“V”模型最初是应用于软件快速开发领域的一个重要模型，它将软件设计过程模块化，同时综合各架构的设计和测试。目前的复杂机电装备借鉴软件领域的“V”模型，构建适用于复杂系统的系统工程开发方法，如图 2.3 所示。和软件开发过程类似，复杂系统的开发也分为三个主要阶段：系统分析、系统开发和系统集成。系统分析对复杂系统的需求定义、功能分析、逻辑架构设计和物理组件规范进行了理论描述，并通过生成的模型进一步说明。系统开发阶段创建复杂系统的开发数据（包括 3D CAD 模型、行为模型等）。系统集成阶段对开发的组件进行模拟和测试，集成到系统中，并进行持续验证和确认。可以发现，指标作为衡量设计质量的标准，从系统分析部分产出。

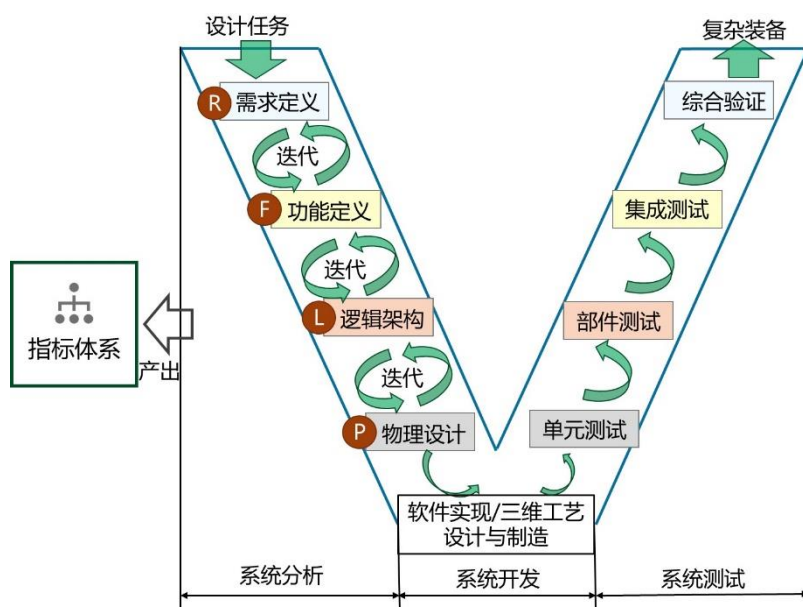


图 2.3 基于 RFLP 流程的 V 字开发过程

应用于复杂系统开发的 V 字研发流程能够全面覆盖霍尔系统工程的“逻辑维”，模型化的“逻辑维”解决了异构信息和无法协同设计的问题。如图 2.3 基于 RFLP 流程所示，RFLP 指的是一种服务于 V 型开发流程的系统分析方法：需求定义—功能架

构—逻辑架构—物理架构（Requirement - Function - Logic - Physics, RFLP），对系统的整体设计、开发和测试提供支持。RFLP 方法的第一阶段为需求定义，包括定义需求和管理需求，实现需求的定义、追溯和确认。RFLP 方法的“F”是功能架构的设计阶段，功能架构的设计是根据设计的规范和需求映射出相应的功能点，生成描述系统为满足需求而提供的能力和服务，以及系统应该执行的任务。功能架构会根据“系统—子系统—部件—零件”的层级细化而细化。“L”是逻辑架构的设计，逻辑架构是物理组件的抽象表示，它从功能架构派生而来，形成能够执行系统功能，而不需要施加技术实现的抽象单元。逻辑架构能够描述系统的动态行为，并且描述各组件的逻辑关系以及实现功能的逻辑原理。系统分析的最后一步“P”为设计物理架构，物理架构的设计定义了具备详细参数的物理设备、及其关系，是实现逻辑架构的具体的、可实现的物理方案。

2.1.2.2 融合 MBSE 方法的霍尔系统工程

面向 MBSE 的霍尔系统工程方法将逻辑维改为系统维，并且重新定义时间维的系统工程方法，如图 2.4 面向 MBSE 的霍尔所示。面向 MBSE 的时间维根据复杂系统全生命周期的特点，将设计和研制阶段统称为生产前的研发阶段，该阶段主要明确设计的详细方案，最终输出的内容包括但不限于 CAD 模型、规格说明等能够用于生产的内容。最后的更新阶段由“淘汰阶段”代替，这是因为复杂系统的更新往往是迭代设计的过程，无法直接在系统上进行更新，而复杂系统的“淘汰阶段”涉及环保、成本等多面问题，是需要着重考虑的阶段。

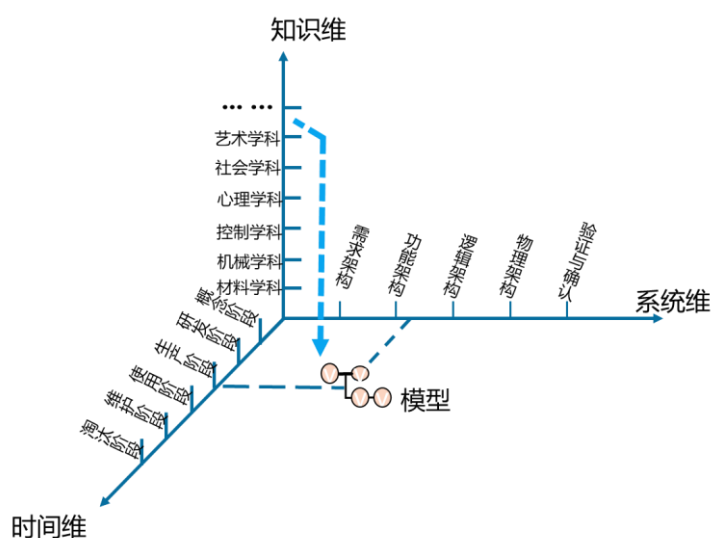


图 2.4 面向 MBSE 的霍尔系统工程

模型是解决方案。在不同的时间维、不同系统维、涉及不同学科知识的领域方案。模型就可以从这三个维度被形式化：

$$Model_{name} = \{\sum Point_{time}, \sum Point_{system}, \sum Knowledge_{subject}\}; \quad (2.1)$$

其中， $Model_{name}$ 是指名为 $name$ 的某一模型， $\sum$ 表达集合，“ $= \{\}$ ”表示一种定义方式， $Point_{time}$ 是指时间维度上的某一节点， $Point_{system}$ 是指系统维度上的某一节点， $\{Knowledge_{subject}\}$ 表示领域知识的集合。其中，系统维的节点和时间维的节点是固定的，可用阿拉伯数字标号表示，如， $Point_{time1}$ 为时间维的阶段 1，即：概念阶段， $Point_{system1}$ 为系统维的阶段 1，即：需求架构，复杂系统设计问题集中在时间维的研发阶段。式 2.1 说明，系统设计过程中的任意模型都能够从这三者中进行定位。

2.2 基于 RFLP 系统分析流程的多层指标体系定义

系统维定义了基于“需求定义—功能架构—逻辑架构—物理架构—验证和确认”的系统研发流程。本章以“需求定义—功能架构—逻辑架构—物理架构”系统分析流程为基础，结合效能指标、性能指标和技术指标，构建多层次指标体系。

2.2.1 RFLP 系统分析流程

本节阐述基于 V 模型开发过程的 RFLP 系统分析流程，如图 2.5 所示。同时，基于该流程定义多层指标体系定义的方法。

首先阐述 RFLP 的基本流程，明确每个阶段的基本步骤：

(1) 需求定义：需求定义只从利益相关人出发，明确任务交付时利益相关人的评估要求。其基本步骤如下：①利益相关人定义：明确系统在整个时间维中，从规划阶段至淘汰阶段过程中涉及的所有利益相关人；②明确利益相关者的顶层期望：通过访谈、调查、问卷等方式明确立意相关者的需求，通常先从任务的战略目标入手逐渐细化。③构建复杂系统的全生命周期用例，明确复杂系统设计过程中的所有方面以及相关的保障和支持。④根据需求，定义衡量任务交付时状态的效能指标。第 4 点即是定义指标的第一步。⑤分配系统功能或人员。明确系统需要完成哪几部分功能，分别涉及哪些需求。⑥根据分系统定义情况细化利益相关者需求。将顶层需求进行分解，确定复杂系统在不同阶段需要解决的问题。

(2) 功能架构：描述分系统或更小级别的结构为满足需求所提供的服务。其基本步骤如下：①场景分析：分析复杂系统可能的运行环境，以及可能出现的状态，剖析

不同场景下的分系统或更小级别的结构的需求。②功能分析：明确满足各类场景需求所提供的功能。可以通过黑盒方法结合物理相应或者定性方法进行分析，明确所有为满足上述需求，该分系统或更小级别的结构能够提供的服务。③功能的权衡和综合：是指评估所有满足需求的服务方案，选择最优或次优的功能架构。并将类似的功能模块进行综合，划分为子功能，进一步探究其逻辑架构。④定义分系统或更小级别的结构在投入环境使用时表现的可量化的性能特征。

（3）逻辑架构设计：逻辑架构设计是定义能够实现子功能的组件，描述功能是如何实现的以及系统的抽象动态变化。逻辑组件是物理组件的抽象表示，它执行系统的功能而不需要施加技术实现的约束。每个功能组可以用一个逻辑组件建模，逻辑组件的名字与功能组的名字相同。①逻辑组件分析：选择并定义能够实现子功能的逻辑元素并进行权衡择优，如果不具有这样的元素那么将子功能进行细化为最小组件，最小功能组件之间互相独立，能够被逻辑元素实现。②定义逻辑架构实现功能情况的技术指标。③逻辑组件的综合：将实现类似功能的逻辑元素进行组合，形成逻辑单元。④定义逻辑架构：描述逻辑单元组成的结构关系、层次关系、逻辑单元状态变化和时序关系。④定义逻辑接口和参数。

（4）物理架构设计：物理架构定义了物理设备及其接口的关系与参数，其目的是描述具体的、可实现的解决方案。①选择并定义物理架构：首先根据技术指标定义物理架构的单元和接口，并进行组合。检查接口的兼容性，定义所有能够实现逻辑功能的物理方案。②定义物理架构的物理参数指标。③物理架构设计空间探索：是否满足物理、逻辑、功能指标，进行运行场景内的评估。④逐步向上验证物理架构。

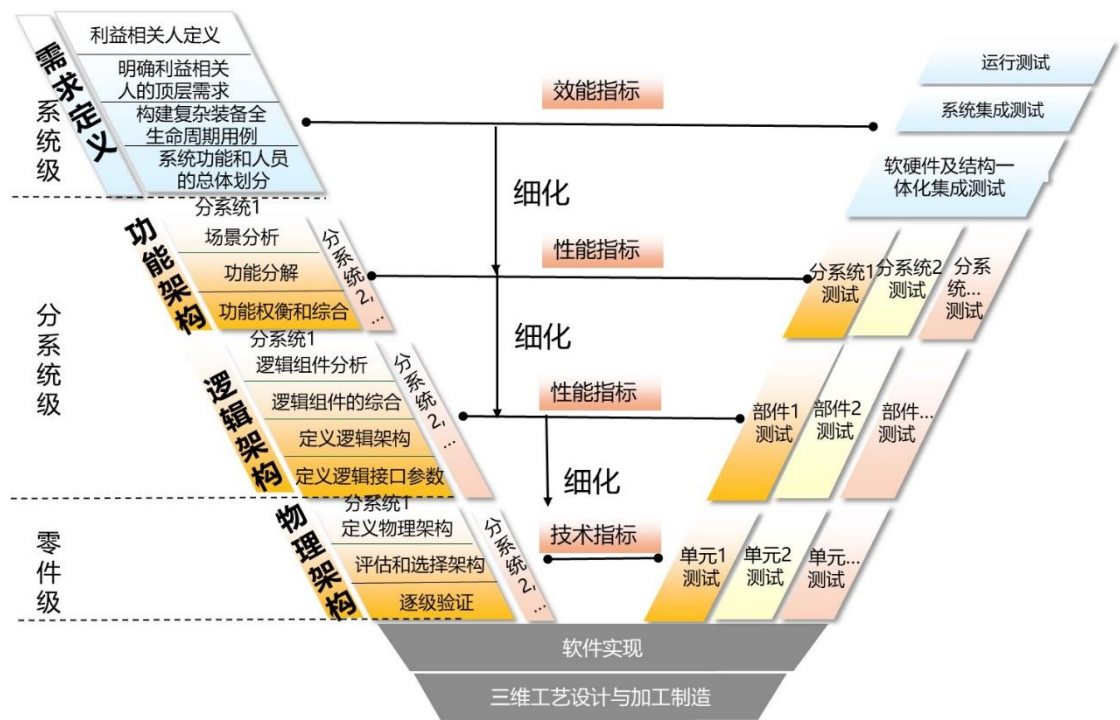


图 2.5 应用于复杂系统设计过程的 V 模型研发流程

图 2.5 展示了在细化 RFLP 流程后的 V 字研发流程。其相较于原 V 字研发流程的特点如下：

（1）现有“V”模型开发流程包含系统级、子系统级、部件级等多级别“V”模型。改进后“V”模型包含多组“V”模型，根据“系统—分系统—部件—零件”的层次，从顶至下均使用 V 字研发流程，也就是系统的“V”模型开发模式中包含多个分系统的 V 模型开发方法。系统、子系统、部件、零件的设计过程均遵循系统分析—系统设计—系统测试过程，保证了系统的设计质量。

（2）系统分析过程包含验证和方案确认的部分。功能架构、逻辑架构和物理设计中都存在评估多种方案和优化的过程。

（3）指标体系的构建依据每个级别（系统级、子系统级、部件级等）的 RFLP 系统分析过程。指标体系来源于系统分析过程，在细化 RFLP 系统分析过程后，可以发现，指标来源于级别的分解（系统级—子系统级—部件级...）和层级的分解（需求定义—功能架构—逻辑架构—物理架构）。

2.2.2 多层指标体系定义方法

根据 RFLP 建模方法，指标体系构建包含效能指标、性能和功能指标，以及技术

指标，其思路如图 2.6 基于 RFLP 系统分析流程定义的多层指标体系所示。

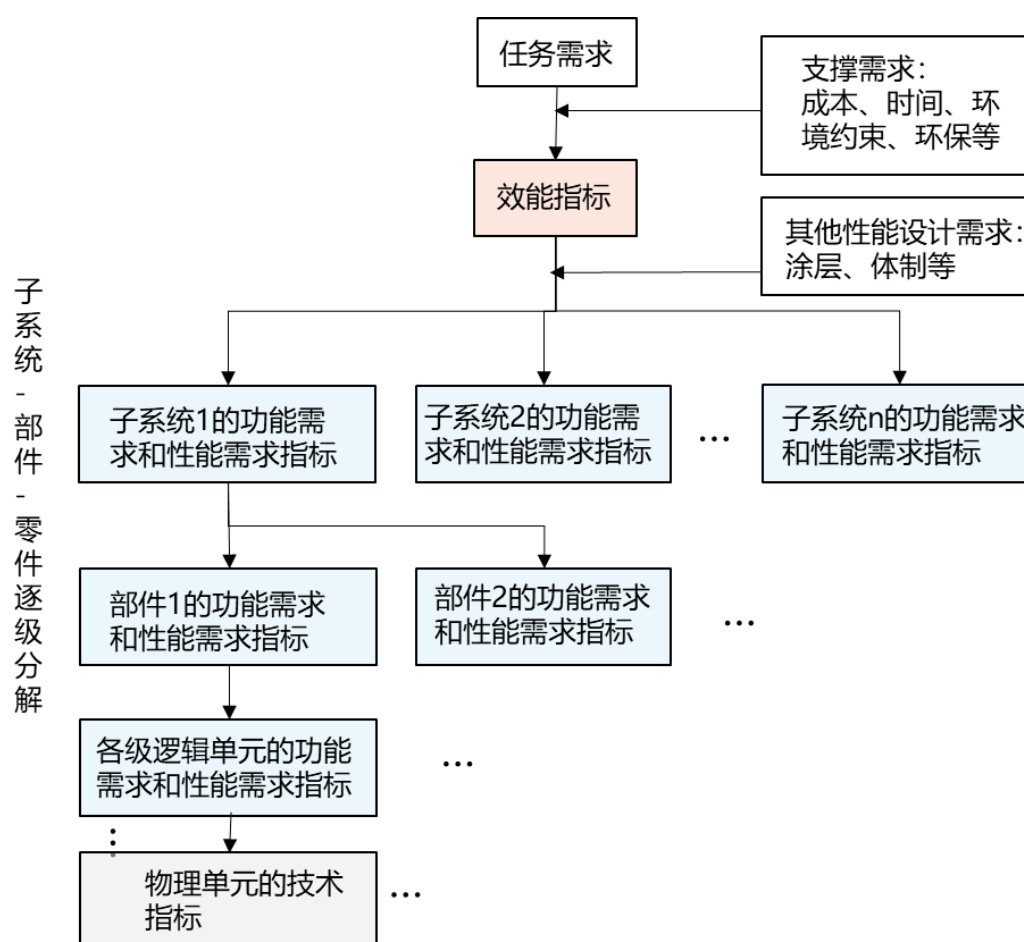


图 2.6 基于 RFLP 系统分析流程定义的多层指标体系

(1) 在需求定义阶段定义针对系统整体的效能指标：从利益相关者的需求中提取出效能指标。效能指标指的是可以直接反应完成任务的情况，明确系统交付时是否满足利益相关人需求的指标。依据需求定义过程中绘制的全生命周期用例，提出其他相关的支撑性效能指标，如：成本、交付、使用寿命、运行周期等。

(2) 在功能架构设计阶段和逻辑架构设计阶段定义针对分系统或更小层级的功能和性能指标：当明确分系统后，功能和效能指标依据整体需求的细分也随之划分，生成多个不同子系统的功能和性能需求指标。功能指标一般指的是能够完成某项功能，即：实现某功能的有效性。这里的性能指标是指能够支撑效能指标实现的描述，表征与系统运行相关的物理或功能属性的度量，如：卫星入轨的功率、地面通信的以太网要求等。逻辑单元是实现功能模块功能的单元。所以，逻辑架构设计阶段也需要将继承功能架构设计阶段的功能和性能指标，并在此基础上细化，形成更微小级别单元的

功能和性能指标。

(3) 在逻辑架构阶段定义子系统技术指标: 各子系统的功能需求和性能需求指标随着系统层次的细化而细化, 生成部件级别的、零件级等其他更微小级别的功能和性能指标。当逻辑单元不能继续细分时, 该单元的功能和性能指标分解为面向物理单元的技术指标, 这些技术指标通常用于衡量实现功能指标、性能指标的物理组件的可实现性, 如实现燃机功率的油箱大小、长宽高、燃烧速率的控制情况等等。

从系统分析过程可以发现, 指标是指导下一层级设计的标准, 指标的验证过程则是成逆向的。物理架构是根据逻辑架构定义的技术指标所定义的, 同时需要验证是否满足前文定义的功能和性能需求指标以及效能指标; 逻辑架构的定义依据功能架构输出的功能与性能指标, 同时需要验证前文的功能和性能需求指标以及效能指标; 功能架构的定义是依据效能指标。

2.3 面向多层指标基于 SPIT 方法的指标维度细化

本节结合“社会层—实践层—信息层—技术层 (Social, Practice, Information, Technic, SPIT)”系统分析方法, 在多层指标体系定义方法的基础上定义不同层次指标需要考量的重点元素, 细化指标体系的横向维度。

2.3.1 SPIT 方法

SPIT 方法为定义指标体系的维度提供了一种更细节的思路。SPIT 方法来源于 Sillitto 提出的对系统共同特征的分层分析方法^[76]。SPIT 方法将超大复杂架构 (Ultra-Large-scale Systems, ULSS) 进行分层分析。它将相同的基本元素进行分组, 并提供一致的抽象级别, 以便理解整个超大复杂系统的共同特征。

SPIT 方法从社会层、实践层、信息层和技术层进行抽象和综合, 其具体的定义和 content 如下:

(1) 社会层: 创建共享的、可理解的模型。从顶至下地决定系统的行为和行动。包含人或组织的理念。

(2) 实践层: 提供政治、战略、实现效果的端到端实践, 这属于系统的过程层, 包含系统的准则。

(3) 信息层: 是指将正确的信息在正确的时间提供给正确的人, 并保证其能够理解和信任, 这属于系统的过程层。

(4) 技术层：能够支持系统研发的、兼容的、可用的、可更新的、可重用的方法，包含设备、设施、方法以及实施逻辑。

2.3.2 基于 SPIT 定义指标体系维度

SPIT 方法将系统构建过程中的必须要素进行抽象，并将类似的要素进行综合表述。复杂系统设计过程的指标体系定义思路已由 2.2.2 节说明。SPIT 方法将细化每层指标的维度，考虑指标定义的方向。将原 SPIT 方法根据复杂系统设计过程进行改进，在指标分层方法的基础上考虑不同层次的分解重点，形成适用于复杂系统设计的 SPIT 方法。



图 2.7 适用于复杂系统设计的 SPIT 方法

如图 2.7 适用于复杂系统设计的 SPIT 方法所示，每层的指标定义维度如下：

(1) 社会层：社会层指标主要用于领域内评估整体方案是否符合不同利益相关人的需求，社会层指标主要来源于各个利益相关人或组织，和利益相关人的要解决的问题相关，属于效能指标。

社会层指标，或者说效能指标，主要用于评估和利益相关人相关的顶层需求。效能指标的定义沿用面向 MBSE 方法的霍尔系统工程方法，从全生命周期阶段和不同知识领域的交叉点获取，如图 2.8 社会层指标定义方法中的交叉点所示。效能指标可以采用“领域知识×生命周期”的方式定义，如：系统使用阶段的监管指标、系统回收阶段的环保指标、系统生产阶段的成本指标等。知识维的领域可以根据需求细化，图中的折线部分表示省略部分，细化知识维如：经济领域分解为成本控制、投资商范围

等。

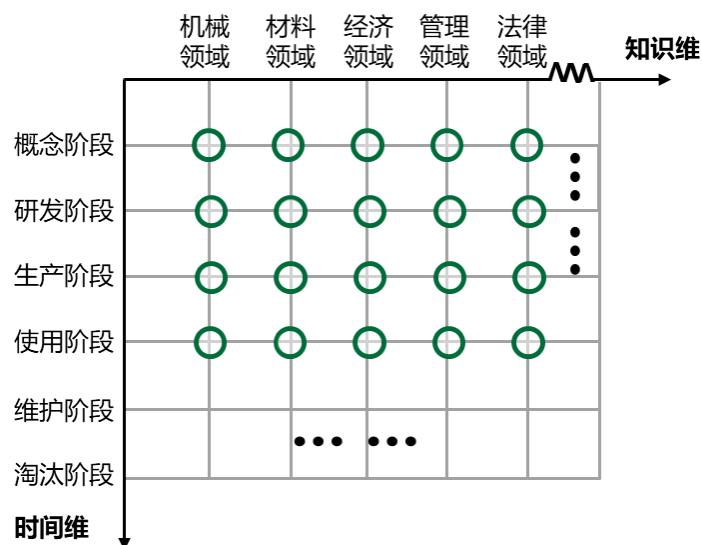


图 2.8 社会层指标定义方法

（2）实践层：实践层指标主要反应各级系统架构中功能和性能的实现情况，属于功能和性能指标。实践层是保障目前的设计架构能够实现效能指标的关键，主要包含两个方面：1）性能指标，评估功能是否实现以及实现情况的指标，如：表征姿态控制精度的精度指标、表征卫星在姿态异常情况下处理紧急状态的应急能力指标等。2）模型构建，保证现有的模型架构是按照正确的方式构建的，如：表征模型构建是否完备的完备性指标、表征模型构建方式是否正确的正确性指标等。实践层的指标贯穿在功能架构、逻辑架构和物理架构之间。

（3）信息层：信息层指标评估数据信息知识的一致性，也属于功能和性能指标。信息层关注的主要是信息传递过程中的一致性和完整性，如：表征指标体系构建过程的追溯性指标、信息传递过程的一致性指标。信息层的关注贯穿在各级模型信息的传递之间。

（4）技术层：技术层指标评估研发过程是否符合技术规格的指标，属于技术指标。技术层指标主要包含物理单元的技术规格，如：星敏器的适用频率，适用角度等。

2.4 面向复杂系统设计过程的多维多层指标体系定义方法

本节在前三节的基础上，构建面向复杂系统设计过程的多维多层指标体系定义方法。如图 2.9 指标体系定义方法所示，本方法以基于 RFLP 系统分析流程的多层指标体系定义方法为基础，添加面向 MBSE 的霍尔系统工程方法和 SPIT 方法扩展指标体

系定义的维度。

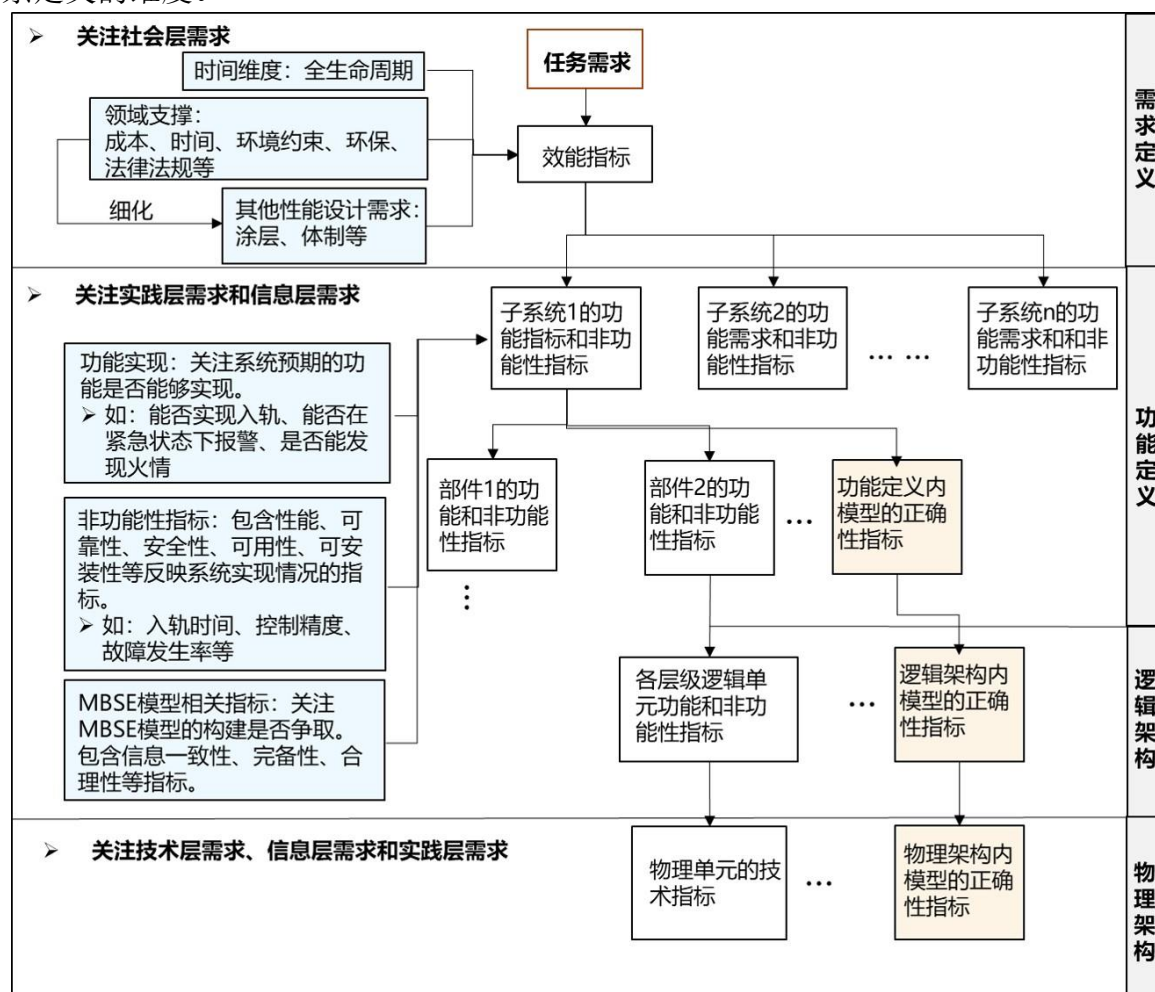


图 2.9 指标体系定义方法

构建面向复杂系统设计过程的设计指标体系定义方法以设计任务需求为输入，以效能指标、非功能和功能指标、以及技术指标组合而成的指标体系为输出。2.2.2 多层指标体系定义方法一节提出基于 RFLP 系统分析流程划分指标层次：在需求定义阶段定义针对系统整体的效能指标，在功能架构和逻辑架构设计阶段定义针对分系统的功能和性能指标，在逻辑架构阶段定义面向物理架构的技术需求指标。

本节在此基础上，融合面向 MBSE 的霍尔系统工程方法和 SPIT 方法，扩充指标体系的维度，形成完整的指标体系。其具体方法如下：

(1) 根据霍尔系统工程方法的三维结构，定义效能指标。效能指标属于需求定义的一部分，以系统维的“需求定义”阶段为轴，考虑霍尔系统工程的时间维和知识维，定义效能指标，如图 2.8 示。以卫星设计为例，从霍尔系统工程出发，定义其效能指

标，即：考虑卫星全生命周期涉及的服务以及其他领域相关的需求，如：设计阶段的经济成本控制指标、回收阶段的环保指标、运行阶段的法律法规指标等。

(2) 结合 SPIT 方法定义系统设计的功能指标和非功能指标。本节结合 SPIT 方法，在功能和性能指标的基础上考量其实践层和信息层的需求，扩展多层指标体系的维度，在功能架构和逻辑架构阶段定义功能指标和非功能指标。目前，功能和非功能指标主要包含功能性指标、非功能性指标和模型正确性指标。

- 功能指标评估系统做什么，关注预期的服务能否实现。例如：卫星能否发送信号，能否探测火情等。
- 非功能性指标关注功能的实现情况，评估系统做的怎么样。结合软件开发的非功能指标和复杂系统的特点，非功能指标主要包含：可用性、可安装性、可维护性、安全性、可靠性、经济成本、可实施性、环保要求^[24]。这些指标可以被依次分解。以卫星设计为例：1) 可用性指标用于评估卫星的运行状况，也就是 2.2.2 节的性能指标，如：某大气环境下传输至某地信号的速率等；2) 可安装性指标评估卫星在制造阶段能被合理安装，可被分解为安装间隙；3) 可维护性指标评估卫星产生问题时的维护能力，可被分解为每个功能单元尽可能独立、携带备件、维护的响应时间等；4) 安全性用于评估卫星通讯的安全性，如：应对恶意攻击的能力；5) 可靠性指标评估系统在规定时间和环境下完成预定功能的程度，如：运行寿命、应急能力等；6) 经济成本指标用于评估卫星运行的成本和收益，如：制造成本、投资额度；7) 可实施性主要关注技术的成熟度和应用能力；8) 环保指标主要评估卫星运行造成的污染情况。
- 模型正确性评估建模过程是否符合规范，主要包括信息一致性、完备性和合理性。信息一致性主要来源于信息层。完备性指标指的是模型构建是否完善，合理性指标指的是模型构建是否依据建模规范。

(3) 定义面向物理架构的技术指标。技术指标反映实现功能和非功能指标的逻辑组件的可实现性，如雷达的传输速率等。最终，得到相应的指标体系。根据上述方法，本章面向卫星姿态控制系统设计得到指标体系，如图 2.10 所示。

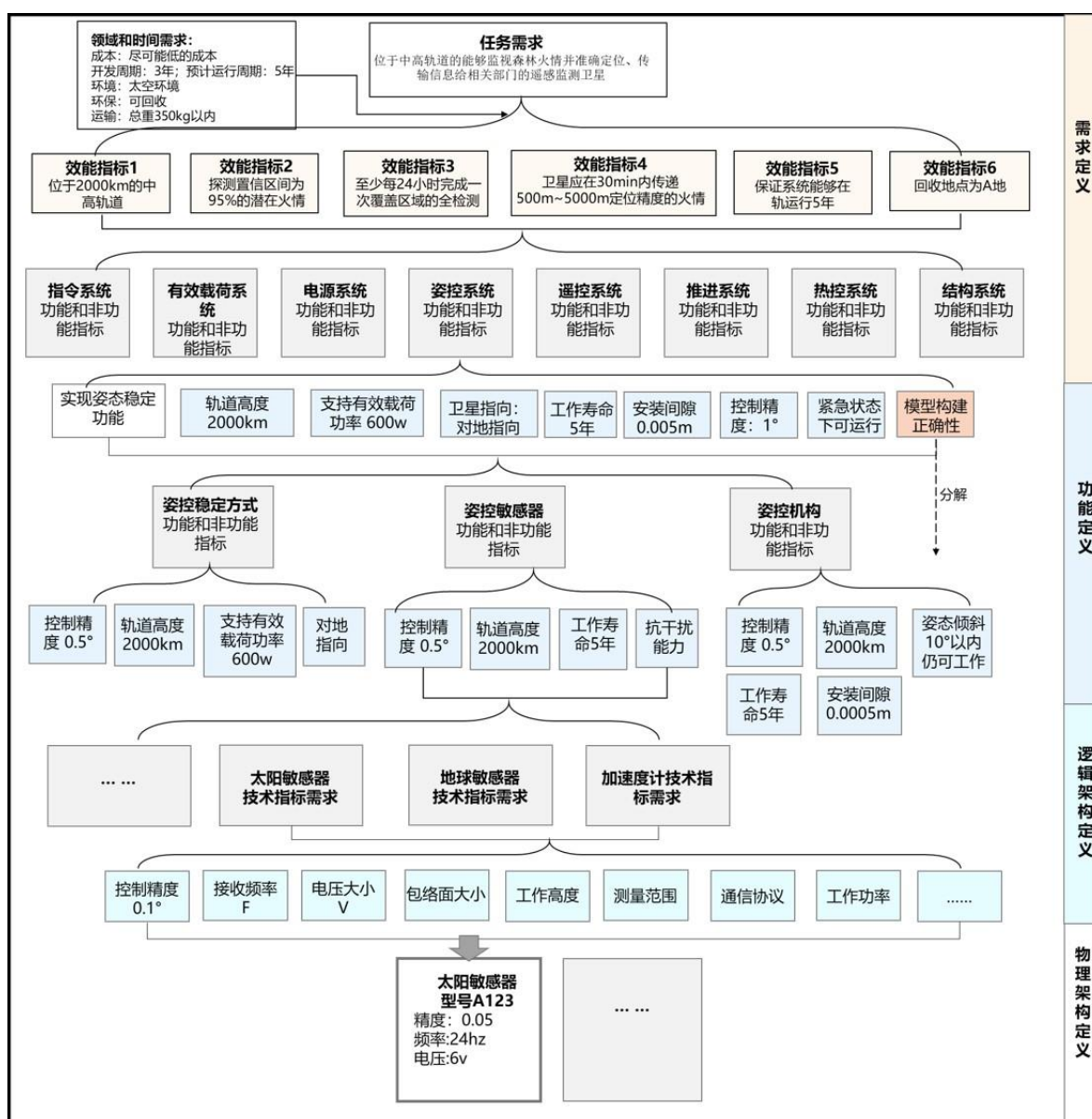


图 2.10 面向卫星设计过程的多维多层指标体系

2.5 本章小结

本章针对指标体系定义不完整, 不具备一种和物理单元关联, 且考虑多维度因素和多层次递进分解的指标定义方法的问题, 提出一种以融合 MBSE 的霍尔系统工程方法为基础, 以 RFLP 系统分析流程细化指标体系分层依据, 结合 SPIT 分层方法定义多维多层指标体系构建方法。该方法相较于其他指标体系定义方法, 完善了指标体系的纵向分层维度, 具备从系统至零件、从需求至物理架构的正向分层过程; 明确了指

标体系的横向维度，定义其从领域至生命周期，从实践至技术的指标范围，构建指标和系统各级单元的映射，保证指标体系构建的完备性和可重用性。

第3章 系统设计指标形式化表达与验证方法

本章针对现有建模语言缺乏精确语义，难以使用统一方法构建模型并进行指标验证、无法保证建模和指标验证数据一致的问题，研究如何形式化描述并验证异构模型和指标之间的约束关系。首先研究分析 GOPPRR-E 元建模方法，明确其在形式化描述异构模型方面的优越性，以及构建模型约束方面的局限性。其次，研究可满足性模理论描述和推理一阶逻辑表达式的能力。最后，融合可满足性模理论和 GOPPRR-E 元建模方法，构建可满足性模理论和 GOPPRR-E 元建模方法之间的映射关系，定义包含静态指标验证和设计方案确认的指标验证类型，形成面向元元模型的指标约束构建和验证方法。

3.1 基于元元模型的元建模方法

GOPPRR-E (Graph, Object, Point, Property, Relationship, Role, Extension) 元建模方法是用于描述异构模型元素和模型关系的基础理论。异构模型指的是不同领域建模语言构建的模型，它们的建模数据不一致。复杂系统往往涉及不同的学科领域、复杂的层次结构，不同领域、层次的异构模型之间具备复杂的交互关系。异构模型由异构的元模型构建，元模型的多样性造成数据传递和统一理解的困难、模型互操作之间的困难，元模型是组成建模语言的基础。基于元元模型的元建模方法是用于解决建模语言异构的问题。本文采用 GOPPRR-E 元建模方法形式化描述多领域建模语言，本节主要介绍 GOPPRR-E 元建模方法的内容和优势。

3.1.1 GOPPRR-E 元建模方法

本文采用 GOPPRR-E 元建模方法作为形式化描述异构模型的基础理论，本节主要介绍 GOPPRR-E 元建模方法的基本元素、建模方法及其形式化表达方法。

3.1.1.1 GOPPRR-E 元元模型基本元素

元模型异构是模型异构的根本原因，元模型是构建建模语言的基础，如：SysML、UML、BPMN、UPDM 和 DML 都是基于元模型进行构建的。GOPPRR-E 元建模方法提取不同领域元模型的共性，整理成六种抽象层次更高的元元模型，支持不同特定域模型的开发。GOPPRR-E 元建模方法的基本元素是六种元元模型：图、对象、属性、

点、对象、角色，以及它们之间的关系，这里所有的元素并非指某个具体的点或图，而是指一种抽象类^[83-85]。其中：

(1) 图 (Graph)：表示对象类、关系类、角色类、点类、角色类的集合，图包含着对象以及对象之间的关系。

(2) 对象 (Relationship)：是指图类中的一个实体（例如，建模语言 UML 中的一个类概念），该实体可以独立于关系或者角色而存在。

(3) 属性 (Property)：是指其他五个元元模型中的属性，它的数据类型通常以数值、布尔、字符串、数组形式存在，且属性实例不能单独存在，必须和其他五种元元模型的实例形成属性连接的关系，属性实例本身不具有属性。

(4) 点 (Point)：是指对象实例的一个端口，如：输入口。点可以被绑定在某个对象实例上，但是并非所有对象实例都需要绑定实例。点可以和角色相连接。

(5) 关系 (Relationship)：是指对象之间，或点之间，或对象和点之间的一种连接；

(6) 角色 (Role)：用于定义映射到相关关系的连接规则，如：输入、输出。角色必须和关系进行绑定，无法单独存在。每个关系实例的两端必须存在角色实例。

扩展概念 (Extension)：GOPPRR-E 元建模方法在 GOPPRR 元建模方法的基础上总结 GOPPRR 元元建模的过程中需要遵循的规律和约束关系，主要包含“包含”、“属性连接”、“剖视”、“分解”、“绑定”和“关联”六种扩展关系。这六种关系是用来描述元元模型之间的关系。这六种关系的具体描述如下。

(1) 包含：包含描述的是元元模型之间的包含关系。如：图包含对象、关系，关系包含点，如式 3.1、3.2 所示，式中 $\in$ 被定义为包含关系。

$$Graph \in \{Object, Relationship, Role, Point, Property\}; \quad (3.1)$$

$$Object \in \{Point\}; \quad (3.2)$$

(2) 剖视：剖视描述某个元元模型实例（对象实例、关系实例、角色实例、点实例）可以映射为多个图的实例。

(3) 分解：分解描述某个元元模型实例（对象实例、关系实例、角色实例、点实例）可以一一映射为图的实例。

(4) 绑定：绑定描述元元模型之间的约束关系，可以用于描述元模型建模之间的规则。例如：一个对象实例化的端点对应多个角色实例，每个角色实例至少对应一个对象实例的端点实例，每个关系实例绑定两个角色实例

(5) 关联：描述对象和角色之间的连接关系，或者对象绑定的点和角色的连接关系。

(6) 属性连接：描述属性实例和其他五种元元模型实例之间的连接关系。

上述六种元元模型可以根据是否包含属性元素分为属性类型 *Property* 和非属性类型 *OtherType* 如。非属性类型的定义如式 3.3。

$$OtherType \in \{Graph, Object, Relationship, Role, Point\}; \quad (3.3)$$

3.1.1.2 GOPPRR-E 元建模方法

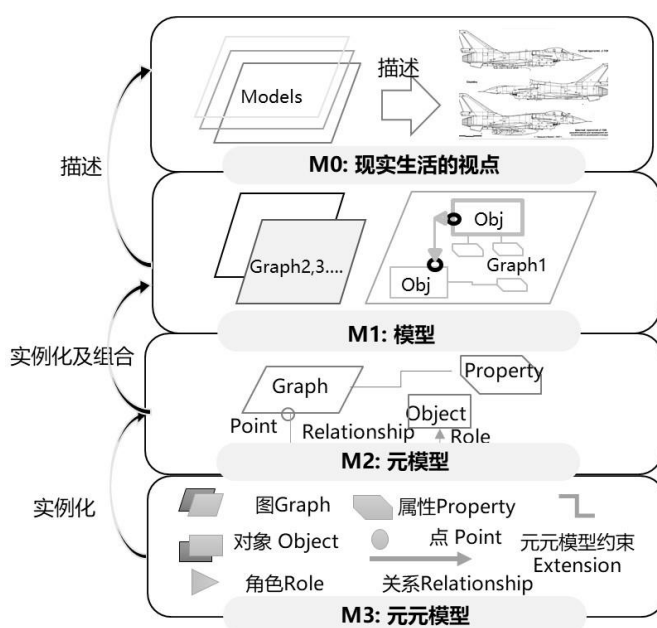


图 3.1 基于 GOPPRR-E 元建模方法的建模方法

基于 GOPPRR-E 的元建模方法如图 3.1 所示。基于 GOPPRR-E 元建模方法的建模方法遵循 OMG 组织提出的元对象建模框架 (Meta-Object Facility, MOF) [102]，依次从元元模型层 (M3 层)，元模型层 (M2)，模型层 (M1)，物理世界视角层 (M0) 进行构建。每一层元素将上层模型元素进行实例化并进行相应的组合。类似的，该层元素的抽象化即下一层元素。其具体描述如下：

(1) 元元模型层即六种元元模型和它们之间的约束关系。

(2) 元模型层是由元元模型的实例化元素组成的，元模型是基于元模型定义的，并且继承了元元模型之间的关系。常见的建模语言是基于元模型定义的，元模型定义了建模语言的概念及其关系，以及建模规则。UML 中的各类图就是一种元模型。这样的元模型用于构造模型，形式化地描述系统的构件和开发过程。在元模型层，图被

描述为 $Graph_{Type}$ ，表示为 $Type$ 类型的图；对象被描述为 $Object_{GraType}^{ObjType}$ ，表示为在 $GraType$ 类型的图里 $ObjType$ 类型的对象；关系被描述为 $Relationship_{GraType}^{RelaType}$ ，表示为在 $GraType$ 类型图下的 $RelaType$ 类型的关系；角色被描述为 $Role_{RelaType}^{RoType}$ ，表示为在 $RelaType$ 类型的关系中 $RoType$ 类型的角色；点被描述为 $Point_{ObjType}^{PoType}$ ，表示为在 $ObjType$ 类型的对象中 $PoType$ 类型的点；属性被描述为 $Property_{otherType}^{ProType}$ ，表示为在其他五种类型的元模型的类型为 $ProType$ 的属性。

(3) 模型层由元模型的实例组合而成，用抽象的方式描述物理世界。图模型的实例化通过 $Graph_{Type}(GraName)$ 表示， $Graph$ 作为图模型的标识符， $Type$ 被表示为图的类型， $GraName$ 指定图的名称。类似的， $Object_{GraType}^{ObjType}(GraName, ObjName)$ ， $Relationship_{GraType}^{RelaType}(GraName, RelaName)$ ， $Role_{RelaType}^{RoType}(RelaName, RoName)$ ， $Point_{ObjType}^{PoType}(ObjName, PoName)$ ， $Property_{otherType}^{ProType}(otherName, ProName)$ 分别指代实例化的对象、实例化的关系、实例化的角色、实例化的点和实例化的属性。以 $Object_{BDD}^{Block}(BDD1, block1)$ 为例，其中 $Block$ 表示该对象的类型，即：SysML 中的模块对象（Block）， BDD 表示对象绑定的图的类型，即：SysML 中的模块定义图，而 $BDD1$ 指的是模块定义图的名称， $block1$ 指的是模块对象的名称。将实例化后的元模型组合，形成模型。

(4) 物理世界的真实视角则是通过模型进行组合，描述利益相关人的视角。

3.1.2 GOPPRR-E 元建模方法的优势

GOPPRR-E 元建模方法并非唯一的元建模方法，之所以选择这一建模理论在于其描述多领域模型的优势。GOPPRR-E 是在 GOPPRR 建模理论的基础上抽象出了建模的规则形成的理论，其基础仍是 GOPPRR 元建模方法。Kern 于 2014 年将多种元建模方法进行分析，包含元建模的概念、关系约束的覆盖程度以及元元建模的模块化和重用三个方面^[83]。分析的对象包含 ARIS, Ecore, GOPPRR, GME, MS DSL TOOLS 以及 MS Visio 这六种支持元建模的工具或方法。

ARIS (The Architecture of Integrated Information Systems) 是一种大型建模方法。用户可以调整现有的建模语言，开发者可以创造全新的建模语言。Ecore 是基于 Eclipse 建模框架的元元模型。Ecore 和操作工具的联系极为紧密，Ecore 是许多模型管理工具和语言的基础，如：EVL。GME (The Generic Modeling Environment) 主要是电气工程领域的特定领域的元元建模工具，具有基于元元模型的语言定义。MS DSL 是微软

开发的基于特定域的元建模语言，支持 Visual Studio 的语言定义和生成。MS Visio 是通用的建模和数据可视化工具，虽然它没有具体的元元模型定义，但是可以从它的众多语言模板中提取出它的元元模型。

表 3.1 元建模方法比较

建模方法	支持的元元模型种类	支持的模型关系
ARIS	4 种（除角色和点）	2 种（支持二进制对象关系和 n 元对象关系）
Ecore	3 种（除角色、点和模型）	1 种（支持引用关系）
GOPPRR	6 种	5 种
GME	6 种	3 种（支持二进制对象关系、n 元对象关系和角色连接）
MS DSL	4 种（除点和模型）	2 种（支持 n 元对象关系和角色连接）
MS visio	3 种（除角色、点和模型）	无

元建模方法如表 3.1 所示。从元元模型的概念出发，对象类型、关系类型和属性是基本的元建模概念，每个元建模方法都支持这些概念。此外，GOPPRR 元建模方法，GME 建模理论和 MS DSL 建模方法支持角色类的复杂关系实现。ARIS, GOPPRR 和 GME 支持类似模型视图的概念。GOPPRR 和 GME 支持端口点的概念。从该角度，GOPPRR 和 GME 更胜一筹。

从建模关系的定义上看，只有 GOPPRR 这一种建模理论能够支持五种建模关系。这五种建模关系分别是模型间的引用关系、一对一的二进制对象关系、n 元对象关系、集合关系、角色关系。这五种关系分别可以对应 GOPPRR 建模理论中的分解、绑定、剖视、集合和关联。

此外，在元元模型的模块化重用方面，GOPPRR 仍然更胜一筹。所有元元模型均提供了分组模块的概念，如：项目、文件夹、语言、包等，但是只有 GOPPRR, GME, Ecore 和 MS DSL 工具支持类之间的继承关系，包含了属性的重用和约束关系^[83]。

综上所述，GOPPRR 元建模方法是表现力最强的建模方法能够支持多种模型的建立，具有良好的表达能力和可扩展性。GOPPRR-E 是在 GOPPRR 的基础上提取元元模型的建模规则而成的，继承了 GOPPRR 的优势，因此选择此种方法作为形式化异构语言构建的模型的依据。

3.2 可满足性模理论

可满足性模理论 (Satisfiable Modulo Theory, SMT) 是构建基于模型元素的约束表达的关键理论^[86, 88]。GOPPRR-E 元建模方法为异构模型的构建和形式化描述提供了理论基础, 然而, GOPPRR-E 元建模方法只能够形式化模型元素, 却无法对模型元素进行量化评估, 也无法验证。可满足性模理论支持一阶逻辑表达式的评估, 还具有相应的求解工具, 能够支持约束关系可满足性的验证。

3.2.1 可满足性模理论的内涵

可满足性模理论是形式化验证的理论基础之一, 它是包含一系列公理的一组逻辑理论, 可以检验包含数学理论的一阶逻辑表达式是否可满足, 是一种形式化验证手段^[88, 90]。其中, 理论是一组公理, 具有确定可满足性的演绎系统, 即通过理论中的公理能对自定义的命题进行推理判断。可满足性指的是至少存在一组解, 使得自定义的表达式成立。可满足性模理论内的理论定义了一阶逻辑命题的语义 (semantics), 如: 字符串理论、等式理论、算数理论、机器数理论等。

3.2.1.1 形式化验证方法

形式化验证方法通常拥有三个要素: 被验证系统的数学或逻辑模型、形式化的表达方法、验证系统是否满足系统范式的验证方法。其中形式化的表达通常被认为是执行形式化方法的第一个步骤: 形式化规约, 具有形式化规约是建立模型的首要条件。第三个要素通常被称为执行形式化方法的第二步: 形式化验证^[59-61]。

形式化规约^[63]指以严格的数学方式定义语义和语法的语言, 在不同层次上对系统进行抽象, 提取需要关注的系统属性, 对属性进行数学表述, 描述系统的行为^[62, 63]。

形式化验证^[63, 64]是指对自定义形式化规格描述的系统的相关特性进行分析和验证, 以评判系统是否符合设计人员定义的需求。常用的形式化验证方法包括模型检验和定理证明两种方法^[61]:

(1) 定理证明: 定理证明是一种半自动的验证方法, 定理证明方法第一步是利用公理和推理规则对系统进行形式化描述构建逻辑模型^[65]。定理证明方法是从定义公理与推理规则出发, 使用这些形式化描述逐步推导出规范成立的证明过程。

(2) 模型检验: 模型检验是一种完全自动验证方法。不同于定理证明, 模型检验是基于模型的。模型检验将规范形式化表述为逻辑公式, 将系统状态空间和行为抽象并形式化表述为语义模型, 并对程序行为空间进行自动化遍历, 是一种基于有限模型

并检验该模型的期望特性的一种技术^[66]。

定理证明方法的可扩展性强，且无需定义系统输入即可验证系统的预定规则，因此定义证明方法对于多状态下的系统验证更有优势。模型检验方法是基于有限模型的技术，通常用于检验模型的有限状态和行为关系，在验证多状态系统和静态属性方面并无优势。因此，本文选用定理证明方法作为形式化验证的手段，可满足性模理论则是定理证明方法的基础理论之一。

3.2.1.2 可满足性模理论的概念

在计算机科学和数理逻辑中，可满足性模理论是用于判定逻辑公式是否可满足的。这种理论是一阶逻辑和数学背景理论的组合。常用的数学理论包含实数理论、整数理论和各种数据结构（如列表、数组、位向量等）的理论。从应用层面说，可满足性模理论可以看作是解决约束满足问题的一种形式，因而是约束规划的一种形式化方法；从形式层面说，可满足性模理论的实例是一阶逻辑中的一个公式，并带有函数和谓词符号的形式表达^[89,90]。

(1) 可满足性理论与可满足性模理论

可满足性模理论是由可满足性理论（Satisfiable Theory, SAT）扩展得到的，它在可满足性理论的基础上融合多种数学理论，是面向一阶逻辑表达、求解的数学逻辑理论^[89]。这种数学逻辑理论是描述、解决约束满足问题的理论基础。与可满足性理论相比，可满足性模理论有更强的约束表达能力，提供了更丰富的约束建模语言。可满足性理论是通过验证选择变量的真或者假，判断布尔逻辑的公式是否可以真。可满足性模理论的验证对象是一阶逻辑表达式。在大部分时候，需要求解的问题不是简单布尔常量（真或假）的逻辑连接，而是算式、函数以及逻辑量词之间的组合。而一阶逻辑表达式是一种由逻辑连接词、任意数据类型变量、量词、函数和谓词构成，可以描述大部分场景。命题公式相关的定义如下：

● 命题公式

命题公式是由命题常元、命题变元、连接词、否定式和圆括号按照一定逻辑关系构成的形式化表达。命题公式由以下四个原则生成：①每一个命题常元或命题变元都是命题公式；②若 A 是命题公式，则 $(\neg A)$ 是命题公式；③若 A 和 B 都是命题公式，则 $(A \wedge B)$ ， $(A \vee B)$ ， $(A \rightarrow B)$ ， $(A \leftrightarrow B)$ 都是命题公式（ A 和 B 之间的逻辑符号称为连接词）；④当且仅当有限次地应用①、②、③所得到的包含命题常元、命题

变元、联结词和圆括号的符号串是命题公式。命题公式本身并不是命题，没有真值，是一个递归式。

可满足性模理论是面向一阶逻辑表达、求解的理论。可满足性模理论用于判断包含多种理论片段（即限定使用某些谓词常项以及函数）的一阶逻辑公式的可满足性，这里“模”的意义和数论中的“模运算”类似，即在限制的理论下讨论公式的可满足性。

● 一阶逻辑命题

一阶逻辑的表达式包括布尔逻辑的布尔运算，但使用了涉及常量、函数和谓词符号的更复杂表达式，而不是命题变量。一阶逻辑的构造看成一种数学语言，这种语言的语法（Syntax）由符号（Alphabet）和构造法则（Formation rules）组成，语法往往跟随不同的形式化规则有不同的变化，如：当 a, b 都为原子命题时，“ $\text{And}(a, b)$ ”和“ $a \& \& b$ ”可以拥有相同的含义。一阶逻辑的构造法则包括项（Terms）和合成公式（Formulas）。项包括变量符号和函数符号，每个变量都是一个单独的项，每个具有变量的函数也是项。另外，常量属于 0 变量的函数，因此也是项，只有由应用上述规则得到的有限式才是项。公式包括谓词符号（Predicate symbols），等式（Equality），逻辑运算符，量词（Quantifiers）等。符号则括逻辑符号和非逻辑符号，如表 3.3 和表 3.2 所示。

表 3.2 一阶逻辑中的非逻辑符号

符号名称	解释
变量	变量又称个体变元。变量指独立的但抽象的客体。常用 $x, y, z \dots$ 表示。变量的取值范围称为定义域（或称论域）。定义域可以是能够穷尽的有限集合，例： $\{2, 4, 8\}$ 、 $\{\text{红}, \text{橙}, \text{黄}, \text{绿}\}$ ，也可以是没有穷尽的无限集合，如：实数集。
常量	常量又称为个体常元，常用 $a, b, c \dots$ 来表示，和变量一起并称为个体词。 常量是指独立切具体的或特定的客体，例如：车、船、卫星设备。
函数符号	函数符号往往用来描述变量之间的关系。通常使用小写字母来表示，例如： $f, g, h \dots$ 。函数的返回类型一般不是布尔类型，其返回类型丰富，需要提前定义，比如： $f(x)$ 可以表示为“ x 的姐姐的姓名”也可以是“ $x-1$ ”的值。

表 3.2 续表

符号名称	解释
谓词符号	谓词是用来描述变量或常量的性质及其相互关系的词。谓词符号一般使用大写字母来表示。谓词定义的命题代表一个返回布尔值的命题，即：谓词命题非真即假。谓词命题的形式通常为大写字母加一小写字母，如： $G(a)$ ， $P(x,y)$ 。谓词同样根据其代指的性质或关系是具体的还是抽象的分为常元谓词和变元谓词。谓词常元指的是某种独立且具体的关系或性质。比如： $P(x)$ 表示“ x 是否是人”，改命题为常元谓词命题。

表 3.3 一阶逻辑中的逻辑符号

符号名称	解释
括号	$()$ 表示运算优先级。
逻辑量词	$\forall$ （全程量词），指的是表示该指定范围内的全体对象或该指定范围整体的含义的词。 $\exists$ （存在量词）是指在逻辑中表示表示个别或部分含义的词，例如：至少存在一个。
逻辑连词	$\neg$ （非）， $\wedge$ （与）， $\vee$ （或）和其他衍生连词，如：异或等
布尔常量	True（是），false（非）
等号	$=/=$ 表示相等

● 赋值

在逻辑中，命题（包含简单命题和一阶逻辑命题）是一个判断返回布尔值的形式化表达，这个返回值可以为真，也可以是假。一个含有变元的命题的返回值是不确定的。只有当公式中的所有变元被指定成特定的值时，公式才成为命题，这个过程被称为赋值。赋值（Assignment）是指变元映射到常量集合中任一元素的过程。若一个公式在赋值过程中，至少有一种赋值使其命题为真，则称该命题为可满足的（Satisfiable）。相应的，如果该命题无论在何种赋值下，取值都为假，则改命题被称为永假式或矛盾式（Contradictory formula），也就是不可满足（Unsatisfiable）。可满足性指的就是该命题是否是可满足的。可满足和不可满足的定义在本文中保持一致。

3.2.2 可满足性模理论的应用域

可满足性模理论的核心理论主要包含：数组理论、机器数理论、核心布尔理论、

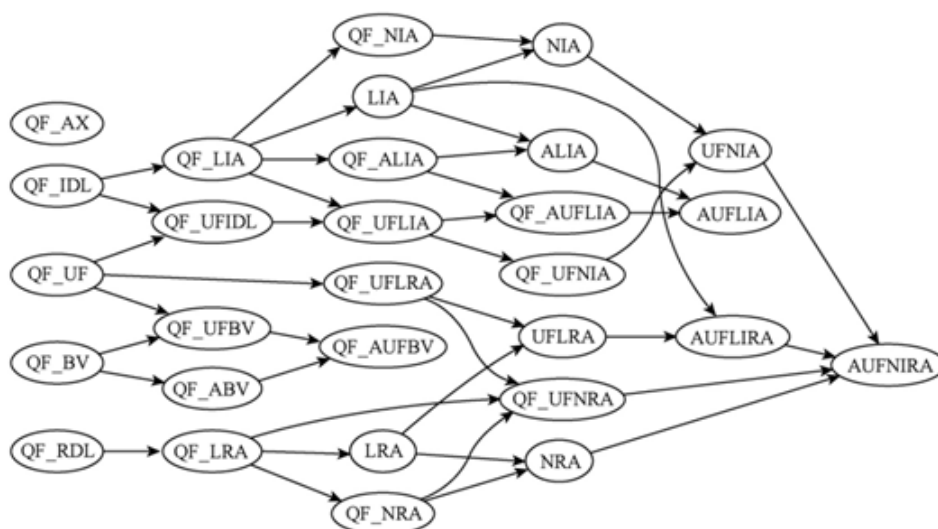


图 3.2 可满足性模理论的应用域

论, “BV”表示固定长度的机器数理论; “FP”表示浮点数理论; “RA”表示实数理论; “IA”表示整数理论; “IRA”表示实数和整数即算数理论; “IDL”表示整数差分理论; “RDL”表示实数差分理论; “L”如果在“IA,RA,IRA”之前中表示线性理论, “N”在“IA,RA,IRA”之前表示非线性理论; “UF”表示未解释函数理论。设下文中各个理论的符号集合为 $\mathfrak{A}_{theory}$, $theory$ 表示理论类型^[91], $\mathfrak{A}$ 表示符号。

3.2.2.1 数组理论

可满足性模理论中数组理论 (Array Theory) 的应用对象是数组 $A(v, d)$ ，它指的是一组一维的有限数据集合。其中 A 表示该数组的名称， v 表示数组存储数据的数据类型，如：整型、实数型、数组型等， d 表示索引的数据类型，索引的数据类型也可以是整型、实数型、数组型等。在该数组 A 中，数据都是一种类型，存在先后顺序，具有相对应的索引。数组理论的符号集合包含“read”和“write”两者， $\mathfrak{A}_{Array} = \{read, write\}$ 。读取 (read) 可以获得数组特定位置的值，写入 (write) 可以在数组特定位置写入值，基本理论定义如下式 3.4 和式 3.5 所示。

$$\forall a, \forall i, \forall v \text{ (read(write}(a, i, v), i) = v \quad (3.4)$$

$$\forall a \forall i \forall j \forall v (i \neq j \rightarrow \text{read}(\text{write}(a, i, v), j) = \text{read}(a, j)) \quad (3.5)$$

$$\forall a \forall b (\forall i (\text{read}(a, i) = \text{read}(b, i)) \rightarrow a = b) \quad (3.6)$$

其中, a, b 为存储任意数据类型数组。 i, j 为数组的索引标识, 通过标识可以获取数组的值。 v 为数组内的存储数据的值, 索引标识和值类型可以为任意数据类型, 包括: 算术类型、布尔类型、字符串、机器数、数组等。式 3.4 体现的是数组的读取理论, 说明首先在数组 a 的 i 处写入值 v , 返回一个新数组, 再提取该数组的 i 处的值, 得到的是值 v 。式 3.5 说明, 当索引 i, j 不相等时, 那么当得到写入 a 数组 i 处 v 值的新数组时, 读取该数组索引 j , 和直接读取原数组 a 的第 j 个值相等。式 3.6 是衍生公理, 该公式说明, 若两个数组内任意位置的值都相等, 则说明这两个数组完全相等。

3.2.2.2 机器数理论

在可满足性模理论的机器数理论 (Bit-vector Theory) 中, 机器数指的是固定长度的位向量^[82]。整数的范围由寄存器的大小决定, 而每个数的值由字节的多少决定。通过简单地将位矢量公式分解为布尔公式, 位矢量理论可以简化为布尔可满足性。机器数理论的符号集包含多种符号, $\mathfrak{M}_{BitVector} = \{\neg, \wedge, <, ==, [], ||, \&\&, \oplus, \ll, \gg, +, -, *, /, \circ\}$, 表 3.4 机器数符号展示了机器数的符合集和意义, 其中, a 和 b 均为机器数。

表 3.4 机器数符号

语法符号	语义	返回的值
$a \wedge b$		
$\neg a$	与一般含义的求和、否定、比较和相等保持一致	布尔值
$a < b$		
$a == b$		
$a[i]$	获取第 i 个字节的布尔值	布尔值
$a b$	针对机器数的每一个字节都进行或运算、与运算、异或运算。	得到和两操作数长度相同的机器数
$a \&\& b$		
$a \oplus b$		
$\sim a$	对每个字节都取反	得到和 a 类型一致的机器数

表 3.4 续表

语法符号	语义	返回的值
$a \ll b$	将 a 中的位向左或向右移动 b 位，分别用零填充。为了对结果进行有意义的定义，我们将 a 和 b 的大小限制为 $\text{size}(a) = 2\text{size}(b)$ ，并且只允许 b 使用无符号类型。这意味着 b 的类型正好覆盖 a 中的数字寻址位范围	得到和 a 类型一致的机器数
$a \gg b$		
$a+b$		
$a-b$	考虑操作数符号的加减乘除的算数运算	机器数
$a \cdot b$		
a/b		
$a \circ b$	a 和 b 的串联，其中 a 填充低位， b 填充高位。	得到长度为 a 和 b 长度之和的机器数
$a[b:c]$	获取 a 位向量的子向量， b 指的是开始的地方， c 指的是长度	长度位 c 的机器数
$c?a:b$	三元运算符，如果原子 c 的计算结果为真，则取 a 的值，否则取 b 的值。	返回为 a 或者 b

例如，假设 x 和 y 是长度为 2 的二进制位向量，即 x, y 可以为 00、01、10、11，若要求符合公式 $x+y=0$ 的解，可以分解 $x+y=0$ ，如式 3.7 和 3.8。

$$X_0 \oplus Y_0 \leftrightarrow \text{false} \quad (3.7)$$

$$(X_1 \oplus Y_1) \oplus (X_1 \cap Y_1) \leftrightarrow \text{false} \quad (3.8)$$

其中 X_0 、 X_1 、 Y_0 、 Y_1 对应于 x 和 y 低位和高位的命题，“xor”是异或运算符。在二进制中，0 和 1 可以表示为“真”或“假”，所以对于机器算式： $x+y=0$ ，可以等价于 x 与 y 相加后，低位与高位的值都为 0 即都为假。可知，式 3.7 的目的是保证 x 和 y 运算后低位为 0。式 1 表明 x 和 y 的低位如果是相同，则经过异或运算后结果为 0，全等于“假”，式 1 可满足，即要求 x 和 y 的低位全为 0 或 1；式 3.8 的目的式保证进位后的高位结构全为 0。若 x 和 y 的低位均为 0，说明高位无需进位，为满足该式为假，则高位经过异或运算也为 0；若 x 和 y 的低位均为 1，说明高位需进位，为满足该式为假，则高位经过异或运算也为 1。结合下表枚举的所有可能性可知，机器算数

可转化为单个位运算的布尔运算。

表 3.5 机器运算的枚举

X_1	Y_1	X_0	Y_0	高位结	低位结
0	0	0	0	0	0
1	0	0	0	1	0
0	0	1	0	0	1
0	1	0	0	1	0
0	0	0	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0
1	0	0	1	1	1
0	1	1	0	1	1
0	0	1	1	0	0
0	1	0	1	1	1
1	1	1	0	0	1
1	0	1	1	0	0
1	1	0	1	0	1
0	1	1	1	0	0
1	1	1	1	1	0

在该例中，本质上是将进位加法器编码为布尔公式。通常，符号常量 b 用于表示位向量，每个符号常量都有一个为该符号固定的相关位宽度。这些理论中的函数和谓词符号可能包括提取、连接、位布尔运算和算术运算。

3.2.2.3 布尔理论

布尔理论（Boolean Theory）是可满足性模理论的核心理论，该理论的对象是布尔值。布尔理论遵循通用的布尔计算方法，布尔理论的符号运算集包含“与”运算符，“或”运算符，“非”运算符，以及等式运算、同或运算、异或运算 $\bowtie_{Boolean} = \{\&\&, ||, \sim, =, \oplus, \odot\}$ 。

3.2.2.4 算术理论

算术理论（Arithmetic Theory）的对象是实数和整型数。算术理论主要包括线性理论（Linear Arithmetic Theory），差分逻辑（Difference Logic），非线性算术理论（Non-linear arithmetic theory）。

(1) 线性理论：也称为加法算法，其符号集合是“+”和“-”。这些函数可以应用于数值常量或变量。除此以外，数值常数与变量相乘也是线性理论的一种。设 $\mathfrak{N}_{num}$ 为算数符号的集合，包含“+”、“-”、“*”、“/”、“ $\leq$ ”及其他等式符号。可以认为线性理论由以下统一的方式构建模型，模型由符号 $\mathfrak{N}_{num}$ 集合定义的谓词和实数组成，如式 3.9 所示。

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n \mathfrak{N}_{conj} C \quad (3.9)$$

这里 $\mathfrak{N}_{conj}$ 可以是等号或者不等号， C 是一个常数， $a_1a_2 \dots a_n$ 是常系数， $x_1x_2 \dots x_n$ 是实数变量（在线性实数演算（Linear Real Arithmetic, LRA）中）或者整数变量（在线性整数演算（Linear Integer Arithmetic, LIA）中）。

(2) 差分逻辑：一般形式如式 3.10 所示。

$$x - y \mathfrak{N}_{conj} C \quad (3.10)$$

x 和 y 可以是实数或者整数。根据 x, y 类型的不同，可以将差分逻辑分为实数差分逻辑（Difference Logic over the Reals, RDL）和整数差分逻辑（Difference Logic over the Integers, IDL）。通过在加权有向图中搜索负环，可以检查不同算术不等式的连接是否满足。如图 3.3 差分不等式的加权有向图所示，每个变量对应一个节点，形式 $x - y \leq c$ 的不等式对应于从 y 到 x 的边，权重为 c 。图中显示了不同不等式和相应图的连接，权重为-1 的负循环用虚线表示。

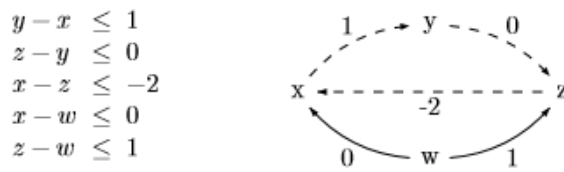


图 3.3 差分不等式的加权有向图

(3) 非线性算数理论：在线性数论基础上发展而来的，其实质是对非线性算子进行四则运算，如式 3.11 所示。

$$a_1y_1 + a_2y_2 + \cdots + a_ny_n \mathfrak{N}_{conj} C \quad (3.11)$$

y_n 表示的是各种非线性运算，如：对数运算（例： $\log_2 y$ 、 $\log_y 2$ ）、开方运算（例： $\sqrt[n]{10}$ 、 $\sqrt[n]{x}$ ）、指数运算（例： y^3 、 y^x ）、三角函数运算（例： $\sin y$ ）。非线性数论的全局优化是不可判定的。不可判定是指没有一种算法可以对非线性算数问题中的任何在有穷步骤内确定是否具有某一特定的性质。所以，可满足性模理论中集成的非线性

算术理论的判定是有限的，无法对所有的非线性问题进行判断。

3.2.2.5 未解释函数理论

在数学逻辑中，未解释函数理论（Uninterrupted Function）又被称为自由函数理论（Free Function），未解释函数的函数符号除了名称和 n 元输入形式外，没有其他属性。所以未解释函数理论的符号集只有未解释函数符号 $\bowtie_{Function} = \{P, Q, R \dots\}$ 。未解释函数是一组空命题公式，形如： $f(a\ b, x\ y\ z); a, b, x, y, z \in N$ ，这表示：名称为 f 的函数包含两个自然数类的输入值，其输出值为三个自然数集中的值，它们之间的关系可以被任意方式定义，例如：声明 $f(1\ 2, 3\ 4\ 5)$ ；这个函数式可以被满足的，但是同时声明 $f(1\ 2, 3\ 4\ 5); f(1\ 2, 4\ 5\ 6)$ ；则是不可被满足的，因为之前已经定义了 $f(a\ b, x\ y\ z); a, b, x, y, z \in N$ 的函数运算方式，即：输入 1、2，会输出 3、4、5。这种函数符号与常量和变量一起构成理论术语。未解释函数理论可以看为谓词形式的发展，许多理论（如：数组理论（Array Theory））的决策问题可以归结为这个未解释函数问题。可以看出，未解释函数理论实际上是谓词的一种体现，如 3.2.1 中可满足性模理论的概念所述。

3.2.2.6 字符串理论

字符串理论在可满足性模理论中是一种由 Unicode 字符组成的字母表上的字符串和正则表达式的理论，目前只支持数字和符号作为字符串的索引。 $\bowtie_{String} = \{++, .len, <, substr, prefix, suffix, index, contain, replace.int\}$ 其中， $str, str1, str2$ 均表示字符串。

表 3.6 字符串理论语法定义

语法符号	语义	返回的值
$str1 ++ str2$	两个字符串单向连接	字符串
$str.len$	求字符串的长度，若字符串不是以 Unicode 的形式定义的，那么它的长度可能会被转义后再计算	整型
$str1 < str2$	比较两字符在 unicode 的顺序大小	布尔值

表 3.6 续表

语法符号	语义	返回的值
	m 是 str 中的一个字符, n 表示字符	
str.substr(m,n)	长度, substr 表示截取 str 中从 m 开始 n 长度的字符串	返回 n 长度的字符串
str.prefix(str1)	判断 str 的前缀是否是 str1	布尔值
str.suffix(str1)	判断 str 的后缀是否是 str1	布尔值
str.contain(str1)	判断 str 是否包含 str1	布尔值
str.index(str1)	判断 str 中首次出现 str1 的位置	整型
str.replace(str1, str2)	将 str 字符串中的 str1 换成 str2, 形成新的字符串	字符串
str.int	将数字的字符串转换为整型	整型

3.2.3 基于可满足性模理论的可满足性求解

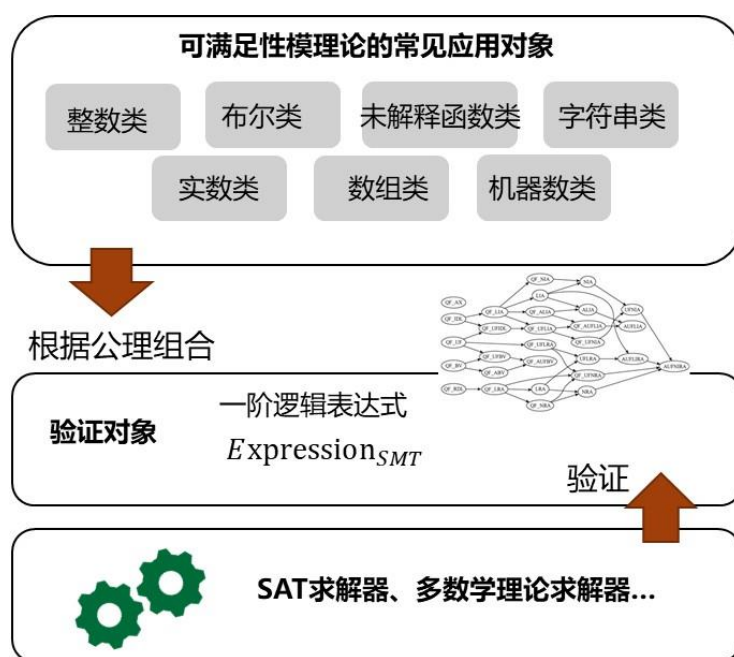


图 3.4 基于可满足性模理论的求解过程

基于可满足性模理论的求解过程如图 3.4 基于可满足性模理论的求解过程所示，基于可满足性模理论的可满足求解就是将上述理论进行组合，形成一阶逻辑表达式，

再通过基于可满足性模理论的求解器对一阶逻辑表达式进行求解。可满足性模理论的应用对象包含整数类、实数类、布尔类、机器数类等等，这些应用对象可以依据公理，进行组合，形成一阶逻辑表达式，也就是验证对象。

3.2.3.1 范式

范式，全称简单范式，是一种规范性的表达方式。对于一阶逻辑表达式的可满足性问题而言，这类问题通常来说求解范围大，不易求解。为了缩小求解域，便于分析问题以及设计实际应用中有效可行的算法，人们对可满足性问题从不同角度设定了约束条件，从而形成了约束子问题。范式（Normal Form）式一种分解一阶逻辑表达式的方法，范式是逻辑等价的标准形式，其结构简单、形式规范，能够将一阶逻辑表达式的可满足性问题分解为多个简单范式的可满足性问题。使用范式进行约束是一种常见的方式，几乎所有的基于可满足性模理论求解器（Solver）都被写作为基于范式的约束子问题，可满足性理论求解器也可以被看作是包含多个范式的布尔方程求解器，这意味着只要输入值可以满足子范式的表达式，那么就有机会有解。

范式是指一个公式中只出现逻辑符号“与”（ $\wedge$ ），“或”（ $\vee$ ），“非”（ $\neg$ ），并且逻辑非（ $\neg$ ）号只加在基本公式前，而不是在与（ $\wedge$ ），或（ $\vee$ ），非（ $\neg$ ）上面。例如式 3.12 是两个典型的范式，其中 X, Y, Z 均为命题公式。

$$X \wedge Y, (\neg X \wedge Y) \vee (X \wedge (\neg Z \vee Y)) \quad (3.12)$$

$$\neg(X \wedge Y), (X \wedge Y) \vee (X \wedge \neg(Z \vee Y)) \quad (3.13)$$

然而，式 3.13 不是范式，因为否定号已经分别前置命题逻辑公式的运算之前。范式一般可以被分为两种：合取范式（Conjunctive Normal Form, CNF），析取范式（Disjunctive Normal Form, DNF）：

- 简单析取式：有限个命题变项及其否定构成的析取式。析取是最常用的逻辑联结词之一，表示“或”的意思。如： $p, \neg q, p \vee \neg q, p \vee q \vee r \dots$
- 简单合取式：有限个命题变项及其否定构成的合取式。合取是最常用的逻辑联结词之一，表示“与”的意思。如： $p, \neg q, p \wedge \neg q, p \wedge q \wedge r \dots$
- 合取范式（CNF）：由有限个简单析取式组成的合取式。也就是指子句内部由逻辑非（ $\neg$ ）或者逻辑或（ $\vee$ ）组成，子句与子句之间由逻辑与（ $\wedge$ ）组合。
- 析取范式（DNF）：由有限个简单合取式组成的析取式。也就是指子句内部由逻辑非（ $\neg$ ）或者逻辑与（ $\wedge$ ）组成，子句与子句之间由逻辑或（ $\vee$ ）组合。

范式拥有相应的定理，这些定理为能够求解出可满足性问题提供了部分数学支持。定理 1 是任何公式均可以找到等效的范式。定理 2 是一个命题逻辑公式均存在等效的、不唯一的合取范式和等效的析取范式。几乎所有解决可满足性问题的求解器都是采用基于合取范式的分解方法，也就是当一阶逻辑表达式的约束输入进求解器后，求解算法会将一阶逻辑表达式分解为多个合取范式，从而找到符合多个合取范式的解。

3.2.3.1 DPLL(T)算法

DPLL(T)算法结构是求解可满足性模问题的核心，它通过通用 DPLL(T)引擎置换不同的求解器 (T) 理论求解器指导 SAT 求解^[89, 92, 93, 97]。其求解对象为一阶逻辑表达式转化的多个合取范式。当求解到相关理论的范式时，DPLL(T)中的 T 替换为某个理论域 T 的 T-Solver 求解器。由于 DPLL(T)结构不局限于特定理论域，而 T-Solver (算法中的置换求解器) 能够判定理论域中由数学理论和命题逻辑构成的表达式，通过合取操作转换为多种范式，验证其可满足性。采用基于 DPLL(T) 算法的可满足性模理论求解器能够保证求解和描述理论的一致性^[93-95]。

3.2.4 可满足性模理论的优势

之所以选择可满足性模理论作为指标量化和验证的理论基础，是因为如下三个原因：1) 可满足性模理论在可满足性理论的基础上融合多种常见的数学理论，其面向对象覆盖布尔值、算数值、机器数、数组、函数等常见理论，可扩展性强，几乎可以覆盖所有指标描述，能支持描述较多应用场景的描述^[97, 101]。2) 可满足性模理论是一种常用的定理证明器的理论基础，定理证明是实现形式化描述和验证的方法之一，能够不加输入的验证系统预定义规则。3) 基于可满足性模理论构建的求解器众多，使用基于可满足性模理论的验证求解器能够保证求解和模型约束描述的理论一致性，减少工具的转化以及计算的不确定性。

3.3 结合 GOPPRR-E 和可满足性模理论的指标验证方法

本节主要研究如何集合 GOPPRR-E 元建模方法和可满足性模理论实现指标验证，并阐述指标验证的形式化表达和不同的指标类型所衍生的不同的验证方法。指标验证类型包含静态指标验证和设计方案确认。上述两节分别说明了 GOPPRR-E 在形式化表达模型元素的优秀表达能力，以及可满足性模理论针对多场景下的一阶逻辑表达式

的表达能力和验证能力。本节将说明如何实现面向模型元素的一阶逻辑表达式的表达和验证。

3.3.1 基于 GOPPRR-E 的指标验证形式化表达

为了实现基于 GOPPRR-E 模型元素的指标验证，支持模型元素的形式化验证，其重点是实现基于 GOPPRR-E 元建模方法的模型元素及其关系以及 SMT 理论中应用对象的映射关系，从而支持基于 GOPPRR-E 模型元素的一阶逻辑表达。

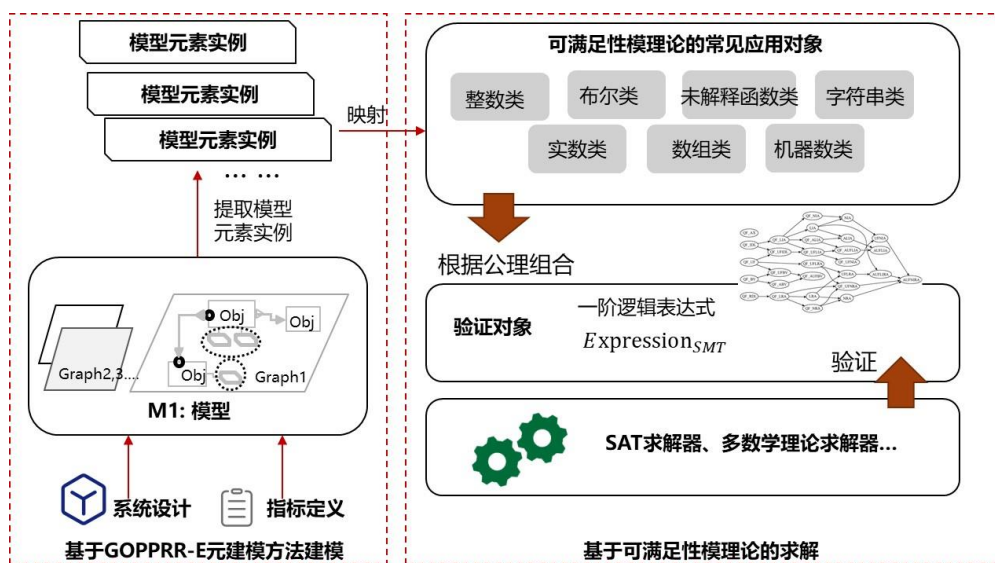


图 3.5 面向异构模型的指标验证方法

如图 3.5 所示的面向异构模型的指标验证方法，首先构建关于指标和现有系统架构的模型，指标一般存在于需求定义的模型，系统架构模型一般是功能、逻辑和物理等架构模型。建模方法基于 GOPPRR-E 元建模方法。再构建面向模型元素的约束，即：使用模型元素组成一阶逻辑表达式的约束形式。已知基于 GOPPRR-E 构建的模型（即模型层，M1 层）中有六种基础元素（图、对象、点、属性、关系、角色）实例化的模型元素实例；可满足性模理论（SMT）包含数组理论、机器数理论、布尔理论、算数理论、未解释函数理论、字符串理论。其中，机器数的操作对象为机器数，为一系列有顺序的布尔值，布尔理论的操作对象为布尔类，算数理论的操作对象为算数类型，包含整数和实数，未解释函数的操作对象为返回为布尔值的空函数，字符串理论的对象为字符，数组理论较为复杂包含所有类型的有顺序的集合。若要构建基于 GOPPRR-E 的一阶逻辑约束表达式，则需要将基于 GOPPRR-E 构建的模型元素之间的关系和可

满足性模理论中的操作对象一一对应，如表 3.7 GOPPRR 模型元素和数学类型的对应关系所示：

表 3.7 GOPPRR 模型元素和数学类型的对应关系

SMT 理论	SMT 理论的操作对象/ GOPPRR 元素返回的类型	对应的 GOPPRR 元素
布尔理论	布尔类	来源于属性，如： $Property_{otherType}^{Boolean}(otherName, ProName)$ 来源于任意模型元素之间的谓词关系 U 如：令角色实例为 Ro ，关系实例为 Re ，令角色和关系之间的规则为关联 $U = C(x, y)$ ，其中， x, y 均为模型元素实例。则角色实例和关系实例的未解释关系可以表示为 $C(Re, Ro)$ ，该未解释函数表明角色和关系之间的关系为关联关系，返回为布尔值。
未解释函数理论	返回为布尔值的谓词函数	
机器数理论	一系列有顺序的布尔值	来源于属性，如： $Property_{otherType}^{BitVector}(otherName, ProName)$
算数理论	整数	来源于属性，如： $Property_{otherType}^{Integer}(otherName, ProName)$ 来源于模型实例的个数，如： $Size(modelElement_{goppr})$
	实数（含小数点）	来源于属性，如： $Property_{otherType}^{Real}(otherName, ProName)$
字符串理论	字符	来源于属性，如： $Property_{otherType}^{String}(otherName, ProName)$
数组理论	一系列有顺序的布尔值或机器数或算数或字符串或数组	数组中存储的数据可以来源于上述方法，其中数组可以来源于属性： 来源于属性，如： $Property_{otherType}^{Array}(otherName, ProName)$

表 3.7 中 $Property_{otherType}^{proType}(otherName, ProName)$ 表示模型中属性实例的形式化表达（定义见 3.1.1.2 GOPPRR-E）。其中， $proType$ 表示属性值的数据类型，可以为

布尔值、机器数、算数、字符串以及数组等, $otherType$ 表示属性连接的其他五种模型元素的类型, 括号内字符分别表示其他五种模型元素的名称以及属性名称。

从表 3.7 GOPPRR 模型元素和数学类型的对应关系中可知, 布尔类、机器数、算数、字符串以及数组都可以来源于属性值。另外, 整数型还可以包括模型实例的个数 $Size(modelElement_{goppr})$, 其中 $Size()$ 方法表示获取实例数量, 获取的整型可以参与进算数理论、数组理论中。同时, 任意模型实例元素之间也可以参与未解释函数理论, 返回布尔值, 参与布尔或机器数理论中。

指标和现有的系统架构均由模型表达, 对两者的模型元素进行量化, 其关系可以表示为一阶逻辑表达式。令指标为 I_{name} , 现在的系统架构为 M_{name} , 判断现有系统架构是否符合指标的约束则是判断两者是否存在冲突, 如式 3.14 所示。

$$I_{name} \wedge M_{name} == \text{true} \quad (3.14)$$

式 3.14 说明了现有系统架构和预期指标之间的约束关系, 将这样的关系定义为模型约束, 如式 3.15。

$$C_{Boolean}^{num} = I_{name} \wedge M_{name} == \text{true} \quad (3.15)$$

其中, $C_{Boolean}^{num}$ 内 C 表示约束, 下标 Boolean 表示约束关系均返回布尔值, $C_{Boolean}^{num}$, I_{name}^{num} , M_{name}^{num} 的上标 num 表示约束、指标、现有系统架构的标号, I_{name}^{num} , M_{name}^{num} 的下标 name 表示指标和系统架构的名称。可以看出, 指标和系统架构之间关系都以基于模型元素的一阶逻辑表达式形式存在的, 两个一阶逻辑表达式之间的布尔关系形成约束仍然为一阶逻辑表达。

结合表 3.7 GOPPRR 模型元素和数学类型的对应关系, 约束 $C_{Boolean}^{num}$ 进一步可以被定义为 3.16 式。

$$\begin{aligned} C_{Boolean}^{num} = & \odot_{SMT} (Property_{otherType}^{ProType}(otherName, ProName), \\ & Size(modelElement_{goppr}), \\ & U(modelElements_{goppr}), \Phi_{SMT}) \end{aligned} \quad (3.16)$$

其中, $\odot_{SMT}$ 表示基于可满足性模理论的运算方法, 支持包含数组理论、机器数理论、核心布尔理论、算术理论、未解释函数理论和字符串理论在内的运算, 支持包含 $\bowtie_{Array}$, $\bowtie_{Bitvector}$, $\bowtie_{Boolean}$, $\bowtie_{num}$, $\bowtie_{conj}$, $\bowtie_{Function}$, $\bowtie_{String}$ 符号集的运算。 Φ_{SMT} 表示可满足性模理论中支持的各种理论的常量集合。 $U(modelElements_{goppr})$ 表示由模型元素定义的未解释函数, 返回值为布尔量。

3.3.2 指标验证的分类及形式化表达

指标验证的对象是静态指标。静态指标指的是不随时间变化的指标，如：大部分结构的有效性指标、可安装性指标成本指标等；与之相对的是动态指标，它随时间变化，如：卫星运行的实时加速度或实时在轨精度。本文所提的指标验证针对的是静态指标。指标验证也可以根据验证的目的分为以下两种应用：1) 判断目前的模型架构是否符合设计指标^[96]，2) 排除设计空间内不满足指标的方案从而生成最佳方案。

3.3.2.1 用于模型评估的指标验证

本文于第 2 章明确了依据“需求定义—功能架构—逻辑架构—物理架构”的指标体系分层方法，依据霍尔系统工程的维度和 SPIT 方法定义指标体系的维度。虽然这些指标的来源繁杂，但它们都可以被模型描述，并且和现有的系统架构模型形成一阶逻辑表达式约束。根据检验指标类型的不同，用于评估的指标验证被分为：局部指标验证、全局指标验证、约束依赖验证。

(1) 局部指标验证

局部指标验证是指确认目前的设计架构是否满足局部指标，局部指标指的是该指标只和设计架构中有限的、独立的元素有关，如：某设计的需求定义是某一电路的入口电压小于 220V，其指标是入口电压，该指标只和电路架构的入端口约束有关。局部指标的评估首先形式化指标，明确指标的值，指标是入口电压 V_{Target}^{in} ，预计值为 220V，构建指标相关的约束 $R_{smt}^{Local}: V_{Target}^{in} < 220$ 。再提取目前模型架构中的信息：入口实际电压 $V_{Reality}^{in}$ 、其属性值 $Value_{Reality}^{in}$ ，构建现架构的约束关系 $M_{smt}^{Local}: V_{Reality}^{in} == Value_{Reality}^{in}$ 。判断现设计架构和指标之间是否存在冲突，构建约束 $C_{Boolean}^{Local}$ 。

$$C_{Boolean}^{Local} = (R_{smt}^{Local} \wedge M_{smt}^{Local} == \text{true}) \quad (3.17)$$

式 3.17 中 R_{smt} 和 M_{smt} 均为基于模型元素的一阶逻辑表达式。上述指标和设计架构均以模型形式存在。式中约束 R_{smt}^{Local} 和 M_{smt}^{Local} 遵循见上方式 3.16 的定义。

遵循式 3.16 的定义方法，在关于入口电压的问题中，预计值 220V 和实际电压值均应以属性形式存在。假设指标预计值 220V 是一对象实例 Req 的整型属性，名为 Target，实际电压为一点实例 Poi 的整型属性，名为 Value，则约束细化定义为式 3.18。

$$C_{Boolean}^{Local} = \left(\left(V_{Target}^{in} < Property_{Req}^{Integer}(V_{Req}, target) \right) \wedge \left(V_{Reality}^{in} == V_{Target}^{in} \right) \wedge \left(V_{Reality}^{in} == Property_{Poi}^{Integer}(V_P, value) \right) \right) == \text{true} \quad (3.18)$$

(2) 全局指标验证

全局指标的验证是指验证目前的系统设计架构是否满足模型中的全局指标，全局指标指的是该指标和架构中一个或多个模型元素相关，不局限于某一个模型元素的关系，如：整个电路模型只能输出高电平，电路的输入电压和电路模型的所有元素及其关系相关。全局指标相关的约束通常不是附加在特定的模型元素之间的关系，而是模型架构之间的逻辑关系，无法被直接提取，需要根据模型架构定义其全局指标的约束关系。

全局指标的验证首先定义全局指标相关的约束关系，如：电路输出为高电平，则电输出电压为指标，预期值为高电平，约束可以构建为： $V_{Target}^{out} == true$ ，此时的电压输出的类型为布尔类型，高电平用“true”表示，低电平用“false”表示；再构建电路输出电压大小和整个电路的关系，即：构建电路模型内各子元件的属性和电路输出之间的关系，如： $V_{Reality}^{out} == M_{smt}^{model}$ ，其中， $V_{Reality}^{out}$ 表示实际输出电压， M_{smt}^{model} 表示模型的逻辑关系，再判断全局指标是否能够被满足。全局指标的表达可以表示为 R_{smt}^{Global} ，相关的属性表达式为 M_{smt}^{global} 。最后构建实际输出电压和目标输出的相等约束。则，全局指标的确认方法可以形式化为式 3.19：

$$C_{Boolean}^{Global} := \forall input, M_{smt}^{global} \wedge R_{smt}^{Global} \wedge (V_{Target}^{out} == \neg V_{Reality}^{out}); \quad (3.19)$$

理论输出：false

针对全局指标的确认，相关指标的表达式不再是针对某一个属性，而是一个模型里的模型元素的逻辑集合。 M_{smt}^{global} 是根据模型内独立的模型元素构成的数学逻辑表达式，定义了全局指标和系统架构内部模型元素之间的关系。当模型元素的属性元素中存在变量时，即模型的属性实例的值不是已经定义的，而是在一个范围内的，这类的属性称为变量型属性，全局指标可以通过取反输出不可满足的情况验证预定义的系统规则。式 3.19 中，电路的输入为变量，则对全局指标进行取反，验证是否存在一种输入令全局关系和取反后的指标相等。若结果为不可满足，则说明该模型不存在让指标不满足的情况，则无论输入值为什么，输出电压始终为高电平。如果表达式中使用全称量词，也可以表示类似的约束定义。式 3.19 也可以通过全称量词写成式 3.20，即：对于任意一种输入，模型关系始终符合指标。

$$C_{Boolean}^{Global} := \forall input, M_{smt}^{global} \wedge R_{smt}^{Global} \wedge (V_{Target}^{out} == V_{Reality}^{out}); \quad (3.20)$$

理论输出: true

在上述整体电路的电压问题中, 约束可以被写为式 3.21:

$$C_{Boolean}^{Global} := (M_{smt}^{model} == V_{Reality}^{out}) == false; \text{理论输出: } false \quad (3.21)$$

其中, 式 3.21 说明约束: 针对任意输入, 模型元素得到的实际输出电压为低电平这一约束恒不成立。上述指标和设计架构均以模型形式存在。式 3.20 和式 3.21 中的约束均为基于模型元素的一阶逻辑表达式, 遵循见上方式 3.16 的定义

(3) 约束的依赖性检验

指标约束依赖性验证通过判断属性之间的约束是否存在依赖关系。虽然属性的依赖性检验不能直接影响模型的有效性, 但是尽可能精简的属性约束能够简化模型, 提高运算速率, 从而提升模型质量。这类指标一般在“技术性需求”中出现。令模型中待检验的相关指标约束为 R_i^{local} , 下标 i 表示约束个数。指标约束的依赖性检验实际上是计算互相独立的指标的个数是否存在差异, 令 *simplify* 方法是用来计算互相独立的约束的个数, *size* 用于计算约束的个数的依赖性检验可以为:

$$num_R = Size(R_1^{local} \wedge R_2^{local} \wedge \dots \wedge R_i^{local} \dots \wedge R_m^{local}) \quad (3.22)$$

$$num_P = Size((R_1^{local} \wedge R_2^{local} \wedge \dots \wedge R_i^{local} \dots \wedge R_m^{local}).simplify) \quad (3.23)$$

$$num_R == num_P \quad (3.24)$$

其中, num_R 表示模型内现存的约束个数。 num_P 表示约简后互相独立的约束的个数, 若属性表达式之间不存在依赖关系, 则实际的约束表达式个数和理论的约束表达式个数应该保持一致。

3.3.2.2 基于指标验证的方案确认

指标验证的实质是评估现有的系统设计架构是否符合预期目标, 它既可以用于评估单一设计方案, 也可以通过评估设计空间内的多个设计方案, 实现设计方案的确认。复杂系统设计涉及大量的设计选择, 譬如: 组件的选择、组件之间的连接、组件的数量、操作模式等^[16]。方案确认是明确设计的必要过程, 它根据设计偏好对不需要、不重要的设计点进行修剪, 进行设计决策。通过指标验证方法, 方案确认可以根据系统的指标评估所有待选的设计方案, 从而排除不符合指标的方案, 选择满足指标、且最接近最优的方案。

设计空间 A 包含多组设计方案 A_i , 每个设计方案均为可能实现目标的解决方案。

如式 3.25 所示。其中， A_i 表示方案空间里的不同方案。

$$A = \{A_1, A_2, \dots, A_i, \dots, A_n\} \quad (3.25)$$

以卫星设计过程的姿态控制系统为例，姿态控制系统是一个整体的设计方案，令其为 A^{ZK} ，设计方案 A^{ZK} 内就包含多个设计方案 A_i^{ZK} 。

每个设计方案是由多个设计选项中的待选项组合而来的。设计选项指的是一组能够实现某种设计功能的待选项集合，如式 3.26 所示。令设计选项为 D_j ，下标表示设计选项的类型，设计选项是集合，集合包含待选项元素。小写字母则表示对应的设计选项集合中的元素，下标 i 说明这些元素是组合成 A_i 方案的特定元素，小写字母的上标表示可取集合中的 0 个到多个元素。式 3.26 说明设计空间内的任一设计方案 A_i 由设计选项中的待选项组合而成。

$$A_i = \{a_i^{0...*}, b_i^{0...*} \dots z_i^{0...*}\}; a \in D_a, b \in D_b \dots j \in D_j \dots z \in D_z \quad (3.26)$$

考虑卫星的姿控系统设计方案空间 A^{ZK} ，包含若干个设计方案，其设计方案均由以下三种设计选项组内的待选项元素组合而成： D_a 姿态稳定方式选项组、 D_b 姿态敏感器设计选项组、 D_c 调姿机构设计选项组。假定 D_a 姿态稳定方式选项组具备三中待选项： α^d 待选项双自旋稳定方法、 α^t 待选项三轴稳定方法、 α^g 待选项重力稳定方法； D_b 姿态敏感器设计选项组包含 6 种敏感器待选项 $b^1 \sim b^6$ ； D_c 调姿机构设计选项组包含 7 种调姿机构 $c^a \sim c^g$ 。则任意一种姿控系统设计方案均从以上三个设计选项组内任意类的待选项组合而成，该设计空间的大小 S_p 可以被计算，如式 3.27 所示。 C_3^1 是因为只能选取一种稳定方式，敏感器和执行机构则可以选择多种，可以看出该设计空间较大。

$$S_p = C_3^1 * (C_6^1 + C_6^2 + C_6^3 + C_6^4 + C_6^5 + C_6^6) * (C_7^1 + C_7^2 + C_7^3 + C_7^4 + C_7^5 + C_7^6 + C_7^7) \quad (3.27)$$

MBSE 领域的待选项由模型构成，则任意待选项 j 由模型实例构成，如式 3.28 所示。待选项可以是属性实例、对象实例、关系实例、角色实例、点实例的一个或多个的组合。

$$\begin{aligned} j \in \{ & \text{Property}_{\text{otherType}}^{\text{proType}}(\text{otherID}_j, \text{proID}_j)^{0...*}, \\ & \text{Object}_{\text{GraType}}^{\text{objType}}(\text{GraID}_j, \text{objID}_j)^{0...*}, \\ & \text{Relationship}_{\text{GraType}}^{\text{relaType}}(\text{GraID}_j, \text{relID}_j)^{0...*}, \\ & \text{Role}_{\text{RelaType}}^{\text{roType}}(\text{RelaID}_j, \text{roID}_j)^{0...*}, \text{Point}_{\text{ObjType}}^{\text{poType}}(\text{ObjID}_j, \text{poID}_j)^{0...*}, \} \end{aligned} \quad (3.28)$$

假设 D_a 姿态稳定方式选项组中三个待选项的差别仅为是否支持敏捷机动性，则该设计选项可以表达为式 3.29。式 3.29 中， D_a 姿态稳定方式选项组是待选项 j 的集合，

仅由一组不同的属性实例组成, $Property_{Block}^{Boolean}(Block_g, AG)$ 表示模块实例 $Block_g$ 的布尔类属性 AG , AG 表示敏捷机动性能, 其值为 $true$, 即可以实现重力稳定方法可以实现敏捷机动。待选项 j_a 是这三个属性实例之一。

$$\begin{aligned} j_a \in \{ &Property_{Block}^{Boolean}(Block_d, AG), Property_{Block}^{Boolean}(Block_t, AG), \\ &Property_{Block}^{Boolean}(Block_g, AG) \} \\ &Property_{Block}^{Boolean}(Block_g, AG) = true; \\ &Property_{Block}^{Boolean}(Block_t, AG) = false; \\ &Property_{Block}^{Boolean}(Block_d, AG) = false; \end{aligned} \quad (3.29)$$

设计空间的定义明确了所有方案的可能性。为了满足指标的方案, 还需要构建关于指标的约束集合 C , 约束集合内包含了多个约束, 如式 3.30 所示。

$$C = \{C_{Boolean}^1, C_{Boolean}^2, \dots, C_{Boolean}^{num}, \dots, C_{Boolean}^m\} \quad (3.30)$$

约束的形式和前文式 3.16 类似, 如式 3.31 所示。

$$\begin{aligned} C_{Boolean}^{num} = &\odot_{SMT} (Property_{otherType}^{ProType}(otherName, ProName), \\ &Size(modelElement_{gopprr}), U(modelElements_{gopprr}), \\ &\Phi_{SMT}) \end{aligned} \quad (3.31)$$

以仅由敏捷机动性组成的设计选项为例, 若要选出符合要求的方案, 则应该在式 3.29 的基础上, 增加相关约束, 如式 3.32, 该约束说明预期目标敏捷机动可以执行, 且姿态稳定方式的敏捷机动性能和指标相符。 $Property_{Requirement}^{Boolean}(Req, AG)$ 表示为从需求对象实例 Req 中的布尔属性 AG 。

$$\begin{aligned} &(Property_{Requirement}^{Boolean}(Req, AG) == true) \& \& (Property_{Block}^{Boolean}(Block_j, AG) \\ &== Property_{Requirement}^{Boolean}(Req, AG)) \end{aligned} \quad (3.32)$$

除此以外, 部分方案空间优化还需要添加优化目标 O , 如式 3.33。 O 的角标 $numerical$ 表明优化目标 O 返回的类型一般为算数类型, 即: 实数类型或整数类型。

$$O = \{O_{numerical}^1, O_{numerical}^2, \dots, O_{numerical}^k, \dots, O_{numerical}^o\} \quad (3.33)$$

$O_{numerical}^o$ 的定义方式同样基于模型元素和可满足性模理论中的对象集, 得到式 3.34 的定义。

$$\begin{aligned} O_{numerical}^o = &\odot_{smt} (Property_{otherType}^{ProType}(otherName, ProName), \\ &Size(modelElement_{gopprr}), \\ &U(modelElements_{gopprr}), \Phi_{SMT}) \end{aligned} \quad (3.34)$$

约束 C 和目标 O 在评估过程中通过“与”连接符连接, 如式 3.35 和式 3.36。

$$\text{Constraint: } C_{Boolean}^1 \wedge C_{Boolean}^2 \wedge \dots \wedge C_{Boolean}^{num} \dots \wedge C_{Boolean}^m == \text{true} \quad (3.35)$$

$$\text{Objective: } O_{numerical}^1 \wedge O_{numerical}^2, \dots, O_{numerical}^k, \wedge O_{numerical}^o == \text{true} \quad (3.36)$$

3.3.3 指标验证方法

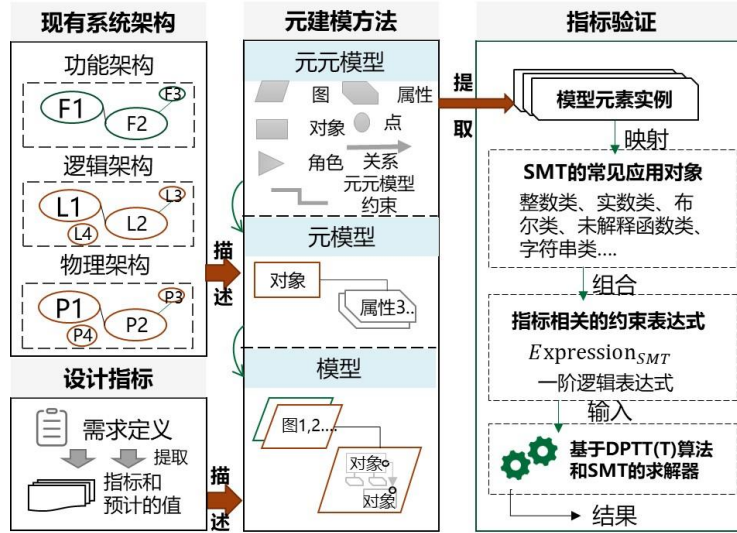


图 3.6 面向模型元素的指标验证方法

本节在 3.3.2 节的基础上，阐述如何对一阶逻辑表达式形式的约束表达式进行求解，如图 3.6 所示。首先，基于元建模方法模型化描述系统架构和指标，再提取模型元素实例，构建可满足性模理论和模型之间的映射关系，形成面向模型元素的一阶逻辑表达。验证方法采用以可满足性模理论为理论基础的定理证明方法。将面向模型元素构建的一阶逻辑表达式转换为多个范式，采用以 DPLL (T) 算法为核心的求解架构对范式的可满足性进行求解。

3.4 本章小结

本章研究面向异构模型的指标形式化表达和验证方法，提出了一种基于 GOPRR-E 元建模方法和可满足性模理论的跨建模语言指标验证方法。本章首先研究 GOPRR-E 元建模方法作为形式化描述异构模型元素的基础。其次，研究可满足性模理论作为量化表征和验证一阶逻辑表达式的理论基础。最终，提出支持面向异构模型的指标验证方法，形成面向模型元素的形式化表达和验证系统架构和指标之间约束的方法，形式化描述指标验证中静态指标验证和设计方案确认的方法。本章提出的面向模型元素的指标验证方法突破了异构语言的限制，支持验证异构建模语言构建的自定义指标。

同时，本章直接使用模型元素构建指标相关约束，保证建模和指标验证的数据一致性、约束表达和验证约束可满足性的理论一致性，减少数据传递带来的不确定性。

第4章 指标验证语言设计与实现

本章针对目前 MBSE 领域内通用建模语言无法支持指标形式化表达和验证, 缺乏统一语言实现面向异构模型的指标验证问题, 研究一种支持建模、指标形式化表达和验证的统一建模语言。本章首先分析基于 GOPPRR-E 元建模方法的统一建模语言 KARMA, 和基于可满足性模理论的程式语言 SMT-LIB, 明确他们的语言结构和语义风格。其次, 定义基于 KARMA 的指标验证语言, 指标验证语言结合 SMT-LIB 语言的语法结构, 沿用 KARMA 语言的语义风格, 形成一种支持跨建模语言建模、指标形式化表达和验证的统一语言。最后, 定义基于指标验证语言的语法和语义。在此基础上, 定义相应的语言编译器, 实现指标验证功能。

4.1 基于 KARMA 的指标验证语言规划

本节首先研究语言的定义规则, 并探究 KARMA 语言和 SMT-LIB 语言这两种语言规范的语法和语义, 最终得到指标验证语言规划。

4.1.1 语法和语义

建模语言可以通过五元组来形式化描述其语法和语义, 如图 4.1 领域建模语言的语法和语义所示。语言的构建包括具体语法 C (Concrete syntax)、抽象语法 A (Abstract syntax)、语义定义域 S (Semantic domain)、句法映射 Mc (Syntactic mapping)、语义映射 (Semantic mapping), 可以由以下的公式进行表示: $L = \langle C, A, S, Mc, Ms \rangle^{[99]}$ 。领域建模语言是各种建模语言的一种统称。

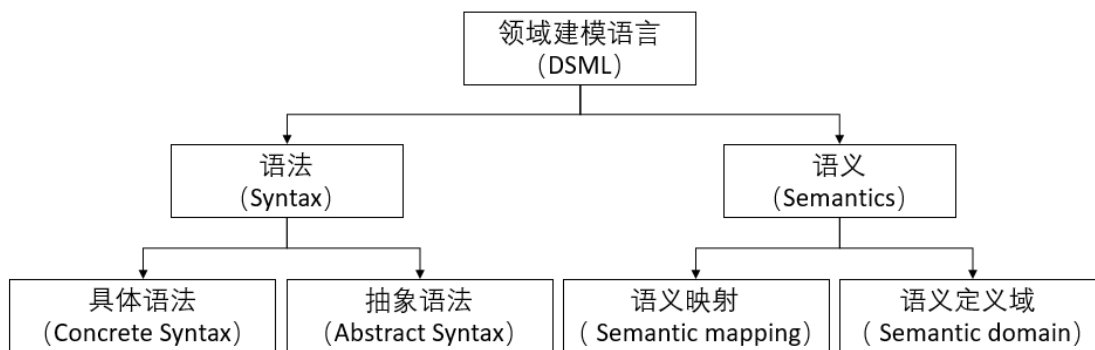


图 4.1 领域建模语言的语法和语义

语法的组成包含三个部分: 抽象语法、具体语法和句法映射, 其中:

(1) 具体语法^[98]: 定义了领域语言中元素的表示方法, 简单来说就是在建模环境中该元素的样式, 具体可以表示为图形形状、文本描述或者二者的混合。

(2) 抽象语法: 定义了领域语言中可用的语言元素的概念、元素之间的关系、元素之间的交互规则及约束, 为建模的正确性提供参考。

(3) 句法映射: 定义了抽象语法到具体语法的映射, 记为: $A \rightarrow C$ 。抽象语法定义了各元素的连接规则, 具体语法是各元素的表现形式, 但是在不同场景下表现方式也可能不同, 因此, 在不同建模语言中需要定义相应的语法映射规则。

语义包含语义的定义域与映射关系:

(1) 语义定义域: 通过数学逻辑定义了各元素的含义。

(2) 语义映射: 定义了抽象语法到语义定义域的映射, 记为: $A \rightarrow S$ 。此外, 语义映射和语义定义域共同描述了领域建模语言的语义, 即元素具备什么含义^[99]。

4.1.2 KARMA 语言分析

KARMA 语言全称为 **Combination of ARchitecture Model specificAtion**, 它是在 GOPRR-E 元建模方法的基础上形成的, 能够支持不同层次(模型层、元模型层、元元模型层)的模型描述。所以, KARMA 语言能够统一描述不同建模语言构建的模型(UML, UPDM, SysML 等建模语言)。

4.1.2.1 KARMA 语言抽象语法结构

KARMA 语言抽象语法如图 4.2 KARMA 语言抽象语法所示, KARMA 抽象类包含类、基础数据类型、执行语句、函数体、方法五大模块。

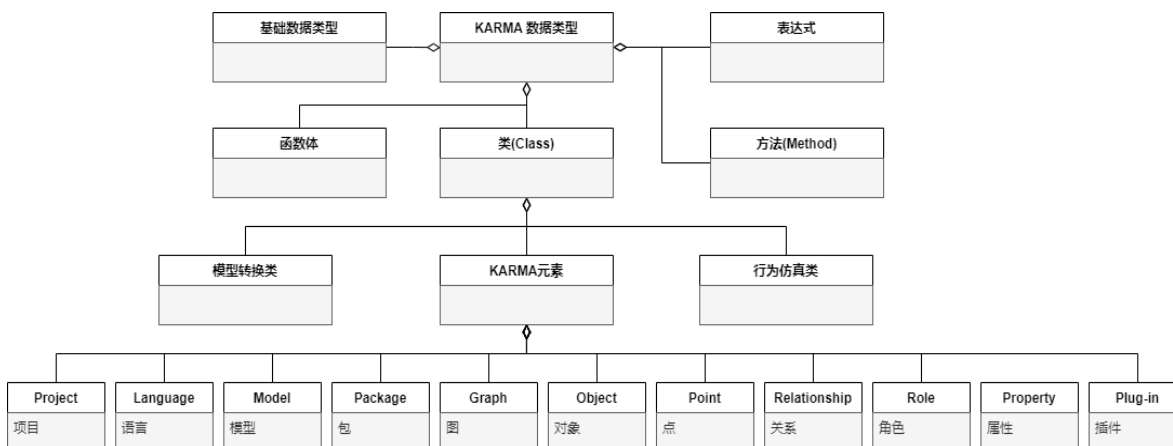


图 4.2 KARMA 语言抽象语法

(1) 基础数据类型 (Primary)：基础数据类型指的是在 KARMA 语言中通用的数据类型。除通用数据类型外，还有各个类或方法独有的数据类型。基础数据类型中包含算数类型（即：整型和实数型）、字符串类型、布尔类型、一维数组类型、矩阵类型等。

(2) 表达式 (Expression)：表达式包含执行语句（循环语句和控制流程语句）、变量的声明和赋值以及各类数据类型的方法。

(3) 类 (Class)：类用于将数据和操作这些数据的方法封装在一起，被用于表示问题域的抽象概念。KARMA 语言中含有两种类型的类，一种为模型元素和结构的类，用于反映模型元素实例的内涵或模型元素之间构成的结构，模型元素类之间的结构如图 4.2 所示；一种为模型操作类，用于对模型进行操作，例如对模型元素进行转换或者行为仿真等。

(4) 方法 (Method)：方法和类的作用类似，用于操作模型。方法不同于类的是，方法并不是一种封装结构，而是一种顺序执行模块。

(5) 函数体 (Function)：函数体包含通用函数、用于描述 svg 的函数以及引用外部函数。通用函数即支持用户自定义函数，使用关键字 func 标识。

4.1.2.2 KARMA 语义风格

KARMA 语言在语义上遵循 Unicode 编码。语义定义域和映射关系遵循常规设置。

(1) 标识符：标识符由字母、数字和下划线组成的字符序列，用于命名 KARMA 语言重的类名称、变量名称、常数名称等其他项目的名称。标识符必须以字母开头，后续字符可以为字母、下划线或数字。

(2) 关键字：关键字是指该语言内预先保留的标识符，具有特殊含义，不能作为其他对象的名称出现。KARMA 语言的常用关键字（不区分大小写）如下：

表 4.1 KARMA 语言关键字

关键字	说明	关键字	说明
Array	数组声明	Project	声明项目
Int	整型声明	Language	声明语言
Real	实数声明	Model	声明模型
Matrix	矩阵声明	Graph	声明图
Boolean	布尔声明	Object	声明对象

表 4.1 续表

关键字	说明	关键字	说明
String	字符串声明	Relationship	声明关系
Maximum	声明取最大值	Property	声明属性
Minimum	声明取最小值	Role	声明角色
ite	二元运算符	Point	声明点
if	if 语句声明	else	if 语句的控制条件
end	表示执行句结束	elseif	if 语句的控制条件
print	声明打印	for	循环语句的声明
Func	函数体声明	return	返回值声明
NEWLINE	打印换行声明	unit	单位声明
true	真值常量	false	假值常量
	或	=>	蕴含
&&	与	+	加
==	相等	-	减
!=	不等	*	乘
^	异或	/	除
**	幂符号	)	右括号
(	左括号	/*, //	注释

KARMA 语言的声明风格通常为首先通过某关键字声明某一模块，如下的代码模块通过关键字“Object”声明了一个参数模块。红色字符表示关键字。

```

1. Object objectIns_object_Parameter3 refer object_Parameter
2.     annotation(
3.         initialLocation=(311,516,158,83),
4.         localLabel="参数",
5.     );
6. end objectIns_object_Parameter3;
```

4.1.3 SMT-LIB 语言分析

SMT-LIB 是基于可满足性模理论设计的语言^[100]，能够作为所有可满足性模理论求解器的通用语言。SMT-LIB 具有 Lisp 语言语法。Lisp 是函数式语言，即：其操作均

以函数传参的形式实现，例如：若要实现 $3+2$ ，在 Lisp 语言中会被写作为 $+(3,2)$ ，将 3 和 2 作为参数传入。

4.1.3.1 SMT-LIB 语言抽象语法

SMT-LIB 的在 C++，Python 以及 java 中都有相应的 API，它们的抽象语法分类和类之间的关系大致相同，其主要抽象类及其关系如图 4.3 SMT-LIB 语言抽象语法所示，SMT-LIB 语言主要包含 7 个类。

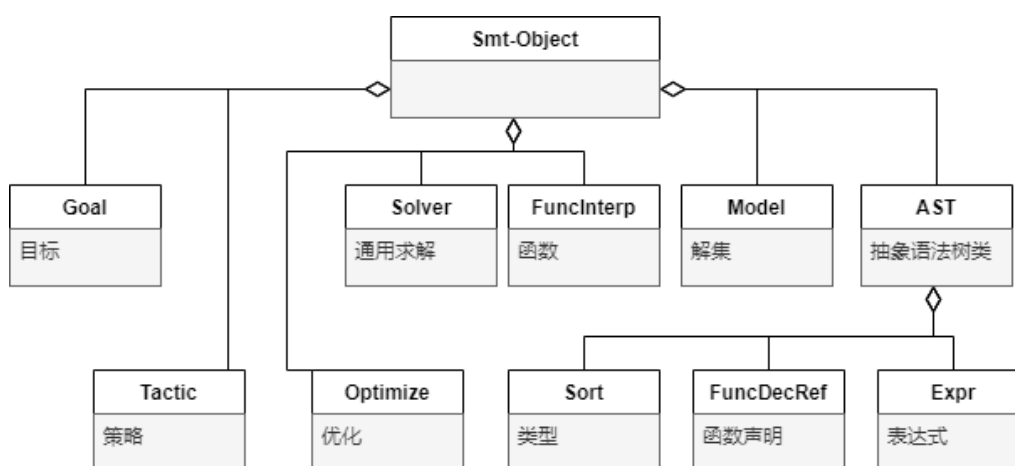


图 4.3 SMT-LIB 语言抽象语法

(1) Solver 求解器类：该类包含检查多个或一个一阶逻辑表达式形式的约束可满足性的操作。

(2) Optimize 优化求解器类：该类支持包含多个约束和优化目标（值的最大或最小）的描述和求解，能够检索包含目标函数的约束。

(3) Tactic 策略求解器类：与只能检查一组约束的可满足性的通用求解器不同，策略将约束转换为一组断言，并形成抽象语法形式的节点，断言语法由代表目标的节点和代表子目标的子节点组成。它为特定问题领域创建自定义解决方案的基本构建块，策略求解器内包含完整的策略列表，以字符串形式存在，如“split-clause”（拆分约束子句），“nlsat”（优先使用非线性理论）。

(4) Goal 目标：策略求解器分解出的一系列约束及节点就是“目标”goal，其本质时一系列约束。

(5) FuncInterp 函数类：包含函数执行语句的类。

(6) Model 解集类：包含求解器求解的结果，以及满足约束时返回的解的操作。

(7) AST 抽象语法树类：是用于表示类型、函数声明和表达式的有向无环图。包含的 sort 是各个数据类型的类别（type），以及函数的声明（不含操作）及各类型表达式的集合。表达式类型包含算数表达式、数组表达式、机器数表达式、布尔表达式、正则表达式、字符串表达式等，数据类别和表达式相对应。

4.1.3.2 SMT-LIB 语言简析

SMT-LIB 语言本身的风格延续 Lisp 的传递参数风格，可读性较差。SMT-LIB 语言在 Python, JAVA 以及 C++中均有相应的接口，适应各个语言的风格。适用于不同语言的 SMT 语言的关键字未变，常用的关键字：check(求解)、solver（求解器声明）等沿用在各类 API 中。因此在接下来的语言定义中，将沿用 SMT-LIB 的关键字和语法结构，而使用 KARMA 语言的语言风格，令 KARMA 语言各模块语言保持统一。

4.1.4 KARMA-指标验证语言规划

指标验证模块语言融合上述两种语言的语法和语义。指标验证模块语言是在 KARMA 语言的基础上，扩展一个方法，实现指标验证功能。该语言模块增加了 KARMA 抽象语法的一个抽象节点，同时使用 KARMA 语言其他模块的方法和数据，如：模型元素的数据、模型元素的描述、常用关键字、常用执行语句（赋值语句、流程控制语句、循环语句）、模块定义方式。因此，该语言应该沿用 KARMA 语言的语言风格，保持语言的一致性，降低学习新语言的成本。同时，指标验证语言应结合 SMT-LIB 语言的语法结构，扩展相应的关键字和句法，实现指标验证功能。

第 3 章 系统设计指标形式化表达与验证方法的研究内容指出了指标验证的理论方法。基于指标验证的理论方法，指标验证的语法至少应该包含以下内容：

- (1) 模型元素的获取方法
- (2) 基于模型元素的约束构建方法
- (3) 求解器的声明方法
- (4) 向求解器内添加约束、进行求解、获取解集的方法
- (5) 优化方法

4.2 KARMA-指标验证语言研究

本节主要描述指标验证模块语言的语法设计和语义设计。KARMA-指标验证语言

沿用 KARMA 语言风格，结合 SMT-LIB 语言的语义和语法，形成一种支持形式化描述异构模型、量化表征指标相关约束、评估约束可满足性的统一语言。

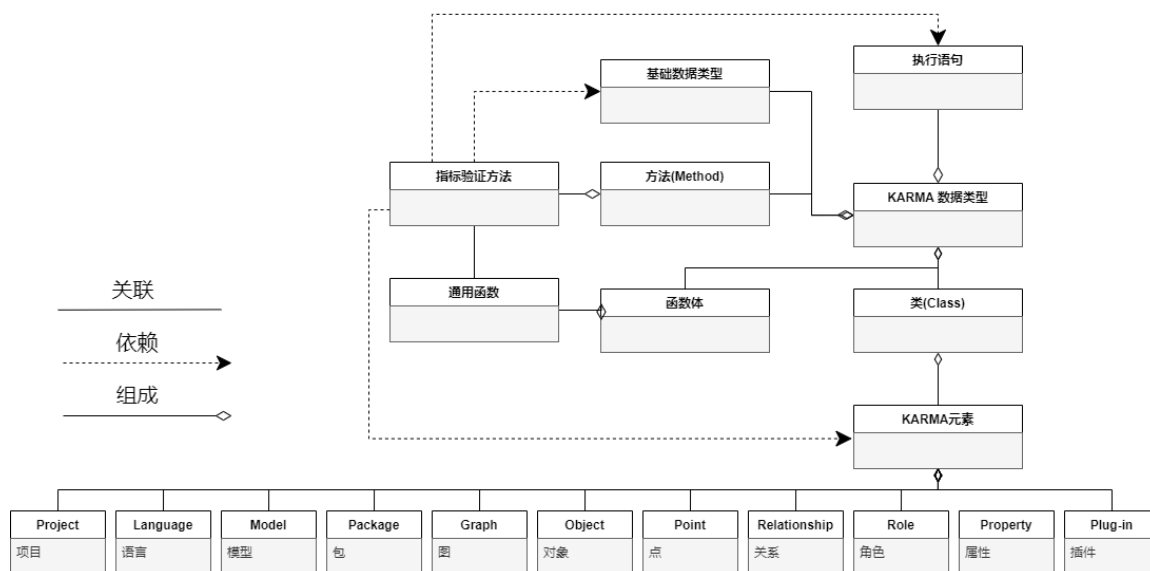


图 4.4 KARMA-指标验证语言抽象树（整体）

4.2.1 指标验证语言的抽象语法

指标验证语言属于 KARMA 语言。增加该模块后，KARMA 语言抽象语法树如图 4.4 KARMA-指标验证语言抽象树（整体）所示。指标验证属于方法类，同时依赖于执行语句（Statement）类、基础数据类型类以及 KARMA 元素类，并能够和函数类的通用函数方法产生关联。

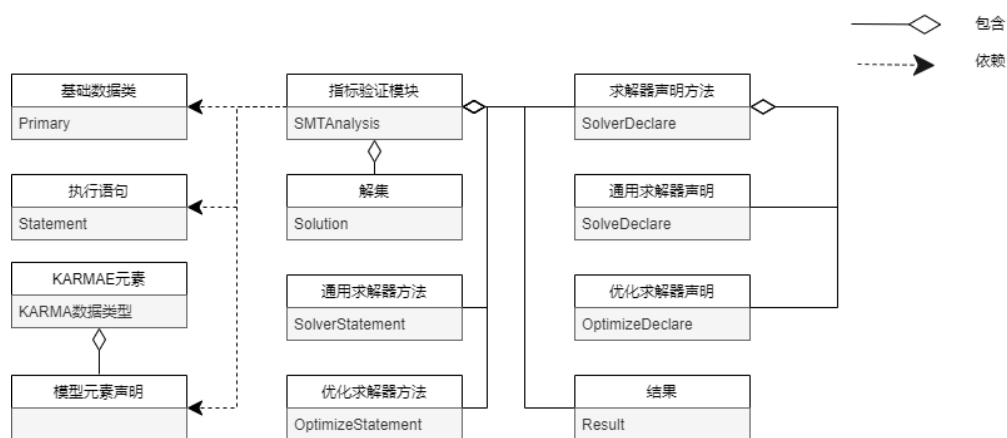


图 4.5 KARMA-指标验证语言抽象语法树（指标验证部分）

指标验证部分细化后的抽象语法树如图 4.5 KARMA-指标验证语言抽象语法树（指标验证部分）所示。指标验证模块包含求解器声明方法（SolverDeclare）、解集（Solution）、结果（Result）、通用求解器方法（SolveStatement）和优化求解器方法（OptimizeStatement），具体解释如下。

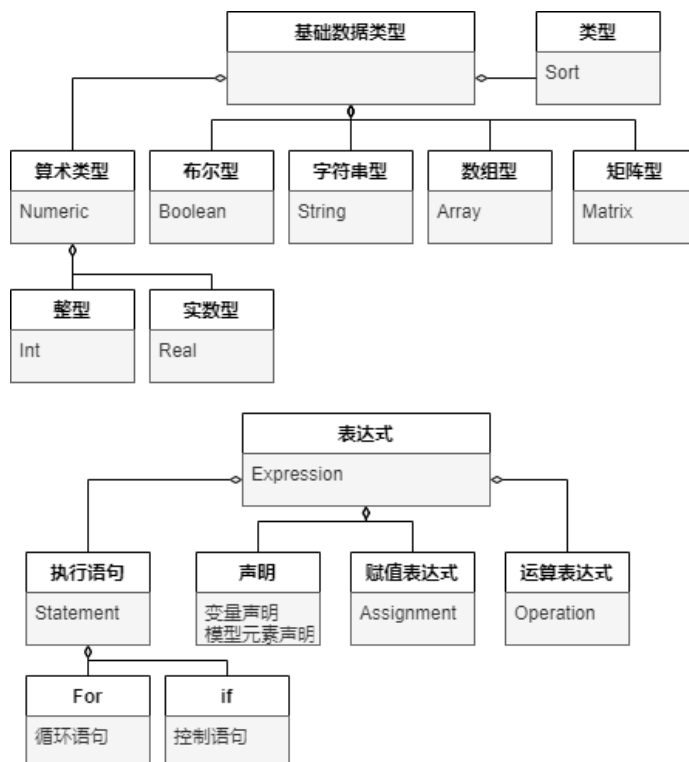


图 4.6 基础数据类型和表达式类型的抽象语法树

（1）指标验证方法：指标验证方法包含指标验证执行过程的所有方法，用以支持模型约束的描述和验证。

（2）求解器声明：求解器声明方法包含在指标验证模块内，一个指标验证模块至少有一个求解器声明方法。指标验证模块目前支持两种求解器的声明：通用求解器和优化求解器。

（3）通用求解器方法：定义通用求解器内可以使用的相关方法，包含添加约束、求解、建立堆栈等。

（4）优化求解器方法：定义优化求解器内可以使用的相关方法，包含通用求解方法以外的软约束构建、最大值声明、最小值声明等方法。

（5）结果：结果在指标验证模块内特指求解约束后输出的结果，包含可满足、不可满足以及未知。

(6)解集: 指的是满足所有约束的一组解集。和 SMT-LIB 语言中的 Model 一致, 为了和 KARMA 语言中的 model 元素区分, 改为 “solution”。

指标验证模块所依赖的基础数据类型和表达式类型的抽象语法树如图 4.6 基础数据类型和表达式类型的抽象语法树所示。基础数据类型包含算术类型、布尔型、字符串型、数组型、矩阵型以及类型。表达式类包含执行语句、变量和模型元素的声明、赋值表达式以及运算表达式。

4.2.2 指标验证语言的语法设计

本节定义指标验证语言的具体语法。

4.2.2.1 关键字

关键字是指定义数据类型和流程声明的特定字符, 自定义的参数和变量名称不可为关键字。指标验证语法在 KARMA 语言的基础上增加下表的关键字, 具体作用将在后文中说明。

表 4.2 指标验证语法关键字

add	Add	AddSoft	addTrack	ArrayAdd	ArrayDel
ArraySort	BoolSort	Check	check	Exsit	Forall
initial	IntSort	ite	lambda	Min	Max
Pop	pop	Push	push	rank	RealSort
SAT	simplify	size	SMTAnalysis	solution	Solution
Solve	UNKNOWN	UNSAT			

4.2.2.2 指标验证方法声明

指标验证方法被定义为 SMTAnalysis 类型, 关键字 “SMTAnalysis” 用于标识一个指标验证方法, 通过使用关键字加模块名称可以定义该方法。指标验证方法包含执行指标验证的所有方法, 且至少包含一个求解器。其具体语法如下:

```
1. SMTAnalysis SMTModuloName // 新建一个名为“SMTModuloName”的方法
2. ... ..// 主体代码
3. end SMTModuloName
```

从代码可知, 指标验证方法由关键字 “SMTAnalysis” 定义, 关键字后跟方法实例

的名称，同时以关键字“end”结尾，“end”后也需要添加相同名称，保持一致。代码主体出现在关键字“SMTAnalysis ”和“end”之间。

4.2.2.3 求解器的声明及执行

求解器类型 Solver 包含三种求解器，分别是：通用求解器 Solve，优化求解器 Optimize，策略求解器 Tactic（暂未定义）。各求解器内均可定义变量、约束、求解方法

（1）通用求解器 Solve 的声明和执行

通用求解器可以判断求解器内约束的可满足性。

1) 通用求解器的声明

通用求解器定义的方式同样通过关键字“Solve”定义，关键字后跟随求解器名称 solveName，求解器内填充求解器相关约束定义，最后通过关键字“end”跟随求解器名称结尾。其声明方式如下：

```
1. Solve solveName
2. // 新建一个名为“solveName”的Solve类型的求解器
3. ... ..
4. end solveName
```

2) 通用求解器的添加约束、求解

添加约束是指将构建完成约束通过关键字“add”，放入指定求解器内。添加约束后返回添加约束后的求解器。在求解器内添加约束。约束必须被表示为布尔类型的形式。约束添加可以只添加一个也可以添加多个，多个约束需要用英文逗号分隔开，如：add(Boolean constraint1, Boolean constraint2,)。其具体语法如下所示，其中 solveName 指代通用求解器的名称。

```
1. solveName.add(x<y, y<2);
2. solveName.add(x<y); solveName.add(y<2); // 两行约束添加情况一致
```

当完成约束的添加后，一般会对约束的可满足性进行求解。具体语法如下所示。

```
1. solveName.check;
```

求解约束是通过关键字“check”定义。求解即是对该求解器内的全部约束进行评估、检验，判断是否存在一组解满足所有约束。返回求解结果状态 Result。如果存在

一个满足求解器内所有约束的解集，则返回“条件可满足”（SAT）；如果不存在这样的解集，则返回“条件不可满足”（UNSAT）；如果无法判断结果，则返回“条件不足，无法求解”（UNKNOWN）。

3) 通用求解器获取通用解

完成求解器的求解后，若求解器返回结果为“条件可满足”（SAT），则可以通过关键字“`solution()`”获取满足所有约束的一组随机解。该解集包含所有变量的取值，这种取值能够令所有约束达到“满足”状态。其具体语法如下：

```
1. Solve solveName
2. // 新建一个名为“solveName”的通用求解器
3. ... ..
4. // 添加主体代码后，进行求解。
5. if(solveName.check==SAT)
6.   solveName.solution( );
7. // 当求解结果是可满足的时候，获取一组变量的一组解
8. else
9.   print("Not satisfying");
10.end
11.... ..
12.end solveName
```

“`solution()`”方法可以获取一组满足约束的所有变量解。若只需要获得某一变量的解，则可以在括号内添加该变量或多个变量的名称，如：`solution(variableName, variableName...)`。具体语法如下：

```
1. solveName.solution(x1, x2...)
```

4) 通用求解器堆栈的建立与弹出

“`push`”以及“`pop`”关键字可以创建求解器的临时堆栈，并将约束暂时存放在堆栈内或从堆栈内弹出。需要创建局部声明时，可以通过关键字为“`push`”的“推”操作，保存当前堆栈大小并且创造一个新的作用域，关键字“`pop`”的操作可以删除与之匹配的“`push`”操作中执行的任何约束。`Check`和`model`操作仍然可以对局部的约束进行操作。如果不添加`push`和`pop`操作，那么在求解器内维护的就是全局堆栈中的约束与声明。`pop(int i)`可以弹出从栈顶计算的`i`项约束，其具体语法如下：

```
1. solveName.push; // 保存当前求解器内所有声明并创建一个临时的作用域。
```

```
2. solveName.pop; // 删除与之前匹配的push 之间执行的所有声明
3. solveName.pop(2) // 弹出栈内前两项约束
```

5) 通用求解器的追踪式约束添加

“addTrack”关键字可以在添加约束的基础上添加追踪约束的实现情况。当已有约束放进创建的通用求解器时，增加一个布尔值用于追踪相应约束的实现。如果对应的布尔表达式存在真值，那么这个约束式可以被满足。该功能是 Solve 类的特定功能。

“addTrack”添加约束时，括号内存在两个布尔类型，为 addTrack(Boolean constraint, Boolean token)，第一个布尔表达式为需要被追踪是否有解的约束条件，第二个布尔表达式为追踪约束条件的标识，如下所示：

```
1. solveName.addTrack(x<5, b) //b 是一个布尔变量，用于追踪对应的约束是否
    可满足
```

(2) 通用求解器 Solve 的声明和执行

优化求解器通过关键字“Optimize”表示，该类型特定于指标验证方法下。优化求解器相较于通用求解器整合了一些优化算法，在指令方面，添加了软约束以及最大化和最小化等额外的操作方法。

1) 优化求解器的声明

优化求解器定义的方式通过关键字“Optimize”定义，关键字后跟随求解器名称 optimizeName，求解器内填充求解器相关约束定义，最后通过关键字“end”跟随求解器名称结尾。其声明方式如下：

```
1. Optimize optimizeName // 新建一个名为“optimizeName”的优化求解器类型的求
    解器
2. ... ..
3. end optimizeName
```

优化求解器的添加约束方法、约束检验方法、获得通用解方法、构建栈的方法和通用求解器的方法基本保持一致，但优化求解器的关键字首字母为大写。所以，适用于优化求解器的关键字为“Add”、“Check”、“Solution”、“Pop”、“Push”。除此以外，优化求解器不具备追踪约束的方法，额外具备添加最大化、最小化、软约束的方法。

2) 优化求解器的最值约束

关键字“Max”和“Min”定义了优化求解器求最值的方法。最大化方法 `Max(Numeric Expr)` 相当于一个特殊的布尔条件，即：使得输入表达式获得最大值的可能，返回值为添加最大值约束后的优化求解器。括号内添加算数表达式，包含整数表达式和实数表达式，表达式形式为变量或变量的组合，意味添加最大化该变量的约束。最小化方法 `Min(Numeric Expr)` 相当于一个特殊的布尔条件，即使得输入表达式获得最小值的可能，返回添加最小值约束的优化求解器。其具体语法实例如下：

```
1. optimizeName.Max(2x+7);
2. optimizeName.Min(x); //x 为算术类型
```

3) 优化求解器的软约束

软约束是针对多布尔条件的一种线性权衡分析。软约束中添加的条件会带有权重，求解器可以根据权重的大小进行判断是否存在合适的解释。布尔条件在加入 `Optimize` 求解器中是带有权重的，权重必须为正实数，如果不填写权重值会被默认为 1。其形式为：`AddSoft(Boolean constraint, int weight, String group)`，该操作需要输入三个参数，依次是类型为布尔表达式的约束，类型为正整数型的权重值，类型为字符串类型的组别名称。权重值表示该约束条件的优先级。约束可以根据组名进行分组，同一组内的权重进行比较。其具体语法实例为：

```
1. optimizeName.AddSoft(x+y<5, 3, "s");
2. optimizeName.AddSoft(y<5, 2, "s");
3. //两约束均为“s”组内的约束，前一个约束的权重为3，后一个约束的权重为2
```

4.2.2.4 变量的声明与赋值

(1) 变量的声明

变量的声明方式为“`datatype Ident`”，即：表征数据类型的关键字加上变量名称。数据类型的关键字为 `Int`（整数型）、`Real`（实数型）、`String`（字符串型）、`Boolean`（布尔类型）、`Array`（数组类型）、`Matrix`（矩阵类型）。其具体语法形式如下：

```
1. Int a; // 声明变量a 为整型数据
2. Real a; // 声明变量a 为实数型数据
3. Boolean a; // 声明变量a 为布尔型数据
4. Array(Int, Int) a; // 声明变量a 为存储整型的数组, 且索引类型为int
5. Matrix m = IntegerMatrix.initial(7,3); // 声明变量m 为存储整型的矩阵
6. Matrix M= RealMatrix.initial(7,3); // 声明变量M 为存储实型的矩阵
```

其中，数组类型的声明为“Array(datatype, dataType) Ident”。关键字 Array 后还需要定义两种数据类型，分别是数组索引域的类型和数组存储的数据类型，如：整型、实数、布尔等。矩阵类型的声明需要通过关键字“IntegerMatrix”或“RealMatrix”明确矩阵类型，即：矩阵存储的是整型或实型，同时通过关键字“initial”初始化，明确矩阵的行列数。

(2) 变量的初始化和赋值

变量的初始化和赋值分为两部分：来自于自定义数据、来自于模型。初始化或赋值后的变量完成对常量的映射。

1) 来自于自定义数据

变量的自定义数据赋值方式一般以正确的形式跟随在等号后。算术类型的自定义数据为 0-9 以及小数点的数字组合；字符串类型以引号和字符组成；布尔类型的自定义数据为常量“true”或“false”；数组类型的自定义数据为小括号和存储值得组合；矩阵类型得自定义数据为中括号和算数值的组合。

```
1. Int a; // 声明变量a 为整型数据
2. a=1; // 为变量a 赋值
3. Int a = 1; // 初始化变量a 为整数1
4.
5. String s ="string contents";
6.
7. Boolean a = true; // 初始化变量a 为布尔类型 true
8.
9. Array(Int,Int) a; // 声明变量a 为存储整型的数组
10. a = Array(1,1,2); // 为变量a 赋值
11. Array(Int,Int) a = (1,1,2); // 初始化变量a， 存储整数1， 1， 2。
12.
13. Matrix m = IntegerMatrix.initial(2,3); // 声明变量m 为存储整型的矩阵
14. m =[1,2,3;1,2,3]; // 为变量m 赋值
15. Matrix m = [1,2,3;1,2,3]; // 初始化 2×3 矩阵m， 存储整数1， 2， 3， 1， 2， 3。
```

2) 来自于模型元素

为了连接求解和模型元素，变量可以由模型元素组成，以“属性”类型为例。等号右边分别说明建模语言的名称、模型名称、对象名称、并通过关键字“Property”说明属性名称，具体语法如下：

```

1. dataType Ident = LanguageName.ModelName.ObjectName.Property[ProName];
2. Int a = SysML.ReqDiagram.ReqObj1.Property[cost];
3. // 整型 a 来源于 sysml 模型中构建的需求模型中的需求对象的名为 “cost” 的属性

```

获取其他模型元素的方式类似，但是其他模型元素的数据类型不能直接应用在指标验证中，需要通过操作模型元素，如计算某图的个数、新建一种谓词函数关联两个对象从而返回布尔值加入运算等。

4.2.2.5 表达式简化法、ite 方法与其他公用方法

指标验证同时依赖于基础数据类型、表达式以及通用函数方法。各类数据类型的表达式可以通过关键字“simplify”进行简化，如“(3+2).simplify”得到的结果是“5”。还可以通过关键字“ite”定义简单的控制流程，形式为“ite (boolExpr, expr1, expr2)”，即：括号内添加一个布尔表达式，两个表达式，若布尔返回真，则该表达式返回 expr1，若结果为假，则返回第二个表达式。

各个基础数据类型的操作方法和通用方法大体类似，表达式中的执行语句包含了控制流程语句和循环语句，这些内容联通通用函数方法均在附录 A 中。

4.2.2.6 指标验证语言设计总结

目前，指标验证的语法设计在模型元素方面能够实现面向属性元素的约束构建、面向其他模型元素的谓词关系构建，但是无法定义模型内部的“包含”、“连接”等关系，如：明确某一对象内属性的数量、明确某角色关联的对象和关系，如表 4.3 所示；在约束的形式化表达和验证方面，指标验证语法支持通用求解器、优化求解器两个求解器的构建，以及常用数据类的约束构建，具体如表 4.4 所示。从中可以看出，指标验证支持面向模型元素的通用约束求解和优化约束求解。

表 4.3 指标验证语言支持描述的模型元素

GOPRR-E 的模型元素	指标验证语言是否支持
模型元素：属性实例、对象实例、图实例、点实例、关系实例、角色实例	支持描述
模型关系：包含、剖视、分解、绑定关联、属性连接	不支持描述

表 4.4 指标验证语言支持的可满足性模理论方法

基于 SMT 方法	指标验证语言是否支持
构建验证类	支持
抽象数据类型的构建	支持
布尔类的声明和操作	部分支持（缺少判断数据类型的方法，可以手动判断）
整型的声明和操作	支持
实数型的声明和操作	支持
机器数的声明和操作	部分支持（缺乏机器数的复杂运算）
序列的声明和操作	部分支持（支持字符串相关声明和方法）
量词的操作	支持
未解释函数的声明和操作	部分支持（支持未解释函数的声明和使用，缺少未解释函数之间的运算，但支持其他函数的运算）
对内存的操作	不支持
通用求解器的声明和操作	支持
优化求解器的声明和操作	支持
策略求解器的声明和操作	不支持

4.2.3 指标验证语言支持指标的形式化表达和验证

本节主要通过一简单案例说明指标验证语言是如何实现构建指标和系统架构之间的约束并验证该约束的可满足性。

该案例以验证车辆底盘质量是否能够满足指标为例。如图 4.7 所示，分别构建面向车辆底盘的需求图和设计模块图。需求图定义其需求和相应的指标。该问题的设计指标为汽车底盘质量，预计质量为不超过 150 千克，则首先声明一个名为“Modulo”的验证模块和一个“abc”的通用求解器。

1. **SMTAnalysis** Modulo
2. **Solve** abc

关于该指标的预计值和指标可以通过指标验证语言形式化表达为：

1. **Int** Tchassis = SysML. car. car_obj_Req_6.**Property**[Req_6_Property1];
2. **Int** chassis;

指标相关的约束可以表示为:

```
1. abc.add(chassis <=Tchassis);
```

模块定义图表示现在系统设计的情况。已经明确车轮模块的质量为 30kg 每个，共有 4 个车轮。除此以外，从图 4.7 可知，悬架部分质量为变量存在最大值和最小值。则以相同语法形式化获取汽车的轮胎质量 *tire*，轮胎个数 *num*，悬架的最大质量 *Cmax* 和最小质量 *Cmin*，这些参数的数据类型均为整型。最后，声明变量悬架质量 *sus*。

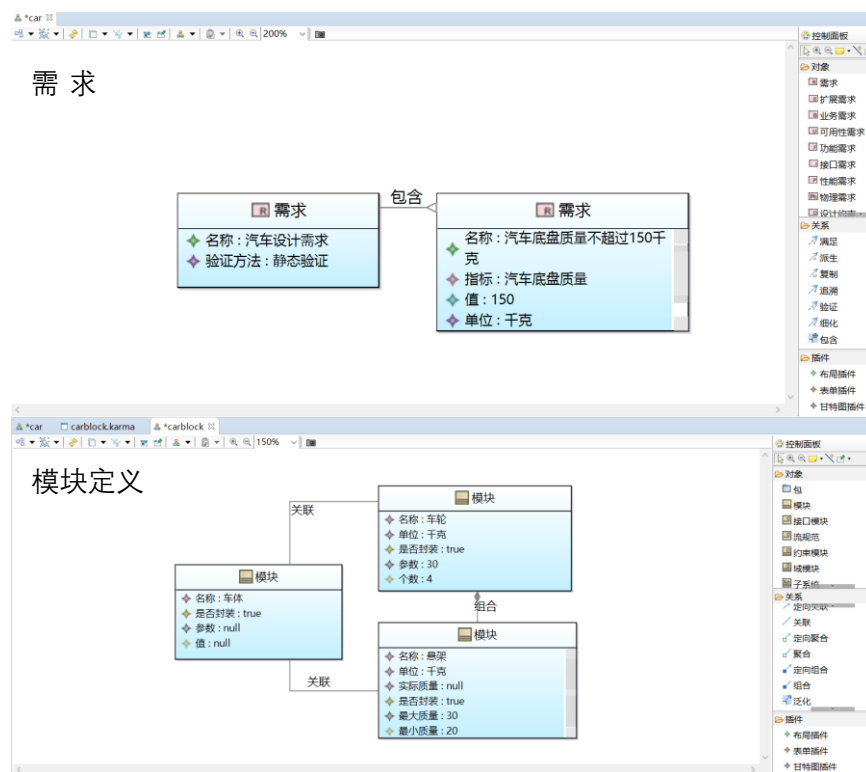


图 4.7 面向车辆底盘质量的建模

基于模块定义图和需求图，构建关于指标和现有系统架构的约束关系，并将此类约束关系投入求解器内，并进行求解，得出可能解。

```
1. abc.add(chassis == num*tire+sus); //添加指标和模型元素的约束
2. abc.add(Cmin<=sus<=Cmax); //添加模型元素自身的约束
3. abc.check; abc.solution();
```

最后，得到如图 4.8 的结果。控制台结果显示约束可满足，以及两个变量的结果。列表分析结果显示变量和所有参与运算的参数的类型和值。

The screenshot displays the car.karma IDE interface. The top panel shows the SMT analysis code for a vehicle chassis. The code defines variables for chassis, tire, and suspension, and performs a series of logical operations to verify the chassis quality.

```

1 SMTAnalysis Car
2   Solve Aabc
3   Int Tchassis = SysML.car.car_object_Object_Requirement_bdb6.Property [Object_Requirement_Property_Source_e1e490_0ce1];
4   Int cha;
5   Int sus;
6   Int tire = SysML.car.carblock_object_Object_Block_e58d.Property [Object_Block_Property_Parameter_b52115_9ad8_e928];
7   Int num = SysML.car.carblock_object_Object_Block_e58d.Property [Object_Block_Property_Value_301d0a_44ce_675a];
8   Aabc.add(cha==(sus+tire*num));
9   Aabc.add(20<=sus,sus<30);
10  Aabc.add(cha<=Tchassis);
11  Aabc.check;
12  Aabc.solution( );
13  end Aabc
14 end Car

```

The bottom panel shows the SMT analysis results. The console displays the message "约束条件可满足(SATISFIABLE)" and the variable definitions. The table below shows the values of the variables.

名称	类型	值
Tchassis	Int	150
tire	Int	30
num	Int	4
sus	Int	20
cha	Int	140

图 4.8 车辆底盘质量验证结果图

4.3 指标验证语言的实现

指标验证语言的实现借助 ANTLR 语言分析器，将 KARMA 语言转化为抽象语法树，并在相应阶段调用基于可满足性模理论的求解器，实现面向约束关系的求解，从而实现指标验证功能。

4.3.1 基于 ANTLR 的构建指标验证语言编译器

指标验证模块的实现在前文定义的语法基础上，定义两个核心模块：基于 ANTLR（ANother Tool for Language Recognition）语法分析器的自定义语法解析器，以及基于可满足性模理论的求解器 Z3 定理证明器^[23]。

ANTLR 语法分析器主要用于定义语言的语法结构，并根据定义的语法结构形成

可重写的类，支持方法的重定义。ANTLR 语法分析器内置词法分析器（Lexer），它可以量化字符流将他们翻译成离散的字符组（Token 流），通过定义关键字（fragment），标识符，符号（symbols）和操作符进行下一步的语法分析。ANTLR 语法分析器支持自定义语法，并将输入文本根据自定义语法生成语法树并可视觉化的显示出来的开源语法分析器，用于读取、处理、执行和翻译结构化文本或二进制文件。

本文中使用的基于可满足性模理论的求解器是由微软研究院研究的最先进的开源定理证明器 Z3 求解器^[103]。该求解器支持 SMT-lib 语言规范，同时提供多个语言的 API。之所以选择该求解器，是因为它在 SMT 理论求解器大赛中的优异表现。SMT 求解器的评价方式包含支持数学理论的种类、测试集的通过率和时间。Z3 求解器是支持最多种数学理论的求解器之一，同时也是已有测试集通过率最高的求解器^[101]。本文通过 ANTLR 语法分析器，融合 Z3 定理证明器，完成基于 JAVA 的 4885 行模块开发代码，实现面向跨建模语言构建的异构模型的指标验证功能，为语言和方法的有效性验证提供基础。

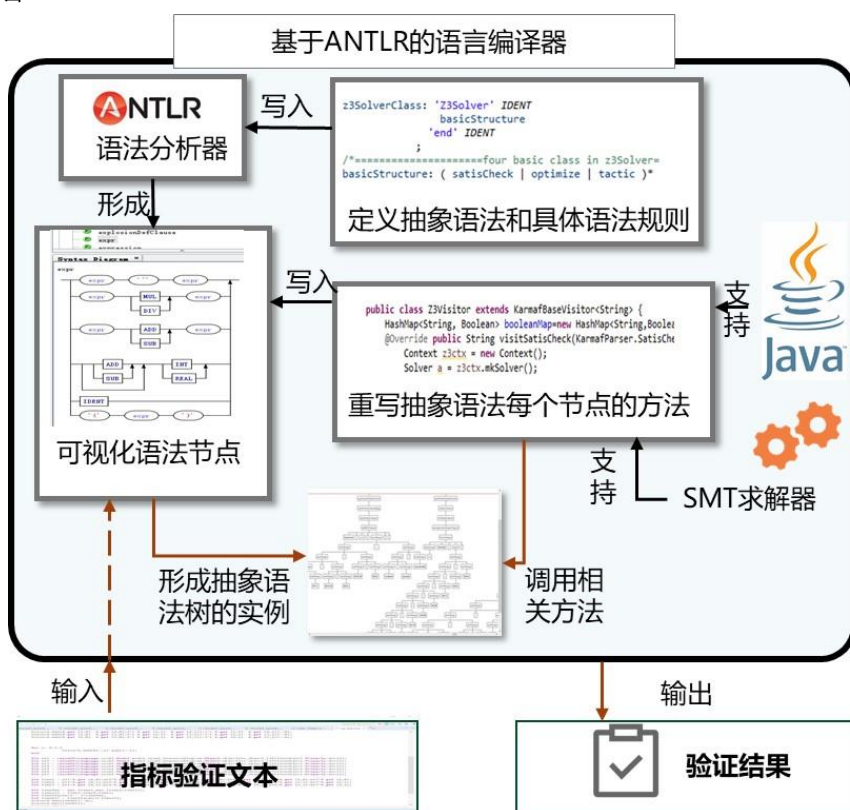


图 4.9 基于 ANTLR 语法分析器的编译器工作流程

图 4.9 基于 ANTLR 语法分析器的展示了基于 ANTLR 的指标验证语言编译器的构建方法。在基于 ANTLR 的语言编译器内部，其工作原理如下：首先使用 ANTLR

语法分析器定义指标验证的词法、抽象语法规则和具体语法规则。ANTLR 能够可视化显示语法节点。再在每个节点上重写执行该语法节点的方法。方法的重写使用 JAVA 语言支持。同时，重写方法时调用支持可满足性模理论求解器的 API，在涉及约束的构建、求解器的构建、求解器的操作等方法时调用该求解器。

图 4.9 同样展示了指验证语言编译器的工作流程。首先，指标验证文本输入基于 ANTLR 的语言编译器。ANTLR 语法分析器将输入的语言文本和预定义的语法规则进行映射，形成抽象语法树的实例，根据实例化的抽象语法树，在相应节点调用已写好的方法，完成求解，最后输出求解结果。

4.3.2 指标验证功能的实现

指标验证的功能集成在在多架构建模工具 MetaGraph 内，该工具是面向 KARMA 语言的统一多架构建模工具，该工具支持基于 KARMA 语言实现异构模型的建立、架构转换、代码生成等模型管理工作。图 4.11 实现指标验证功能的工具设计方案展示了指验证功能集成在 MetaGraph 工具内的设计方案。其中，黑框表示用户界面，蓝框表示工具其他模块，红框表示功能。如图 4.11 实现指标验证功能的工具设计方案所示，指标验证模块和建模模块共同使用 KARMA 语言作为描述方法，ANTLR 语法指标验证模块支持所有 KARMA 语言的解析，基于可满足性模理论的求解器支持指标验证模块的构建。指标验证模块具备文本输入、完成验证、结果展示和结果保存四种基本功能。

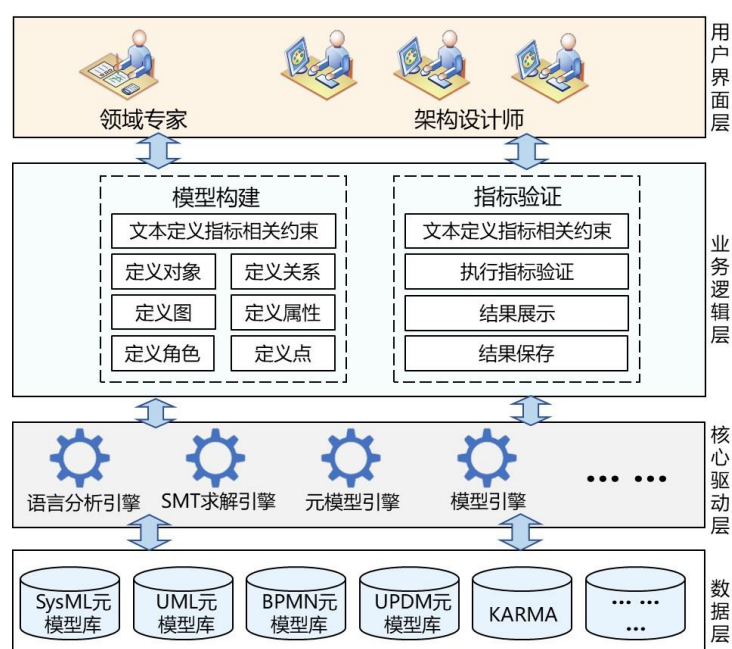


图 4.11 实现指标验证功能的工具设计方案

指标验证实现过程如图 4.10 指标验证实现方法所示。指标验证首先构建基于 KARMA 语言的指标验证文本。指标验证方法先从模型层提取模型元素，再结合待验证问题，形成面向模型元素的一阶逻辑表达式约束，将构建约束的文本输入进语言编译器。进入语法编辑器的语言形成抽象语法树，在相应的节点调用基于可满足性模理论的求解器，检验约束的可满足性，最终输出相应的结果。

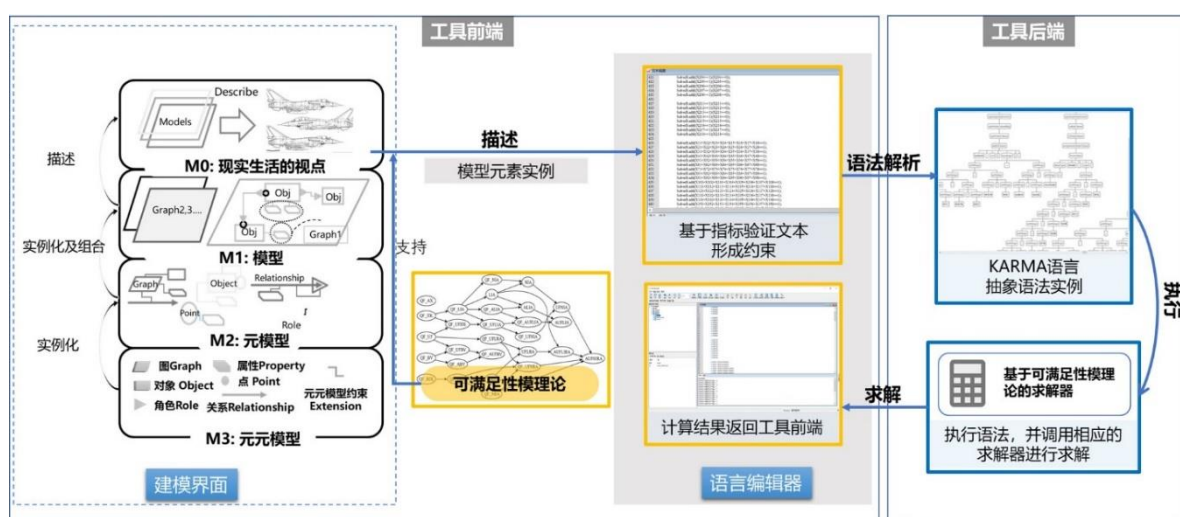


图 4.10 指标验证实现方法

图 4.12 KARMA 指标验证文本输入界面表示 KARMA 语法的代码输入界面，包含代码输入、资源导航和控制栏。

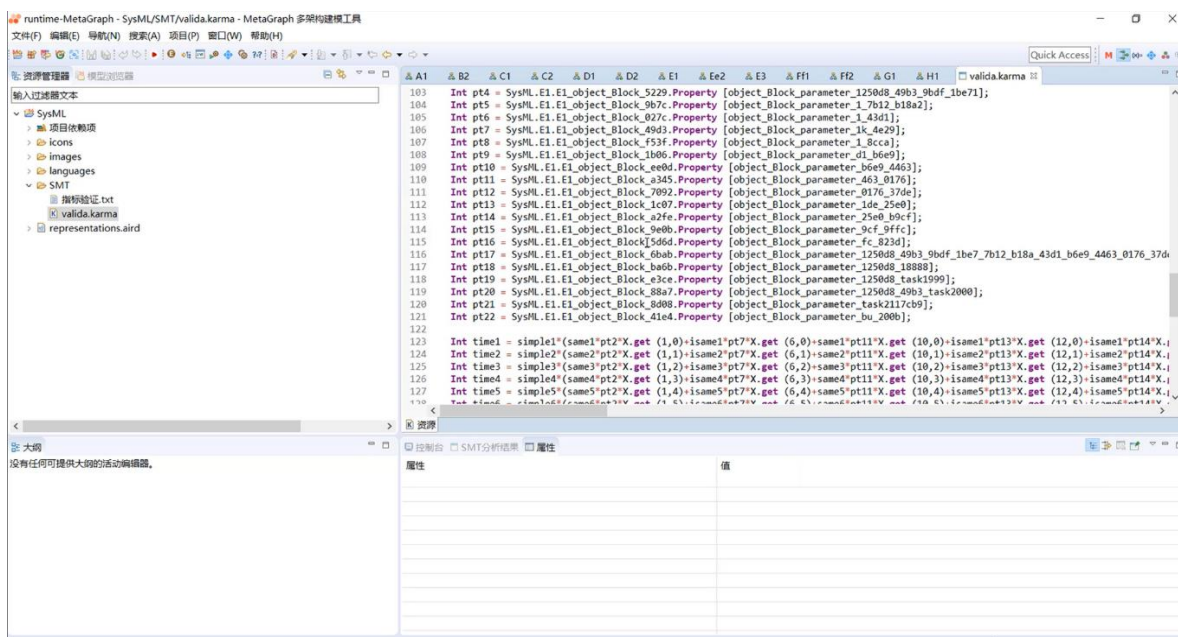


图 4.12 KARMA 指标验证文本输入界面

4.4 本章小结

本章首先分析 KARMA 语言和 SMT-LIB 语言的语法和语义，形成基于 KARMA 语言的指标验证语言规划。其次，根据语言规划定义 KARMA-指标验证语言具体语法。最后，根据定义语法构建基于 ANTLR 的语言编译器，实现指标验证功能，并集成在统一建模软件 MetaGraph 中。本章所提出的指标验证语言支持构建异构模型、形式化描述和验证指标与模型架构之间的约束，保证异构建模、指标形式化描述和验证的语言一致性。同时，采用基于可满足性模理论的求解器保证约束构建和验证的理论一致性。

第5章 面向卫星姿态控制系统设计的案例研究

本章针对前两章提出的指标形式化和验证方法，以卫星的设计过程为验证对象，验证本文所提出的指标验证方法的有效性。首先，本章依据“需求定义—功能架构—逻辑架构—物理架构”的系统分析流程，定义卫星的总体架构模型。并依据卫星的总体架构模型，定义相应的卫星指标体系。最后，选取卫星设计过程中姿态控制系统的逻辑单元选型问题、姿控系统逻辑架构可靠性指标验证问题作为验证对象，评估本文所提出的指标验证方法在设计方案确认和静态指标评估问题上的可行性。

5.1 卫星设计问题

卫星，特指人造卫星，是环绕地球在空间轨道上运行的无人航天器，一般用于科学实验、技术试验或应用。卫星典型的复杂系统，其结构复杂、零部件交互复杂、精度要求高，它包含了有效载荷分系统、电源分系统、位置和姿控分系统、推进分系统、遥测指令分系统、总体电路分系统、机构与结构分系统和热控分系统。



图 5.1 卫星示意图

传统的卫星设计采用串型式、文档式的设计方式。各分系统之间的设计和管理相对比较封闭，设计方式主要依靠文件方案和图纸设计，同时借助多种设计工具。而部门间沟通的方式，以及设计的确认和更迭主要依赖于电话、文件之类的文字沟通，这主要导致了以下三个问题：1) 卫星设计过程涉及众多利益相关方，其配合关系复杂，

沟通成本高。若仅通过自然语言描述传递的信息，难以完整表述真实的设计的全过程，且由于设计变更、设计模糊，可能产生信息延迟、歧义等问题。2) 卫星设计过程存在多种异构数据源，其数据与信息缺乏统一的表达，难以对设计过程的数据进行集成和重用。3) 缺乏设计卫星系统设计全周期的系统级的验证和确认。确认卫星设计是否满足用户需求通常在卫星的物理结构设计阶段，验证和确认的不及时会导致问题发现的延迟大大降低，降低设计效率。验证和确认的对象层级一般也较细微，如：零件的校核、部件的仿真等，如果要进行部件的组合确认，其操作较为复杂。因此，将确认和验证迁移，同时支持多层级卫星系统的验证和确认是必要的。为了应对上述挑战，本章采用 MBSE 方法，创新卫星设计方式，以模型为核心结合指标验证的手段，应对卫星设计的跨域管理、异构数据、多层验证的需求。

本文以遥感卫星为例，其总体任务为“设计一款位于中高轨道的能够监视、定位、传输森林火情的遥感监测卫星”^[67]。基于该任务，本章提取卫星总体需求，细化卫星设计方案。以卫星的姿态控制系统为研究对象，使用指标验证方法面向逻辑单元进行设计方案确认、验证姿态控制系统的可靠性指标，从而明确指标验证方法在设计方案确认问题^[68]和静态指标验证问题领域的可行性。

5.2 卫星设计语言和工具

本案例依据“需求定义(R)—功能架构(F)—逻辑架构(L)—物理架构(P)”系统分析流程进行卫星设计。卫星设计的模型库采用 SysML 模型库，首先，基于 KARMA 多架构建模语言的扩展 SysML 库，使得扩展后的 SysML 库满足卫星设计的需求。再使用指标验证语言实现指标形式化表达和验证指令定义。本案例使用面向 KARMA 语言开发的多架构统一建模平台 MetaGraph 进行建模、指标验证。

KARMA 多架构建模语言支持 SysML 语言元模型库的构建，同时根据卫星的特殊领域扩展了模型元素的属性，用以构建更完善的模型。SysML 语言是一种面向对象的建模语言，它从系统角度进行设计，通过不同的图描述系统的结构和功能等^[39]。KARMA 语言依据 SysML 标准，基于图、对象、点、角色、关系、属性六种关系构建了 SysML 模型库内的 9 种图元模型、以及 84 个对象元模型、12 种点元模型、93 种属性元模型、32 个关系元模型、33 个角色元模型，同时支持 UML、Capella、UPDM 的模型库构建。SysML 建模语言在系统工程领域的特定域语言，支持描述系统的多个视点。该语言包含三大类：用于描述系统功能和状态的行为图、描述需求的需求图、

描述系统组成的实体及其之间关系的结构图。其中，行为图包含描述功能流动的活动图（Activity Diagram, Act）、描述信息传递的序列图（Sequence Diagram, Seq）、描述事件状态变化的状态机图（State Machine Diagram, Stm）、描述系统所能提供的服务的用例图（Use case Diagram, UC），结构图包含描述模块单元组合的模块定义图（Block Definition Diagram, BDD）、描述模块内部关系的内部模块图（Internal Block Diagram, IBD）、描述系统具体参数状态的参数图（Parameter Diagram, Par）、描述各类模型元素层级关系的包图（Package Diagram, Pkg）。

MetaGraph 是面向 KARMA 多架构统一建模语言建模平台，该平台支持异构模型构建、模型转换等功能，本文所开发的指标验证功能也集成在该工具内。该工具是基于 Eclipse 构建的建模平台，包含资源导航边栏、建模界面、KARMA 语言编辑界面，以及控制台界面。资源导航边栏能够显示项目包含的模型、KARMA 语言的树状结构，建模界面支持模型的构建，KARMA 语言编辑界面支持 KARMA 语言的编辑从而支持模型的形式化、模型转换、指标验证等功能，控制台则负责输出相关的错误或结果。

5.3 卫星总体架构模型构建

本案例依据“需求定义（R）—功能架构（F）—逻辑架构（L）—物理架构（P）”定义构建模型，建模流程如下图 5.2 卫星模型构建流程所示。

1) 需求定义：首先使用用例图或活动图定义卫星的全生命周期活动，并用用例图明确相关的利益相关人，列出利益相关人。并利用需求图根据利益相关人关注的内容总结卫星顶层需求。同时根据顶层需求定义效能指标。

2) 功能架构: 使用用例图以及活动图根据卫星的顶层需求, 提取相应的顶层功能。再使用模块图根据顶层功能划分支持实现这些活动的分系统。面向这些分系统, 结合卫星运行机理, 使用用例图或者活动图识别分系统带实现的功能; 再将待实现的功能使用模块定义图识别为功能模块, 同时将派生需求细化为功功能模块的性能和功能需求, 汇总至需求定义的细化需求图, **形成面向功能架构的功能和非功能指标**。再构建功能模块之间的功能流图, 明确功能单元之间的信息或物质的传递。同时进行需求确认, 在功能组成复杂的情况下进行功能验证。最后, 输出定义功能模块和功能流的架构模型以及各功能的性能和功能需求。

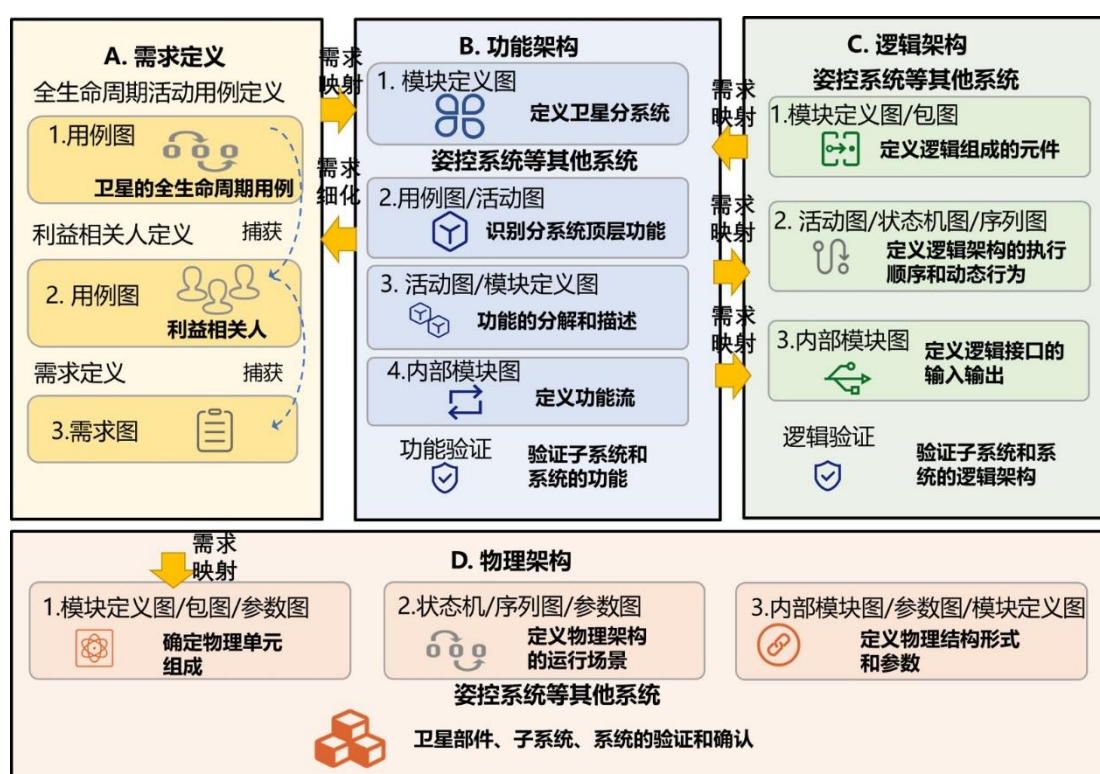


图 5.2 卫星模型构建流程

3) 逻辑架构: 分系统根据定义的功能架构模型与相关派生需求, 根据功能单元识别能够实现功能的逻辑单元, 并选择符合需求的逻辑单元, 在必要时进行设计方案确认。该过程根据模块定义图或者包图进行定义。同时, 细化面向功能架构的性能和功能需求, 得到面向逻辑架构的需求, **形成面向逻辑架构的功能和非功能指标**。定义能够实现性能和功能需求的相关技术需求, 汇总至细化需求图, **形成技术指标**。再定义新的活动图明确各逻辑单元的执行顺序, 再定义状态机图或者序列图构建逻辑单元之间状态变化和传递等动态图, 并基于内部模块图定义逻辑单元内部的信息传递。

最后,针对分系统逻辑架构以及整体逻辑架构进行验证,并对多重技术需求进行综合。最终输出可行的逻辑架构以及技术指标需求。

4) 物理架构:分系统根据逻辑架构和技术指标需求定义物理单元的组成,并通过模块定义图定义由所有可行的物理单元组成待选方案,进行设计方案确认,并通过状态机图、序列图、活动图定义这些待选方案的动态变化,再通过内部模块图、参数图、模块定义图明确物理结构的关系和参数。最后通过指标验证、动态仿真等多种方法判断物理架构在各种可能条件下是否符合顶层需求。

5.4 模型描述

本节使用 SysML 模型库实践图 5.2 卫星模型构建流程的需求定义部分,并基于系统分析流程构建符合指标体系。

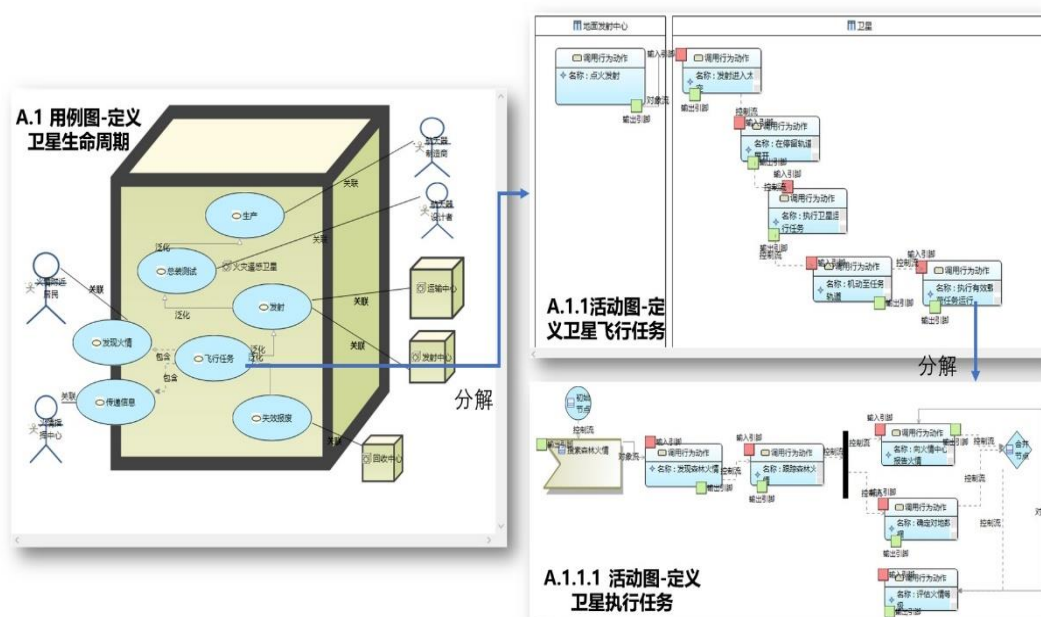


图 5.3 卫星全生命周期用例图及细化飞行任务活动图

首先定义用于定位森林火灾卫星的全生命周期用例 A.1，如图 5.3 所示。A.1 全生命周期用例使用用例图元模型，其整体用例包含：生产、总装和测试、发射和部署、飞行任务、失效。五大用例之间使用关系用例“关联”连接。其中，飞行用例包含发现火情用例、传递火情信息用例两个细化用例。这些用例定位了相关的主要利益相关人：航天器设计制造人员、航天器设计测试人员、运输中心、发射中心、报废中心以及火情附近居民和火情指挥中心。除此以外，利益相关人还包括科技部、赞助人等相关单位，如图 5.3 卫星全生命周期用例图及细化飞行任务活动图所示。其中飞行用例可以被分解为更详细的活动图 A.1-1，飞行任务首先发射，进入停留轨道后展开，执行卫星运行任务、进入任务轨道、开展火情监测任务。动作模块“开展火情监测任务”可以继续被分解，为搜索森林火灾、继而发现火灾，继续跟踪火灾，并且同时报告火灾情况给火情指挥中心以及确定火灾对地定位顺序，继续分析评估火灾情况，报告火灾详情。

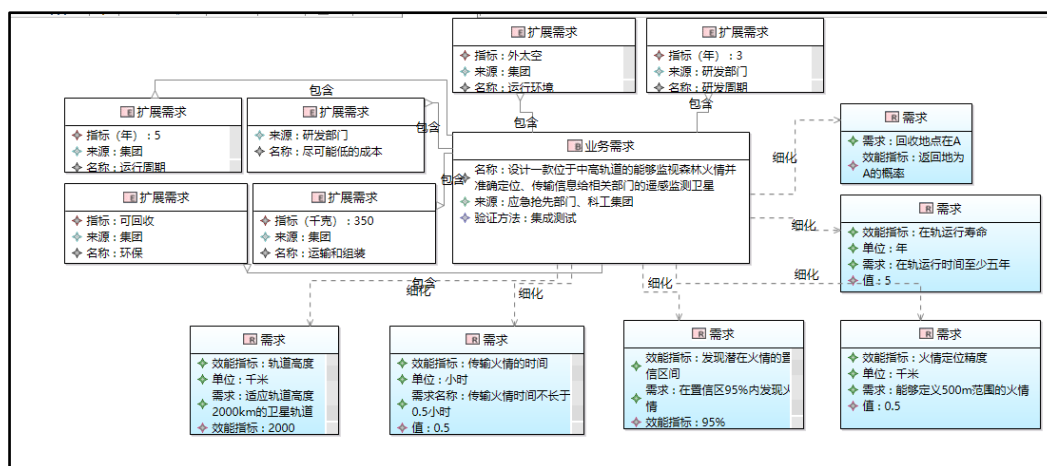


图 5.4 A.3 总体需求和指标图

根据上述全生命周期活动的分析明确其任务边界，即：任务卫星根据全生命周期用例、活动、以及利益相关人的需求明确整体需求：0) 卫星发射、飞行、回收任务的有效性；1) 位于 2000km 的中高轨道；2) 探测置信区间为 95% 的潜在火情；3) 至少每 24 小时完成一次覆盖区域的全检测；4) 卫星应在 30min 内传递 500m~5000m 定位精度的火情；5) 运行寿命为 5 年；6) 低于 350kg 和 7) 明确其可靠性、可安装性、地面中心、回收地点以及尽可能低的预算。如图 5.4 所示的 A.3 卫星总体需求图。

根据卫星的全生命周期活动分析以及利益相关人的需求定义，结合系统边界，从卫星的顶层需求和用例中抽象出实现这些需求和用例功能的模块，如图 5.5 所示：

- (1) 姿控与位置分系统：卫星能够在轨道上以正确姿态完成卫星任务

(2) 遥测控制分系统：监控自身设备状态数据并发送数据给地面，同时接受来自地面的指令并发送指令控制其他分系统。

(3) 推进分系统：为卫星提供动能、保障卫星发射、入轨、调姿。

(4) 电源分系统：为卫星提供电能

(5) 热控分系统：保障太空内卫星内外温度

(6) 指令数据分系统：处理卫星数据并形成指令

(7) 有效载荷分系统：有效载荷是所有卫星执行任务系统的统称，对于遥感卫星，有效载荷分系统执行遥感成像或探测功能，一般是成像仪或合成孔径雷达等。

(8) 结构分系统：卫星受力和支撑件的统称。

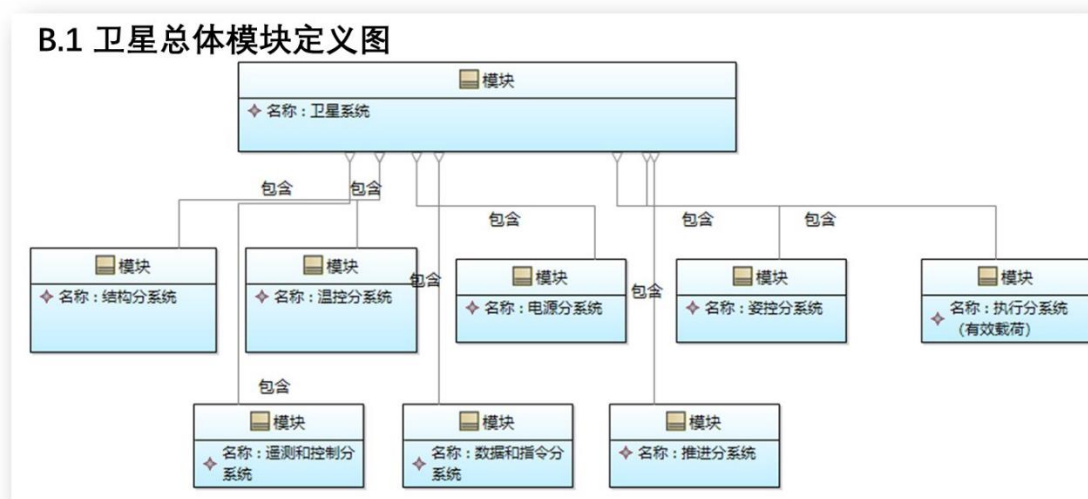


图 5.5 卫星总体模块定义图

表 5.1 根据顶层需求定义的效能指标及其细化

效能指标	细化指标	相关分系统
轨道高度	轨道高度 h	姿控系统、电源系统，推力系统， 遥测系统，有效载荷分系统
	信号孔径尺寸 a	有效载荷分系统
置信度	光谱分辨率 λ 空间分辨率 r	有效载荷分系统
持久覆盖	轨道高度 h , 轨道倾角 α , 平台仰角 β ,	姿控系统、推力系统、有效载荷分
	有效载荷功率 w_v	系统、电源分系统

续表

及时定位	地面站位置 A ，传输速度 g	遥测控制分系统、指令数据分系统
	轨道姿态精度 θ 、测算误差 δ 、无敏	<u>姿控分系统</u> 、遥测分系统、指令数
	捷机动性能	据分系统
运行寿命	电源选择，功率 w	电源分系统
	轨道高度 h	<u>姿控系统</u> 、电源系统，推力系统， 遥测系统，有效载荷分系统
	各分系统寿命	所有系统
质量	功率 w	电源系统
	下放至各分系统	<u>所有分系统</u>
可靠性	固有频率 f ，随机振动，静态载荷，材	结构分系统
	料选择	
	温度控制 t	温控系统
	技术管理，零部件供给，测试要求， 应急处理能力、结构鉴定、接口管 理，数据管理等	<u>所有分系统</u>
可安装性	布局合理性	结构分系统
	体积	<u>所有分系统</u>
成本	下放至各系统	<u>所有分系统</u>

面向各分系统，顶层需求依据机理被细化为派生需求。顶层需求对应的效能指标和派生需求定义的功能和非功能指标如表 5.1 根据顶层需求所示，表中字母是评估指标的预期值

5.5 指标体系构建

本章面向卫星的总体设计流程，构建相应的指标体系。指标体系的建立结合卫星设计过程，以姿态控制系统为重点研究对象，说明基于 RFLP 流程构建的多维多层指标体系。

如图 2.10 面向卫星设计过程的多维多层指标体系所示，首先依据 RFLP 设计流

程,明确设计任务,并在此基础上,分析其顶层需求和对应的效能指标,根据顶层需求,将卫星分为八个模块,并分析各模块的功能,定义出相应的功能模块。根据功能模块,细化效能指标,分解为功能模块的功能和非功能指标。这个过程中,功能层级逐渐缩小,从子系统级缩小至部件级,至零件级等,最终得到独立的逻辑单元。再根据逻辑单元细化进一步细化功能架构的功能和非功能指标,得到逻辑单元的功能和非功能指标。最终,根据逻辑单元提取面向物理单元的设计需求,得到面向物理单元的技术指标。该过程在建模软件内完成。

图 5.4 是顶层需求和效能指标的模型化，总体需求图包含了需求和相应指标，如需求为卫星位于中高轨道，则其评估指标为支持的轨道高度，预期值为 2000，单位为千米。依据总体需求划分如图 5.5 卫星总体模块定义图定义的卫星总体模块图。针对姿态控制系统细化效能指标，得到如图 5.6 所示的功能和非功能指标。以可安装性为例，需求为保证姿态控制系统的紧配合，指标为安装精度，值为 0.05，单位为毫米。

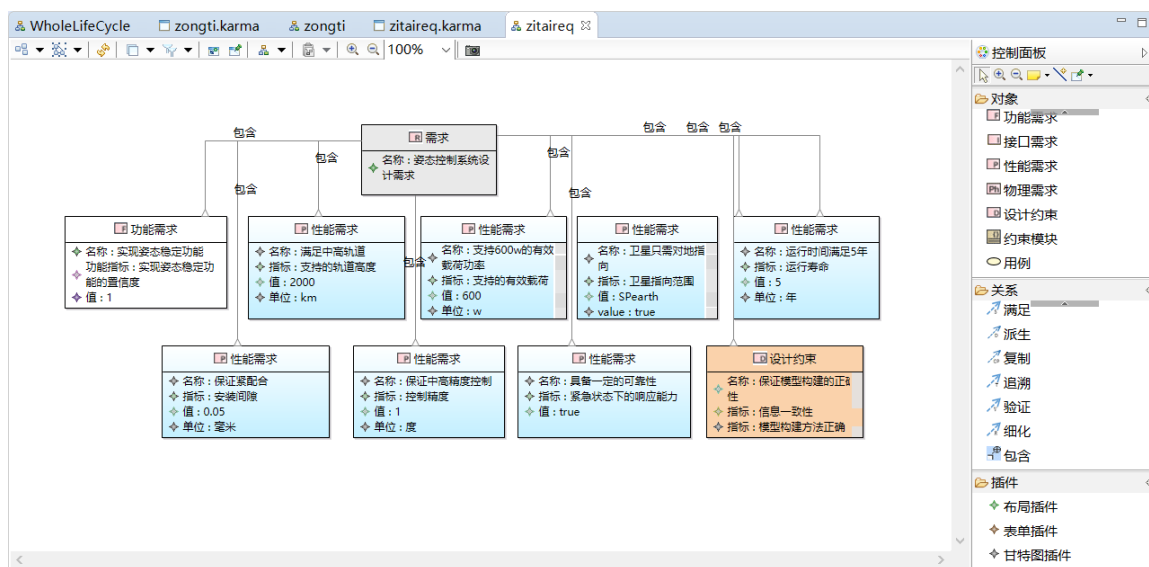


图 5.6 姿态控制系统的功能和非功能指标

5.6 指标验证

本节使用指标验证的方法实现卫星姿控系统的逻辑单元选型，并验证姿控系统的逻辑架构设计能否满足可靠性指标。

5.6.1 面向卫星姿控系统逻辑单元的设计方案确认

从上文建模流程图 5.2 中可以看出, 复杂系统的功能架构、逻辑架构和物理架构

的定义都存在一定的设计选项的方案确认问题。本节聚焦卫星姿态控制分系统的逻辑架构定义，通过指标验证方法解决姿控系统逻辑单元的选型探索问题。

5.6.1.1 问题陈述

如图 5.6 的指标体系表 5.1 根据顶层需求可知，姿态控制系统设计的需求包括控制指向精度为小于 0.5° ，要求卫星平台保持对地指向，无敏捷机动要求，位于 2000km 的中高轨道，有效载荷功率为 600w，工作寿命长于 5 年等。依据需求，定义相关的评估指标：控制精度、对地指向能力、敏捷机动能力、轨道高度、载荷功率最大值。为了满足姿态控制的需求，实现控制姿态的功能，首先定义姿态控制系统的功能图，将姿态控制功能依据功能流的分解将姿态控制功能进行划分。

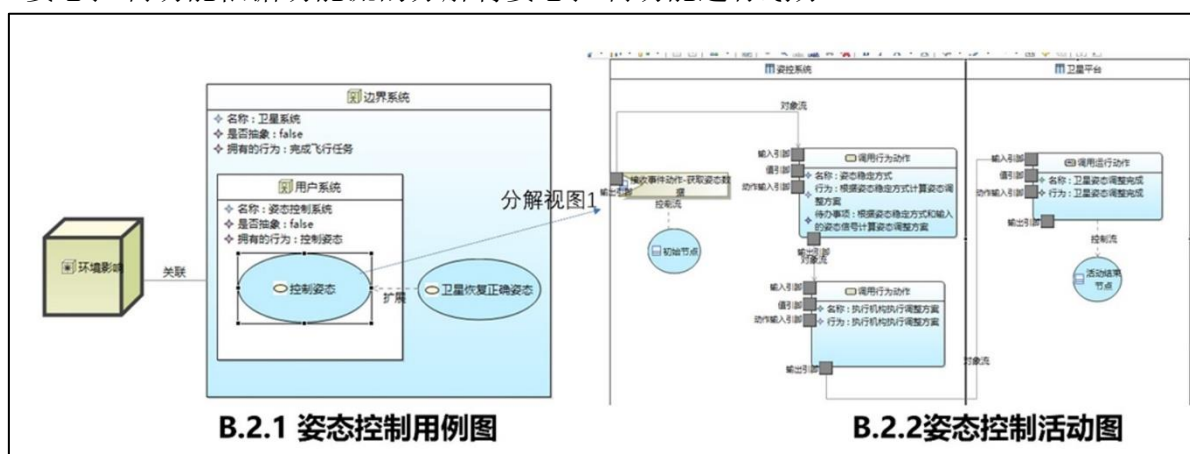


图 5.7 姿态控制系统的用例图及分解的活动图

如图 5.7 所示，根据姿态控制系统的用例图得知，姿控系统输入的是卫星目前的姿态数据，输出的是调整后的卫星姿态。为了完成该用例，将该用例分解为相应的活动图，活动图展示完成获取数据、传递数据、根据姿态稳定方式计算调整程度、完成调整。根据活动图，映射出实现功能对应的功能单元，确定姿态控制系统首先需要根据总布置确定具体的卫星姿态控制方式，再确定传输数据的姿态敏感器，以及调整卫星姿态的执行机构，得到姿态控制的模块定义图，如图 5.8 姿态控制模块定义图所示。

根据图 5.6 所示面向姿控系统的指标，进一步形成面向每个逻辑单元的功能和非功能指标，即：分别面向姿态稳定方式、姿态敏感器、姿态执行机构的控制精度、轨道高度、有效载荷功率、倾角支持对地、敏捷机动性指标。

根据图 5.6 的定义的需求和指标，细化卫星姿态控制系统，得到卫星姿态控制系统的逻辑架构。首先根据设计约束进行逻辑单元的选型匹配。该过程需使用指标验证

功能对多种候选设计选项进行探索。根据卫星姿控模型库可知，姿态稳定方式主要有三轴稳定、双自旋稳定、重力梯度三轴稳定；姿态敏感器（Attitude Sensor）主要包含太阳敏感器、陀螺组件、磁强计、地球敏感器、加速度计、星敏感器；执行机构包含零动量轮空系统、控制力矩陀螺、伸杆电机、推力器、磁力矩器、偏置动量轮控系统、章动阻尼器。他们具有不同的属性，不同的应用条件，具体如表 5.2 卫星常见姿态稳定方式的特性、表 5.3 姿态敏感器类型及相关特性、表 5.4 姿态控制系统执行机构特性所示。其中，控制精度是指姿控系统对卫星平台的控制精度，成本等级指的是姿控稳定方式的开发成本，其中等级数目越高代表成本越高，所以三轴稳定较其他两种方法而言，三轴稳定的成本最高。



图 5.8 姿态控制模块定义图

表 5.2 卫星常见姿态稳定方式的特性

姿态稳定方式	最高控制精度	敏捷机动	有效载荷功率是否无限制	轨道是否无限制	空间指向是否任意	成本等级
A. 三轴稳定	0.1° ~0.01°	true	true	true	true	2
B. 重力梯度三轴稳定	0.3°	false	true	false（小于1000km）	false（只能对地指向）	1

续表

			false			
C. 双自旋稳定	0.3°	false	(500w~1000w)	true	true	1

表 5.3 姿态敏感器类型及相关特性

姿态敏感器	使用精度和目的	是否不受轨道影响	适配稳定方式	备注
1.地球敏感器	($>0.1^\circ$) 提供姿态信息	true	all	功能类似, 成本
2. 星敏器	($<0.1^\circ$) 提供姿态信息	true	A	随着及精度的升高而升高
3.磁强计	($>1^\circ$) 提供姿态信息	false (1000km 以内的近地轨道)	all	和 5 号敏感器配合
4.加速度计	主动章动控制、摇摆角控制的测量	true	C	如果使用 C 方式, 必须使用
5.太阳敏感器	($>0.5^\circ$) 卫星初始姿态捕获	true	all	不可以单独使用
6.陀螺组件	($>0.1^\circ$) 提供各类姿态信息	true	A,B	和 3, 1 号敏感器配合使用

表 5.4 姿态控制系统执行机构特性

执行机构	精度和目的	是否不受轨道影响	适用方式
a.控制力矩陀螺	敏捷机动功能实现, 需要卸载	true	A
b.零动量轮空系统	($<0.1^\circ$), 需要卸载	true	A
c.章动阻尼器	快速稳定	true	B,C
d.偏置动量轮控系统	($>0.1^\circ$), 需要卸载	true	A,B
e.推力器	大幅度机动 ($\approx 0.5^\circ$)	true	all
f.伸杆电机	稳定姿态	true	B
g.磁力矩器	控制推力机动 ($>1^\circ$)	false (1000km 以内)	all

依据上述姿态稳定方法、姿态敏感器和执行机构的定义，构建相应的模块图，如图 5.9 逻辑单元选型-模块图所示，进行筛选。从图中可以看出，各个模块可以分解为相应的模块定义图，每个逻辑单元有多种实现方法，此时需要根据需求进行分析评估，从而在设计空间中选择符合约束的。

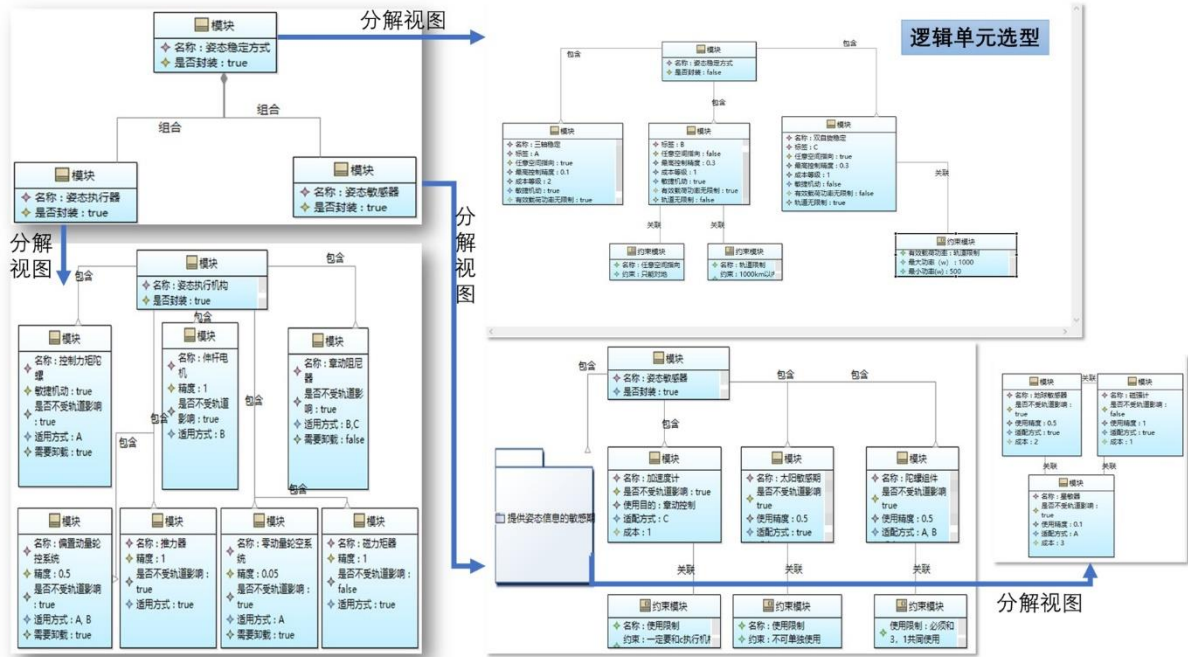


图 5.9 逻辑单元选型-模块图

5.6.1.2 姿态控制方式的约束条件定义

上述模型基于 SysML 模型库构建了姿态控制系统需求架构，功能架构以及逻辑架构中的逻辑单元的设计空间。为了找到满足姿控系统需求的逻辑单元，需要判断逻辑单元是否能够符合姿态控制单元的需求，所以需要分别定义姿态控制单元的指标，以及各个逻辑单元自身的属性约束。本节通过指标验证的方式判断哪种逻辑单元能够符合姿态控制系统的需求。

(1) 设计指标的形式化表达

根据卫星姿控系统设计的需求，可知姿态控制系统相关的需求包含精度、空间指向、敏捷机动要求、轨道要求、有效载荷功率要求。可以列为式 5.1~5.6。其中， P_{target} 表示控制指向精度，单位为度， $SP_{target}(earth)$ 表示对地指向， AG_{target} 表示

敏捷机动要求, $height_{target}$, 表示卫星目标高度, 单位为千米, $payLoad$ 表示有效载荷, 单位为瓦。式 5.1 说明控制精度的目标为 0.5° ; 式 5.2 说明能够实现空间指向对地范围; 式 5.3 表明不需要支持敏捷机动性; 式 5.4 表明卫星目标高度为 2000km; 式 5.5 表明卫星携带的有效载荷功率为 600w; 式 5.6 表明设计目标包含最小化成本。表达式前的 Req_{num} 表示表达式的标号。

$$Req_1 : P_{target} = 0.5; \quad (5.1)$$

$$Req_2 : SP_{target}(earth) = true; \quad (5.2)$$

$$Req_3 : AG_{target} = false; \quad (5.3)$$

$$Req_4 : height_{target} = 2000 \quad (5.4)$$

$$Req_5 : payLoad = 600 \quad (5.5)$$

$$Req_6 : \min(Cost) \quad (5.6)$$

(2) 各待选设计选项表达

以姿态稳定方式为例, 构建三种姿态稳定方式的属性表达方法。每个姿态稳定方式都包含相同的属性, 具有不同的属性值。令待选的稳定方式为 i , 将三轴稳定方式、重力梯度三轴稳定方式、以及双自旋稳定方式定义为标号 A、B、C, 则 $i \in \{A, B, C\}$, 符号 $\in$ 表示属于, 即: 待选稳定方式一定是三者之一。稳定方式是否被选择使用 $Using_i$ 表示, 若某种稳定方式被选择, 则 $Using_i$ 的值为“真”。已知三种稳定方式有六种主要属性: 控制精度 P_i , 敏捷机动性 AG_i , 功率载荷是否无限制 $payLoad_i(\infty)$, 轨道是否无限制 $height_i(\infty)$, 空间指向是否无限制 $SP_i(\infty)$, 成本等级为 $cost_i$, 其中下标表示姿态稳定方式的类型, 取值为 A、B、C。括号内 ∞ 表示无限制、任意情况, $payLoad_i(\infty)$ 中的 ∞ 表示任意大小的载荷、 $height_i(\infty)$ 表示任意高度的轨道、 $SP_i(\infty)$ 表示任意指向。若某种方式被选择, 则如式 5.7 所示:

$$\begin{aligned}
Constranit_1: & \left((i == A) \& \& (P_i == P_A) \& \& (AG_i == AG_A) \& \& (payLoad_i(\infty) = \right. \\
& = payLoad_A(\infty)) \& \& (height_i(\infty) == height_A(\infty)) \& \& (SP_i = \\
& = SP_A) \& \& (cost_i = \\
& = cost_A) \left. \right) \vee \left((i == B) \& \& (P_i == P_B) \& \& (AG_i = \right. \\
& = AG_B) \& \& (payLoad_i(\infty) == payLoad_B(\infty)) \& \& (height_i(\infty) \quad (5.7) \\
& == height_B(\infty)) \& \& (SP_i == SP_B) \& \& (cost_i == cost_B) \left. \right) \vee \left((i \right. \\
& == C) \& \& (P_i == P_C) \& \& (AG_i == AG_C) \& \& (payLoad_i(\infty) = \\
& = payLoad_C(\infty)) \& \& (height_i(\infty) == height_C(\infty)) \& \& (SP_i = \\
& = SP_C) \& \& (cost_i == cost_C) \left. \right)
\end{aligned}$$

结合上述属性，构建三种稳定方式的属性约束表达式。三轴稳定的控制精度、敏捷机动性、功率载荷无限制、轨道是无限制、空间指向是否无限制、成本等级的约束表达式如式 5.8~5.13。

$$Pro_{A,1} : 0.01 < P_A < 0.1 \quad (5.8)$$

$$Pro_{A,2} : AG_A == true \quad (5.9)$$

$$Pro_{A,3} : payLoad_A(\infty) = true \& \& (payLoad_A(\infty) == true) \Rightarrow (payLoad_A \leq 12000) \quad (5.10)$$

$$Pro_{A,4} : height_A(\infty) == true \& \& (height_A(\infty) == true) \Rightarrow (height_A \leq 35786) \quad (5.11)$$

$$Pro_{A,5} : SP_A(\infty) == true \& \& (SP_A(\infty) == true) \Rightarrow (SP_A(earth) == true) \quad (5.12)$$

$$Pro_{A,6} : cost_A = 2 \quad (5.13)$$

其中， P_A 表示三轴稳定方式的控制精度， AG_A 的值为“真”表示三轴稳定方式能够实现敏捷机动，关系前的 $Pro_{i,num}$ 是为了简便表达冒号右边的一阶逻辑表达式的简要标号，式中的“&&”表示“与”关系。式 5.10 表示三轴稳定方式能够支持无限制有效载荷功率，“ $\Rightarrow$ ”符号表示逻辑表达式的蕴含关系，表示“如果...那么..”的关系，表明支持无限制载荷功率就能够支持有效载荷功率小于 12000w，12000w 为已知的卫星有效载荷最大功率。式 5.11 表示三轴稳定方式支持无限制高度轨道，且只要支持无线高度轨道，就一定支持最高地球静止轨道（GEO 轨道，高度为 35786km）的运行。式 5.12 表示三轴稳定能够支持空间任意指向，且只要能够支持空间任意指向就可以满足支持指向地球。式 5.13 表示三轴稳定的成本等级为“2”，等级越高，则成本越高。

重力梯度三轴稳定方法的控制精度、敏捷机动性、功率载荷无限制、轨道是无限

制、空间指向是否无限制、成本等级的约束表达式如式 5.13~5.18。

$$Pro_{B,1} : P_B \geq 0.3 \quad (5.13)$$

$$Pro_{B,2} : AG_B == false \quad (5.14)$$

$$Pro_{B,3} : payload_B(\infty) = true \&\& (payload_B(\infty) == true) \Rightarrow (payload_B \leq 12000) \quad (5.15)$$

$$Pro_{B,4} : height_B(\infty) == false \&\& height_B \leq 1000 \quad (5.16)$$

$$Pro_{B,5} : SP_B(\infty) == false \&\& SP_B(earth) == true \quad (5.17)$$

$$Pro_{B,6} : cost_B = 1 \quad (5.18)$$

其中, 式 5.16 表示重力梯度三轴稳定不支持高度无限制的轨道, 且最高高度为 1000km。式 5.17 表示重力梯度不能满足任意空间指向, 但是可以实现对地指向。

双自旋稳定方法的控制精度、敏捷机动性、功率载荷无限制、轨道是无限制、空间指向是否无限制、成本等级的约束表达式如式 5.19~5.24。其中, 式 5.21 表示双自旋稳定不支持功率无限制的有效载荷, 且最高功率为 1000 瓦。

$$Pro_{c,1} : P_c \geq 0.3 \quad (5.19)$$

$$Pro_{c,2} : AG_c == false \quad (5.20)$$

$$Pro_{c,3} : payload_c(\infty) = false \&\& payload_c \leq 1000 \quad (5.21)$$

$$Pro_{c,4} : height_c(\infty) == true \&\& (height_c(\infty) == true) \Rightarrow (height_c \leq 35786) \quad (5.22)$$

$$Pro_{c,5} : SP_c(\infty) == true \&\& (SP_c(\infty) == true) \Rightarrow (SP_c(earth) == true) \quad (5.23)$$

$$Pro_{c,6} : cost_c = 1 \quad (5.24)$$

根据式 5.7 可知, 待选方案一定是三者之一, 构建式 5.25~式 5.30 的指标和待选设计项之间的关系, 说明待选方案还必须符合目标约束, 并且满足成本最小约束, 从而排除不符合需求的稳定方法。

$$P_{target} == P_i; \quad (5.25)$$

$$SP_{target}(earth) == SP_i(earth); \quad (5.26)$$

$$AG_{target} = AG_i; \quad (5.27)$$

$$height_{target} = height_i; \quad (5.28)$$

$$payload == payload_i \quad (5.29)$$

$$Min(cost_i) \quad (5.30)$$

5.6.1.3 实现方法及姿态稳定方式结果

本案例使用本文提出的指标验证方法评估各方案是否满足指标，从而排除不符合约束条件的，且最满足需求的姿态稳定方法。首先提取待选模型的属性，构建关于待选项和指标之间的约束，如图 5.10 所示。

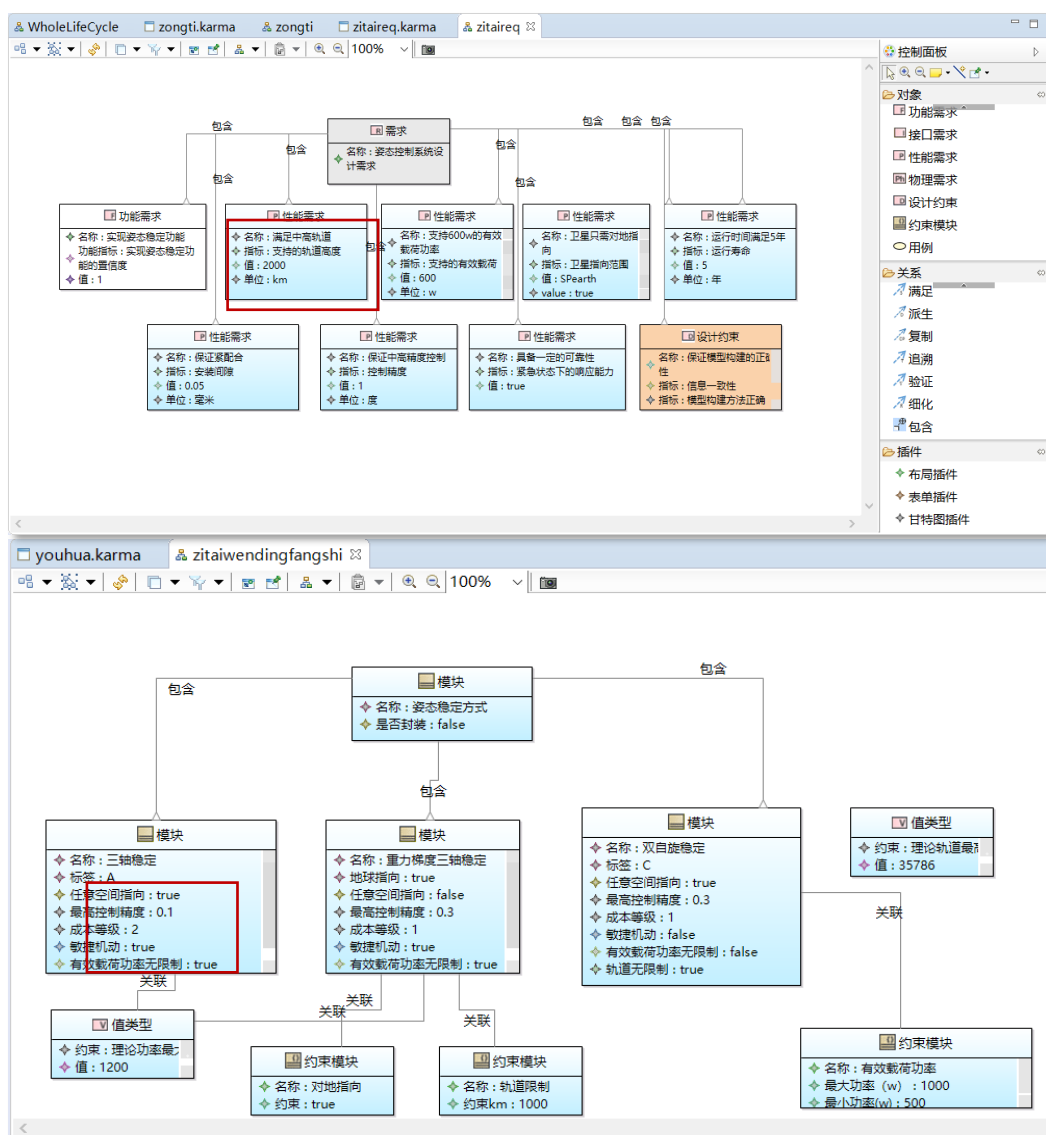


图 5.10 提取模型属性

图 5.11 说明了指验证的方式，首先在文本编辑界面定义相关的指标文本，具体代码见面向逻辑单元设计方案确认问题的指标验证代码所示。其次，选择指标验证按钮，选择待验证的文本名称，进行验证。最后得到如图 5.11-③所示的结果。

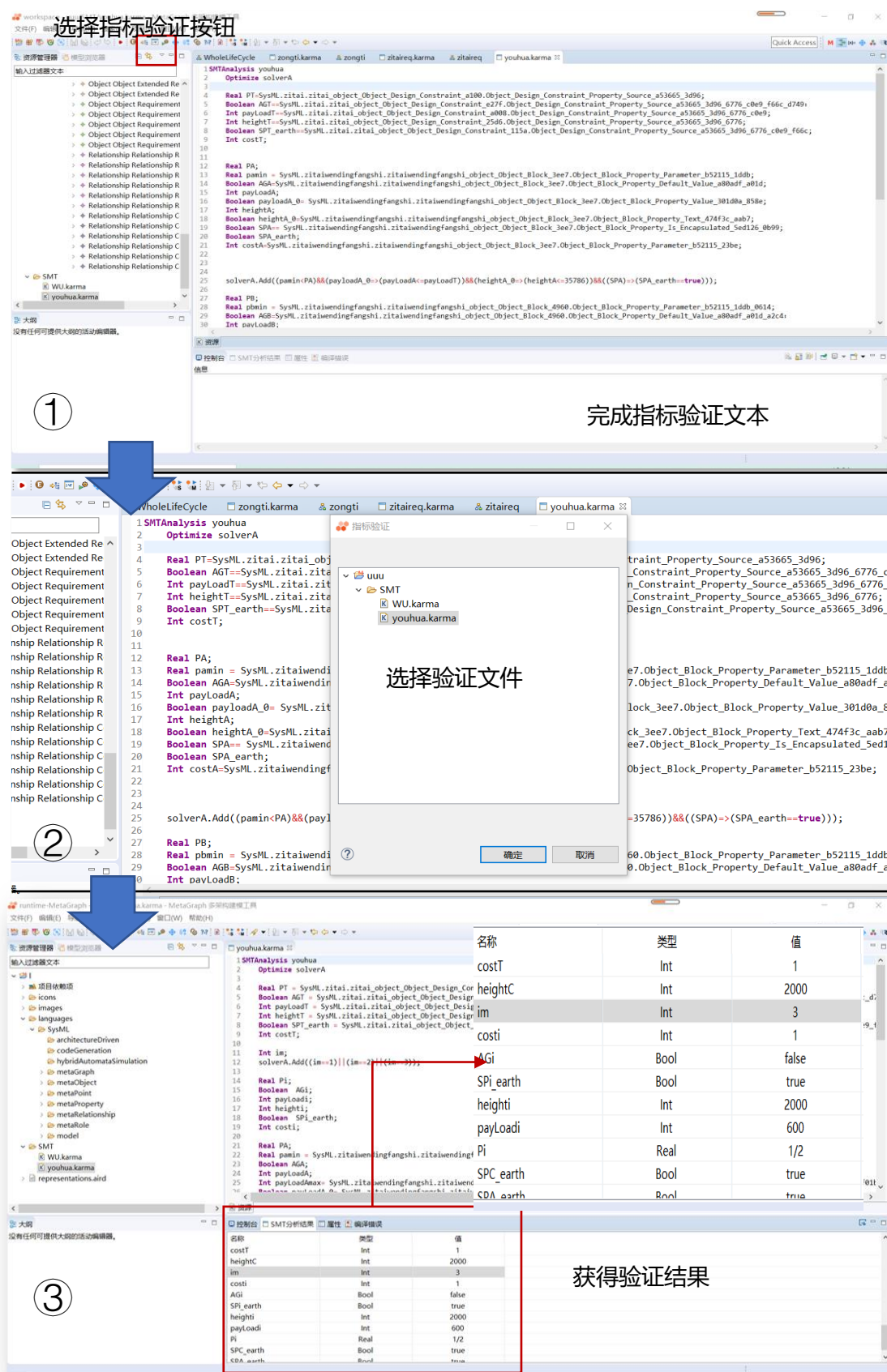


图 5.11 指标验证应用于姿态稳定方式的选型过程

面向逻辑单元设计方案确认问题的指标验证代码如下：

```

1. SMTAnalysis youhua //定义指标验证模块
2. Optimize solverA //定义一个优化求解器 solverA
3. // 从需求图获取姿态稳定方法的目标值
4. Real PT = SysML.zitai.zitai_objecta100.Property [a53665_3d96];
5. Boolean AGT = SysML.zitai.zitai_objecta1.Property [a535_3d97];
6. Int payLoadT = SysML.zitai.zitai_objecta10.Property [a53_3d98]
7. Int heightT = SysML.zitai.zitai_objecta20.Property [a65_3d96]
8. Boolean SPT_earth = SysML.zitai.zitai_ob1.Property [a55_3d96];
9. //定义三种姿态稳定的标号 1, 2, 3 即 A, B, C
10. Int im;
11. solverA.Add((im==1)||(im==2)||(im==3));
12. //定义待选方案的属性
13. Real Pi;
14. Boolean AGi;
15. Int payLoadi;
16. Int heighti;
17. Boolean SPi_earth;
18. Int costi;
19. //定义方案 1 即三轴稳定方法的属性：最小控制精度、敏捷机动、有效载荷、轨道高度、指向、成本
20. Real PA;//控制精度
21. Real pamin = SysML.fangshi.ck_3ee7.Property [ddb];//最小控制精度
22. Boolean AGA;//敏捷机动性能
23. Int payLoadA;//实际有效载荷
24. Int payLoadmax= SysML.fangshi.ck_3ee7.Property [ddb1];//理论最大有效载荷
25. Boolean payLoadA_0= SysML.fangshi.ck_3ee7.Property [ddb2];//有效载荷无限制
26. Int heightA;//实际轨道高度
27. Boolean heightA_0= SysML.fangshi.ck_3ee7.Property [ddb3];//轨道高度无限制
28. Int heightAmax = SysML.fangshi.ck_3ee7.Property [ddb4];//最大轨道高度
29. Boolean SPA= SysML.fangshi.ck_3ee7.Property [ddb5];//空间指向无限制
30. Boolean SPA_earth;//对地指向
31. Int costA= SysML.fangshi.ck_3ee7.Property [ddb6];//成本
32. //定义方案 1 的内部约束即：实际控制精度大于最小控制精度、无限制载荷说明实际载荷功率可达理论最大载荷功率、无限制轨道说明可达理论最高、空间指向无限制说明可以实现对地指向。
33. solverA.Add((pamin<=PA)&&(payLoadA_0=>(payLoadA<=payLoadAmax))&&(heightA_0=>(heightA<=heightAmax))&&((SPA)=>(SPA_earth==true)));
34. ... //方案 2, 3 的属性和约束已省略
35. //添加待选方案约束，若选择某一方案，则所有属性均需满足

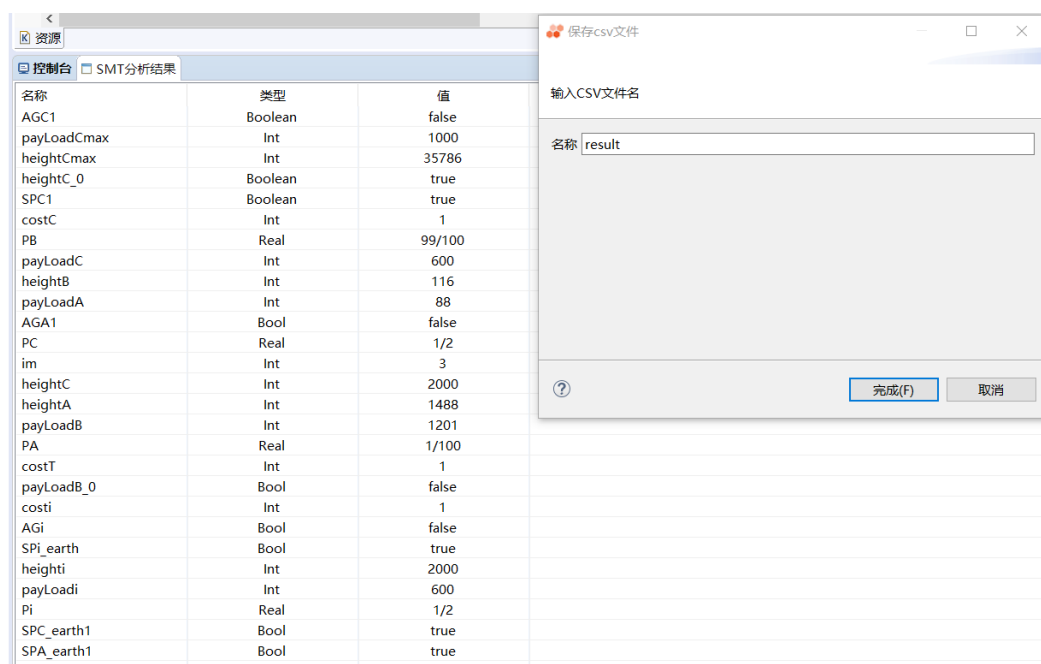
```

```

36. solverA.Add(((im==1)&&(Pi==PA)&&(AGi==AGA)&&(payLoadi==payLoadA)&&(heighti==heightA)&&(SPi_earth==SPA_earth)&&(costi==costA))||((im==2)&&(Pi==PB)&&(AGi==AGB)&&(payLoadi==payLoadB)&&(heighti==heightB)&&(SPi_earth==SPB_earth)&&(costi==costB))||((im==3)&&(Pi==PC)&&(AGi==AGC)&&(payLoadi==payLoadC)&&(heighti==heightC)&&(SPi_earth==SPC_earth)&&(costi==costC)));
37. //添加待选方案需满足指标属性
38. solverA.Add((Pi==PT)&&(payLoadi==payLoadT)&&(heighti==heightT)&&(SPT_earth==SPi_earth)&&(AGi==AGT));
39. solverA.Min(costi); //添加最小化待选成本约束
40. solverA.Check; //求解
41. solverA.Solution(); //获得满足上述约束的解集
42.
43. end solverA
44. end youhua

```

如图 5.11 所示，最终可以在结果输出所有符合约束的变量解，结果可以保存为 CSV 形式的数据，如图 5.12 所示。



名称	类型	值
AGC1	Boolean	false
payLoadCmax	Int	1000
heightCmax	Int	35786
heightC_0	Boolean	true
SPC1	Boolean	true
costC	Int	1
PB	Real	99/100
payLoadC	Int	600
heightB	Int	116
payLoadA	Int	88
AGA1	Bool	false
PC	Real	1/2
im	Int	3
heightC	Int	2000
heightA	Int	1488
payLoadB	Int	1201
PA	Real	1/100
costT	Int	1
payLoadB_0	Bool	false
costi	Int	1
AGi	Bool	false
SPi_earth	Bool	true
heighti	Int	2000
payLoadi	Int	600
Pi	Real	1/2
SPC_earth1	Bool	true
SPA_earth1	Bool	true

保存csv文件

输入CSV文件名

名称 result

完成(F) 取消

图 5.12 姿态稳定方式求解结果

为了更好的分析上述内容，将表格导出，并于 excel 中进行分析。如图 5.13 Excel 形式的姿态稳定方式选型结构所示，可以从结果中看出，标识符 im 的值为 3，即使用 C 方案：双自旋稳定方式。从图 5.13 可知，待选方案 i 的属性和 C 方案属性保持完全一致，且符合目标 T。

名称	类型	值	名称	类型	值
im	Int	3	heightCmax	Int	35786
AGi	Bool	FALSE	AGC1	Boolean	FALSE
costi	Int	1	costC	Int	1
heighti	Int	2000	heightC	Int	2000
Pi	Real	1/2	heightC_0	Boolean	TRUE
payloadi	Int	600	payloadC	Int	600
SPi_earth	Bool	TRUE	payloadCmax	Int	1000
AGT1	Boolean	FALSE	PC	Real	1/2
costT	Int	1	PCmin	Real	1/2
heightT	Int	2000	SPC_earth1	Bool	TRUE
PT	Real	1/2	SPB1	Boolean	TRUE
payloadT	Int	600	AGB1	Boolean	FALSE
SPT_earth1	Boolean	TRUE	costB	Int	1
AGA1	Bool	FALSE	heightB	Int	116
PA	Real	1/100	payloadB	Int	1201
PAmin	Real	1/100	payloadB_0	Bool	FALSE
costA	Int	2	payloadBmax	Int	1200
heightA	Int	1488	PB	Real	99/100
heightA_0	Boolean	TRUE	PBmin	Real	1/2
heightAmax	Int	35786	heightBmax	Int	1000
payloadA	Int	88	SPB_earth1	Boolean	TRUE
payloadA_0	Boolean	TRUE	SPB1	Boolean	FALSE
payloadAmax	Int	1200			
SPA_earth1	Bool	TRUE			
SPA1	Boolean	TRUE			

图 5.13 Excel 形式的姿态稳定方式选型结构

5.6.1.4 姿态敏感器和执行机构的方案确认

和姿态稳定方式的设计选型方案确认方法类似，首先构建面向姿态敏感器和执行机构的属性约束条件，再将两者的属性约束和需求目标进行结合，获得判断哪种姿态敏感器或执行机构是满足需求的。

(1) 姿态敏感器的属性约束

令姿态敏感器为 j ，包含地球敏感器、星敏器、磁强计、加速度计、太阳敏感器、陀螺组件六种敏感器，将他们的标号定义为 1 至 6，则 $j \in \{1,2,3,4,5,6\}$ 。姿态敏感器的属性包含使用精度、使用目的、是否不受轨道高度影响、适配的稳定方式、成本、以及必须配合的敏感器。由于敏感器可以选多个，使用 $Using_j$ 作为待检查的变量，若该值为“真”，则说明经过指标验证，该敏感器被选中作为使用功能的逻辑单元。且使用目的可以分为：提供姿态信息 $Function_j(Pos)$ ，提供初始姿态信息 $Function_j(iniPos)$ ，章动控制/摇摆角控制的角度 $Function_j(Nut)$ 。地球敏感器的属性约束如下式 5.31~5.34， $Proj.num$ 表示为属性关系返回为一阶逻辑表达式。

$$Pro_{1.1}: P_1 \geq 0.1 \quad (5.31)$$

$$Pro_{1.2}: Function_1(Pos) == true \&\& Function_1(iniPos) = \\ = false \&\& Function_1(Nut) == false \quad (5.32)$$

$$Pro_{1.3}: height_1(\infty) = true \&\& (height_1(\infty) == true) \Rightarrow (height_1 \leq 35786); \quad (5.33)$$

$$Pro_{1.4}: Met_1^{array} = \{A, B, C\}; \quad (5.34)$$

上述式中的角标 1 表明地球敏感器的标号“1”。式 5.31 表明地球敏感器的精度最高位 0.1° ；式 5.32 表明地球敏感器只能执行提供姿态信息的功能；式 5.33 表明地球敏感器不受轨道高度影响；式 5.34 中的上标 array 表明该属性为数组类型的，包含 ABC 三个字符串，表明地球敏感器适配的稳定方式包含 A, B, C 三种。

星敏器的属性约束如下式 5.35~5.38。式 5.35 表明星敏器的精度最高位 0.01° ；式 5.38 中表明星敏器只和 A 稳定方式适配。

$$Pro_{2.1}: P_2 \geq 0.01 \quad (5.35)$$

$$Pro_{2.2}: Function_2(Pos) == true \&\& Function_2(iniPos) = \\ = false \&\& Function_2(Nut) == false \quad (5.36)$$

$$Pro_{2.3}: height_2(\infty) = true \&\& (height_2(\infty) == true) \Rightarrow (height_2 \leq 35786); \quad (5.37)$$

$$Pro_{2.4}: Met_2^{array} = \{A\}; \quad (5.38)$$

磁强计的属性约束如下式 5.39~5.43。式 5.39 表明磁强计的精度最高位 1° ；式 5.41 表明磁强计不能不受轨道影响，其最高轨道为 1000km 以内。式 5.43 表示如果使用了 3 号磁强计那么一定要使用 5 号太阳敏感器，反之不然，如果使用了 5 号太阳敏感器不一定要使用 3 号磁强计。

$$Pro_{3.1}: P_3 \geq 1 \quad (5.39)$$

$$Pro_{3.2}: Function_3(Pos) == true \&\& Function_3(iniPos) = \\ = false \&\& Function_3(Nut) == false \quad (5.40)$$

$$Pro_{3.3}: height_3(\infty) = false \&\& height_3 \leq 1000; \quad (5.41)$$

$$Pro_{3.4}: Met_3^{array} = \{A, B, C\}; \quad (5.42)$$

$$Pro_{3.5}: Using_3 == true \Rightarrow Using_5 == true \quad (5.43)$$

加速度计的属性约束如式 5.44~5.47。式 5.45 表明加速度计只能执行章动控制、摇摆角控制的度数测量。

$$Pro_{4.1}: P_4 \geq 0.1 \quad (5.44)$$

$$Pro_{4.2}: Function_4(Pos) == false \&\& Function_4(iniPos) = \\ = false \&\& Function_4(Nut) == true \quad (5.45)$$

$$Pro_{4.3}: height_4(\infty) = true \ \&\& \ (height_4(\infty) == true) \Rightarrow (height_4 \leq 35786); \quad (5.46)$$

$$Pro_{4.4}: Met_4^{array} = \{C\}; \quad (5.47)$$

太阳敏感器的属性约束如下式 5.48~5.52。式 5.49 表明太阳敏感器只能执行捕获初始姿态信息的功能，式 5.52 表明如果使用 5 号太阳敏感器，那么其他的 1 至 6 号敏感器至少有一个是使用的。

$$Pro_{5.1}: P_5 \geq 0.5 \quad (5.48)$$

$$Pro_{5.2}: Function_5(Pos) == false \ \&\& \ Function_5(iniPos) = true \ \&\& \ Function_5(Nut) == false \quad (5.49)$$

$$Pro_{5.3}: height_5(\infty) = true \ \&\& \ (height_5(\infty) == true) \Rightarrow (height_5 \leq 35786); \quad (5.50)$$

$$Pro_{5.4}: Met_5^{array} = \{A, B, C\}; \quad (5.51)$$

$$Pro_{5.5}: Using_5 == true \Rightarrow (Using_1 || Using_2 || Using_3 || Using_4 || Using_6 == true) \quad (5.52)$$

陀螺组件的属性约束如下式 5.53~5.57。式 5.57 说明如果使用 6 号陀螺组件，则必须要使用 1 号地球敏感器或 3 号磁强计。

$$Pro_{6.1}: P_6 \geq 0.5 \quad (5.53)$$

$$Pro_{6.2}: Function_6(Pos) == true \ \&\& \ Function_6(iniPos) = false \ \&\& \ Function_6(Nut) == false \quad (5.54)$$

$$Pro_{6.3}: height_6(\infty) = true \ \&\& \ (height_6(\infty) == true) \Rightarrow (height_6 \leq 35786); \quad (5.55)$$

$$Pro_{6.4}: Met_6^{array} = \{A, B\}; \quad (5.56)$$

$$Pro_{6.5}: Using_6 == true \Rightarrow (Using_1 || Using_3 == true) \quad (5.57)$$

结合需求，可以总结出下列约束。式 5.58 说明若选用某种姿态敏感器，那么目标精度和轨道高度和敏感器的精度和适用高度保持一致且姿态敏感器关于适配稳定方式的类型内必须包含这种稳定方式。式 5.59 是对 4 号加速计使用过程的一个补充，即：如果使用 C 类型双自旋稳定方式，则必须使用加速度计。除 2 下列约束以外，还需要添加 $Pro_{j.5}$ 相关约束。

$$if \ Using_i == true; P_{target} == P_j; height_{target} = height_j; Met_j^{array}.get(i) == true \ j \in \{1,2,3,4,5,6\}, i \in \{A, B, C\} \quad (5.58)$$

$$im == 3 \Rightarrow Using_4 == true \quad (5.59)$$

在上述约束中, $Using_i$ 是变量, 添加上述约束, 最终判断保持上述约束全都满足的条件时, 哪种 $Using_i$ 能够为真值, 即使用对应的姿态敏感器。

(2) 姿态执行机构的属性约束

令姿态执行器为 k , 包含控制力矩陀螺、零动量轮控系统、章动阻尼器、偏置动量轮控系统、推力器、伸杆电机、磁力矩器 7 种执行机构, 将他们的标号定义小写字母 a 至 g , 则 $k \in \{a, b, c, d, e, f, g\}$ 。姿态敏感器的属性包含使用精度、是否实现敏捷机动功能、是否不受轨道高度影响、适配的稳定方式、成本。姿态执行机构可以选多个, 使用 $Using_k$ 作为待检查的变量, 若该值为“真”, 则说明经过指标验证, 该机构被选中作为使用功能的逻辑单元。控制力矩陀螺的属性约束如下式 5.60~5.63, $Pro_{k.num}$ 表示为属性关系返回为一阶逻辑表达式。

$$Pro_{a.1}: P_a \geq 0.01 \quad (5.60)$$

$$Pro_{a.2}: AG_a = true \quad (5.61)$$

$$Pro_{a.3}: height_a(\infty) = true \&\& (height_a(\infty) == true) \Rightarrow (height_a \leq 35786); \quad (5.62)$$

$$Pro_{a.4}: Met_a^{array} = \{A\}; \quad (5.63)$$

上述式中的角标 a 表示为控制力矩陀螺的标号“a”。式 5.60 表明控制力矩陀螺的精度最高位 0.01° ; 式 5.61 表明控制力矩陀螺能够实现敏捷机动功能; 式 5.62 表明控制力矩陀螺不受轨道高度影响; 式 5.63 表明控制力矩陀螺适配的稳定方式只有 A。零动量轮控系统的属性约束表达如下式 5.64~5.67。

$$Pro_{b.1}: P_b \geq 0.01 \quad (5.64)$$

$$Pro_{b.2}: AG_b = false \quad (5.65)$$

$$Pro_{b.3}: height_b(\infty) = true \&\& (height_b(\infty) == true) \Rightarrow (height_b \leq 35786); \quad (5.66)$$

$$Pro_{b.4}: Met_b^{array} = \{A\}; \quad (5.67)$$

章动阻尼的属性约束如下式 5.68~5.71。章动阻尼的精度为 0.5° 左右, 不能实现敏捷机动要求, 不受轨道限制, 适用于 B 重力梯度三轴稳定方法, C 双自旋稳定方法。

$$Pro_{c.1}: P_c \geq 0.5 \quad (5.68)$$

$$Pro_{c.2}: AG_c = false \quad (5.69)$$

$$Pro_{c.3}: height_c(\infty) = true \&\& (height_c(\infty) == true) \Rightarrow (height_c \leq 35786); \quad (5.70)$$

$$Pro_{c.4}: Met_c^{array} = \{B, C\}; \quad (5.71)$$

偏置动量轮控系统的属性约束如下式 5.72~5.75。偏置动量轮空系统的精度为 0.5° 左右，不能实现敏捷机动要求，不受轨道限制，适用于 A 三轴稳定方法，B 重力梯度三轴稳定。

$$Pro_{d.1}: P_d \geq 0.5 \quad (5.72)$$

$$Pro_{d.2}: AG_d = false \quad (5.73)$$

$$Pro_{d.3}: height_d(\infty) = true \&\& (height_d(\infty) == true) \Rightarrow (height_d \leq 35786); \quad (5.74)$$

$$Pro_{d.4}: Met_d^{array} = \{A, B\}; \quad (5.75)$$

推力器的属性约束与偏置动量轮控系统约束在推力器的精度，敏捷机动要求，轨道限制方面保持一致，但它适用于所有稳定方法，如式 5.76。

$$Pro_{e.4}: Met_e^{array} = \{A, B, C\}; \quad (5.76)$$

伸杆电机的其他属性表达与推力器属性约束报纸一致，除它只使用于 B 重力梯度三轴稳定方法，如式 5.77 所示

$$Pro_{f.4}: Met_f^{array} = \{B\}; \quad (5.77)$$

磁力矩器的其他属性表达与推力器属性约束报纸一致，除它受到轨道高度影响，如式 5.78 所示。

$$Pro_{g.3}: height_g(\infty) = false \&\& height_g \leq 1000; \quad (5.78)$$

除上述属性约束外，结合需求，可以总结出下列约束。式 5.79 说明如果使用了某种稳定方式，那么姿态执行器属性中适配稳定方式的类型必须包含这种稳定方式，且目标精度和轨道高度和执行机构的精度和适用高度保持一致。式 5.80 是对 c 章动阻尼器使用过程的一个补充，即：如果使用 4 号加速度计，则必须使用章动阻尼器。

$$if Using_i == true; \quad (5.79)$$

$$P_{target} == P_k; height_{target} = height_k; Met_k^{array}.get(i) == true;$$

$$k \in \{a, b, c, d, e, f, g\}, i \in \{A, B, C\}$$

$$Using_4 == true \Rightarrow Using_c == true \quad (5.80)$$

将上述约束通过 KARMA 语言在编辑界面进行编辑，得到最终的结果，如图 5.14 所示，即：

heighti	Int	2000
payLoadi	Int	600
Pi	Real	1/2
SPC_earth	Bool	true
use5	Bool	true
use4	Bool	true
use3	Bool	false
use2	Bool	false
use1	Bool	true
payLoadB_01	Bool	false
PB1	Real	3/10
useD	Bool	false
useE	Bool	true
useB	Bool	false
useA	Bool	false
useC	Bool	true

图 5.14 姿态敏感器和调整机构结果

根据结果可知，姿态敏感器使用太阳敏感器、地球敏感器、加速度计。执行机构选用推力器和章动阻尼器。可以得出，最终姿态控制系统的选型为：

表 5.5 姿控系统选型后结果

姿态控制系统功能单元	方案确认后姿态控制系统的逻辑单元选型
姿态稳定方法	C. 双自旋稳定方式
姿态敏感器	1. 地球敏感器、4. 加速度计、5. 太阳敏感器
执行机构	c. 章动阻尼器、e. 推力器

5.6.2 姿态控制系统的可靠性验证

本节基于 5.6.1 节选型后的姿态控制系统组成：双自旋平衡方式、太阳敏感器、地球敏感器、加速度计、推力器和章动阻尼器构建姿态控制系统的逻辑架构，验证该逻辑架构的可靠性。可靠性是指在在在一定情况下保证正常运行的可能，可靠性指标经过分解，面向逻辑架构阶段的姿态控制系统的可靠性是当卫星处于异常状态时，姿控系统能否及时响应紧急状态，从而保证运行的可靠。

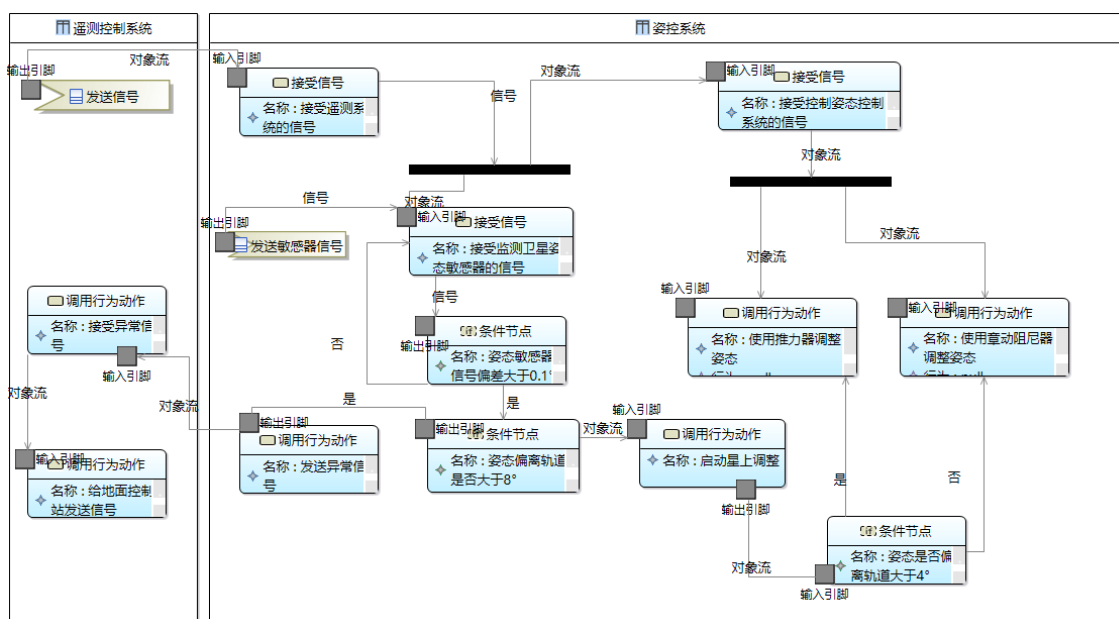


图 5.15 姿态控制系统的活动图

首先构建逻辑单元之间的活动图，声明姿态控制系统的工作流程，如图 5.15 姿态控制系统的活动图所示。姿态控制系统从遥测控制系统接受控制信号 $Signal_{YC}$ ，该信号可能令姿控系统实时监测姿态敏感器信号 $Signal_{vis}$ ，或者控制姿态系统执行机构信号 $Signal_{cont}$ 。若接受的是监测姿态敏感器信号，则开始实时接受姿态敏感器信号。若发现异常信号（姿态敏感器检测出轨道偏差大于 1° ），则进行处理。处理方法如下：若姿态异常信号为偏离轨道 8° ，则向遥测系统发送异常信号，遥测控制系统向地控中心传递该异常信号，若异常信号不足 8° ，则进行星上调节。星上调节需要再判断姿态敏感器获得的偏离角度是否大于 4° ，若角度不大于 4° ，则使用章动阻尼器调节，若角度大于 4° ，则使用推力器调节。若姿态控制系统获得的信号是控制信号，则控制信号可能是控制章动阻尼器或者推力器。

现基于该活动图，验证姿控系统逻辑架构能否及时响应紧急状态，从而保证运行的可靠，即：当信号出现异常时（异常姿态大于 1° ），卫星姿态系统处于应急活动状态，应急活动状态包含紧急信号发送、执行机构活动。

5.6.2.1 指标的形式化表达

当信号出现异常时，即当姿态偏离度数出现异常时，卫星处于应急状态，即：卫星姿态控制系统处于紧急状态时，则姿控系统一定发送了紧急信号发送、或者执行机

构 c 推力器、或者执行机构 e 章动阻尼器活动。式 5.81 定义了紧急状态，式 5.82 定义了可靠性指标的形式化表达。

$$State_{eme} = Signal_{eme} || Working_c || Working_e \quad (5.81)$$

$$\begin{aligned} &If \ Yw_n > 1 \\ &State_{eme} == true \end{aligned} \quad (5.82)$$

5.6.2.2 约束的形式化表达

如果遥测控制系统发送了信号，则只会输出两种信号：敏感器实时监测信号、控制信号，如式 5.83 所示。

$$(Signal_{yc} == true) == ((Signal_{vis} == true) || (Signal_{cont} == true)) \quad (5.83)$$

若输出的是敏感器实时监测信号，则当信号异常且偏离角度大于 8° 时，则发送紧急信号，如式 5.84。

$$\begin{aligned} &if \ Yw_n > 8 \ \&\& \ Signal_{vis} == true \\ &Signal_{eme} == true \end{aligned} \quad (5.84)$$

若输出的是敏感器实时监测信号，则当信号异常且偏离角度小于 8° 时，则进行星上调节，当偏离角度小于 4° ，使用章动阻尼器微动调节，当偏离角度大于 4° ，使用推力器微动调节，如式 5.85。

$$\begin{aligned} &if \ Signal_{vis} == true \\ &if \ 4 < Yw_n < 8; \ Working_c == true; if \ Yw_n < 4; \ Working_e == true \end{aligned} \quad (5.85)$$

若输出的是控制信号，则章动阻尼器和推力器一定会至少有一个行动，如式 5.86 所示。

$$if \ Signal_{cont} == true, \ Working_c == true || Working_e == true \quad (5.86)$$

5.6.2.3 实现方法和结果

若要证明在任意下的姿控系统能够及时相应紧急状态，即：当信号出现异常时（异常姿态大于 1° ），卫星姿态系统处于应急活动状态，应急活动状态包含紧急信号发送、执行机构活动；可以使用量词。若使用全称量词（ $\forall$ ）对于任意情况，只要信号出现异常，则卫星一定处于应急活动状态。可以看到该问题中，不同逻辑单元的状态较多，若使用全称量词则需要对包括遥测信号的有无、遥测信号的种类、姿态情况等均包含其中，较为麻烦。因此，使用存在量词进行反证，即：验证是否存在一种情况令信号出现异常时，卫星姿态并不处于应急活动。若该种情况存在，得出的结果是“可

满足”，则说明姿态控制系统的可靠性不足，若得出的结果是不可满足，则说明姿态控制系统的可靠性验证通过，其具体代码如下：

```

4. SMTAnalysis anquan
5. Solve solverB //定义通用求解器 solverB
6. Int Yw; //异常信号
7. Boolean stateEme; //紧急状态
8. Boolean sigEme; // 紧急信号
9. Boolean Workc; // 推力器运动
10. Boolean Worke; // 章动阻尼器运动
11. solverB.add(stateEme==(sigEme||Workc||Worke)); //紧急状态和其他三状态的关系
12. Int jixian1 = SysML.erliziAct. 9b7b.Property [erlproperty_2a97];
13. Int jixian2 = SysML.erliziAct. 658f.Property [erlproperty_2290];
14. Int jixian = SysML.erliziAct. 543f.Property [erlproperty_5b60];
15. //获取姿态较大变化、星上修补条件以及异常信号的值
16. Boolean sigYC; //遥测信号
17. Boolean sigvis; //姿控探测信号
18. Boolean sigcont; //姿控控制
19. solverB.add(((sigYC==true)=>(((sigvis==true)&&(sigcont==false))||((sigvis==false)&&(sigcont==true))))); //遥测一旦发送信号能且只能发送一种信号
20. //当姿态大于 8° 时且发送的是姿控探测信号，姿控发送紧急信号
21. solverB.add(((Yw>jixian1)&&(sigvis==true))=>(sigEme==true));
22. //当姿态小于 8° 但大于 4 度时且发送的是姿控探测信号，推力器运动
23. solverB.add(((sigvis==true)&&(Yw<= jixian1)&&(Yw> jixian2))=>(Workc==true));
24. //当姿态异常小于 4 度且发送的时监测信号时，章动阻尼器运动
25. solverB.add(((sigvis==true)&&(1<Yw<=4))=>(Worke==true));
26. //若遥测发送的是控制信号，则一定有一个机构运动了
27. solverB.add((sigcont==true)=>((Workc==true)||(Worke==true)));
28. 判断若存在异常信号时，没有紧急状态的情况是否存在
29. solverB.add(Yw>=1);
30. solverB.add(stateEme==false);
31. if(solverB.check==SAT)//若存在上述状况，定义一种满足上述状况的解
32. solverB.Solution();
33. else
34. print("Not satisfying");
35. end
36. end solverB
37. end anquan

```

最终结果如图 5.16 所示，最终结果为“satisfied 条件可满足”说明存在一种可能可满足上述约束，即说明：至少存在一种情况，使得姿态控制系统姿态存在异常时，

姿态控制系统不处于紧急状态，也就是说该设计的可靠性无法得到保证。从输出的结果可以看到，当异常姿态角度为 9° 时，若遥测信号为“false”，即：遥测控制系统未向姿控系统发送信号，则系统的可靠性难以保证。

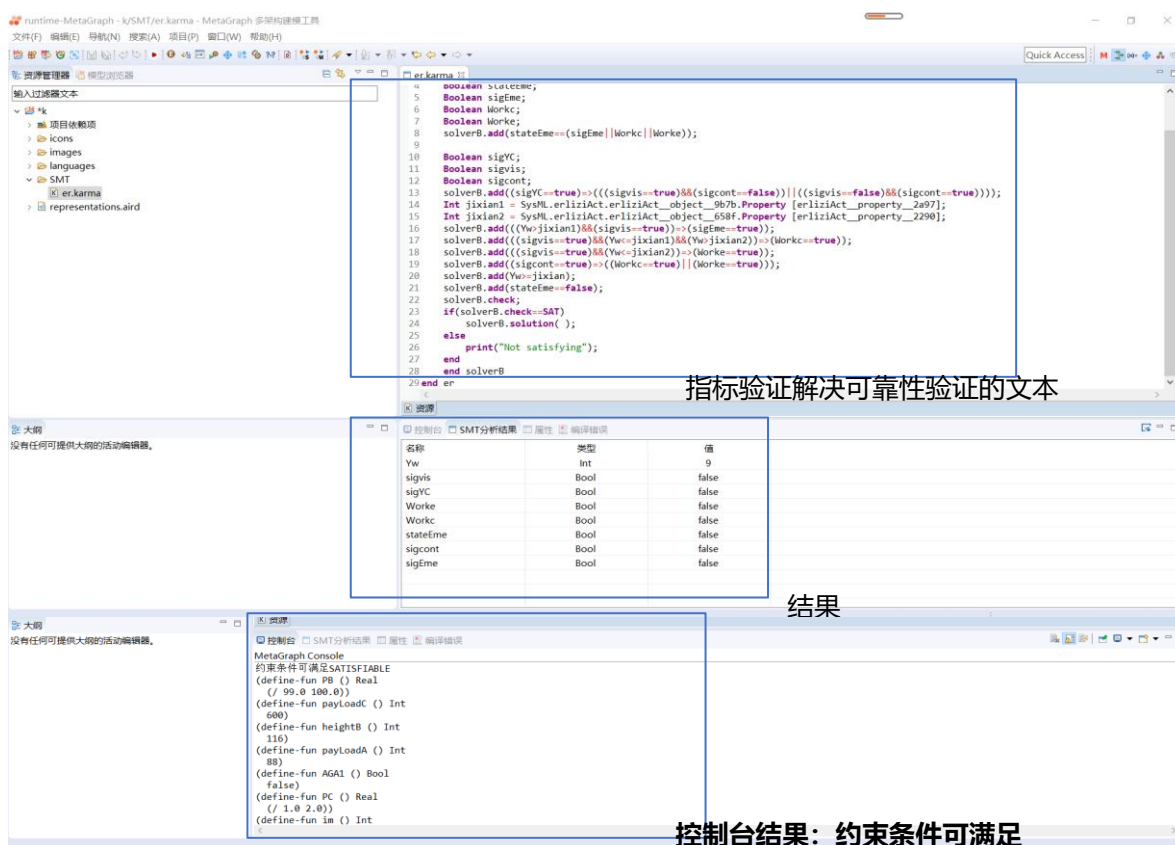


图 5.16 可靠性验证结果

5.7 案例讨论

本案例针对卫星姿态控制系统设计问题，使用 MetaGraph 多架构建模平台和基于 SysML 模型库，完成指标体系的构建、卫星姿控系统逻辑单元的方案确认和姿控逻辑架构的可靠性确认。首先构建面向卫星设计的整体建模框架，并使用 KARMA 语言根据卫星设计场景扩展 SysML 库，构建适用于卫星设计的模型库。再依据卫星设计整体框架，定义整体需求，以姿控系统为研究重点，在模型中输出总体-姿控系统指标体系。根据指标评估，明确姿控系统逻辑单元的方案确认，并构建选择后的姿控逻辑架构。最后，验证姿控逻辑架构的设计可靠性。

该案例首先应用第二章定义的多维多层指标定义方法，分层构建面向卫星姿控系统的多维指标体系，明确所提方法的有效性、完整性。指标体系在模型中以多张需求

图组合的方式表现，并且用定义的指标和设计单元相关联，指导设计的选型和验证。该案例说明所提多维多层方法能够应用在复杂系统设计中，具备一定的可行性。

该案例通过指标验证手段实现卫星姿控分系统的逻辑单元设计选项的方案确认问题，并基于活动图验证了姿态控制系统构建的可靠性，突破了现有建模语言不精确和指标验证方法不通用的障碍。该案例在解决上述两问题的过程中共建立 13 个模型，分别使用了 79 行 KARMA 指标验证代码和 25 行 KARMA 指标验证代码。KARMA 语言相较于现有指标验证语言是抽象层次更高的建模语言，能够形式化描述模型元素，指标验证语言基于 KARMA 扩展，保证了建模语言和指标验证语言的一致性，建模数据和验证数据的一致性。同时，该 SysML 模型库是面向卫星设计场景扩展的，不拘泥于原模型库，展现了 KARMA 语言在描述模型库方面的通用性，因此，该案例也证明了所提方法能够突破异构语言模型描述，实现跨异构语言的指标验证。其中，方案确认问题使用了优化求解器，可靠性问题使用了通用求解器，两问题共同运用了算数理论、布尔理论、数组理论，而字符串理论等方法暂未使用。可以看出，该方法可以通过较短的代码实现问题的解决。语言案例的成功实行证明了本文提出的指标验证手段能够实现跨模型、自定义的验证，并且保证了模型数据和求解数据的一致性，以及形式化描述和求解的理论一致性。

该案例具备一定的扩展性，即：不只是在卫星设计中实践，也可以在其他复杂系统设计过程中实践。之所以具备这样的扩展性，一是因为 GOPPRR-E 的优异表达能力，能够定义跨领域的异构建模语言，支持多种复杂系统设计；二是因为可满足性模理论描述约束的能力优异，支持多种复杂系统设计的多种指标的形式化验证。案例中以 GOPPRR-E 元建模方法为基础，统一地描述卫星架构模型的建立过程，描述逻辑单元设计选项的方案确认问题和可靠性验证的模型元素。GOPPRR-E 建模理论以元元模型表达为基础，统一支持不同领域内的各类模型。GOPPRR-E 建模理论被认为是开发特定域模型的最佳建模理论^[83,85]，其灵活性强，抽象层次高，支持不同领域内的元模型和模型建立以及模型之间关系约束的表达。此外，GOPPRR-E 建模理论也支持图形化模型的表达，所以采用图形化手段描述模型能够增强场景的表达能力、减少问题传达的不一致情况。可满足性模理论的求解对象是泛布尔类型（包含多种数学理论的布尔类）的约束关系，任意指标和模型架构之间的约束只需要被表达或转化为一阶逻辑表达式即可以被求解。其应用域较为广泛，包含算数、布尔、机器数、数组、未解释函数等多重理论，支持构建复杂数学表达式，具备较强的约束描述能力。同时，将

GOPRR-E 建模理论融合可满足性模理论用于实现指标和模型架构之间的形式化表达与验证。

本文所提出的方法在解决静态指标验证问题和设计选项方案确认问题具备可行性、扩展性，并且保证了该方法在模型和约束描述之间的数据一致性、约束描述和验证的理论一致性，降低了求解结果的不确定性。

5.8 本章小结

本章应用所提指标验证方法解决卫星设计过程中的面向逻辑单元设计方案确认问题和姿态控制系统的可靠性验证问题。首先，采用 MBSE 方法定义卫星设计问题的整体框架，使用由统一建模语言 KARMA 构建的 SysML 模型库，在统一建模工具的建模界面构建所需模型视图。并基于此，再在统一建模工具的文本编辑界面使用 KARMA-指标验证语言描述案例的指标、现有设计架构、以及两者之间的基于一阶逻辑表达式的约束关系。最后，执行指标验证文本，得到最终结果。本章案例的成功实现验证了本文所提指标验证方法的可行性、可扩展性和优越性。

结论与展望

在基于模型的系统工程领域内，指标验证方法能够形式化描述指标和现有系统模型方案之间的约束关系，并评估约束的可满足性。指标验证方法用于解决静态指标验证，及设计方案确认问题，实现验证前移，大大降低验证和确认的时间成本，提高设计的质量和效率。本文针对目前指标验证方法缺乏面向复杂系统设计的多维多层指标体系定义方法、难以突破跨建模语言的验证、缺乏指标形式化表达和验证的统一方法等问题，提出基于 GOPRR-E 元建模方法和可满足性模理论的指标形式化表达和验证方法。论文的主要工作总结如下：

(1) **形成面向复杂系统设计的多维多层指标体系定义方法。**该研究内容以“需求定义—功能架构—逻辑架构—物理架构”的系统分析流程为指标体系纵向的分层依据，形成“效能指标—功能和性能指标—技术指标”的多层指标体系。同时，引入霍尔系统工程方法的知识维和时间维扩展指标体系的维度。最后，从社会、系统、信息、技术四个角度出发，细化功能非功能指标和技术指标的维度，从而形成完整的面向复杂系统设计的多维多层指标体系，为后文中的指标验证提供了验证对象。该方法明确了指标体系的层次划分和维度分析的依据，构建指标和系统各级单元的映射关系，支持面向复杂系统设计构建指标体系，具备完备性和重用性。

(2) **定义面向系统设计的指标形式化表达和验证方法。**本研究在理论层面提出一种指标验证的实现方法。首先，该方法基于 GOPRR-E 元建模方法形式化描述异构模型的模型元素。再研究基于可满足性模理论定义和验证一阶逻辑表达式的方法。最后，构建模型元素和一阶逻辑表达式的映射关系，支持面向模型元素的指标形式化表达和验证方法，定义包含静态指标验证和设计方案确认的指标验证类型。该方法突破异构模型的验证局限性，支持面向异构模型元素的指标验证。同时，该方法保持模型数据和指标验证数据的一致、指标形式化表达和验证方法的理论一致，这种一致性降低因模型数据转换带来的不确定性。

(3) **系统指标验证语言的设计和实现。**本研究在上一研究的基础上，定义指标验证语言的语法和语义。首先，本文分析多架构统一建模语言 KARMA 和基于可满足性模理论的 SMT-LIB 语言的语法结构和语义风格。其次，本文扩展 KARMA 语言的语法和语义，结合 SMT-LIB 语言形成面向模型元素的指标验证语言。最后，根据预定义的语法，基于 ANTLR 语法分析器构建应的指标验证语言编译器，集成至统一建模平

台 MetaGraph, 实现指标验证功能。本研究通过扩充 KARMA 语言的语法和语义, 实现面向异构语言的指标验证, 解决目前建模语言无法形式化指标的问题, 支持使用一种语言实现异构模型的建立、指标相关约束的形式化表达和验证, 降低因数据转换带来的验证结果不确定性和学习多种工具的成本。

(4) **基于案例研究验证方法的可行性。**该研究内容使用指标验证方法解决卫星设计过程中面向逻辑单元设计方案确认问题以及姿态控制系统的可靠性验证问题, 证明了本文所提出方法的可行性。

本文创新点

基于上述研究内容, 本文的主要**创新点**如下:

(1) 基于 GOPPRR-E 理论和可满足性模理论定义指标形式化表达及验证方法

本文提出一种支持跨建模语言的、支持自定义指标的指标验证方法。该方法结合 GOPPRR-E 元建模方法和可满足性模理论, 使用 GOPPRR-E 元建模方法支持跨建模语言的模型建立和描述, 使用可满足性模理论实现面向模型元素的指标形式化表达和验证。该方法突破了面向异构模型验证的困境, 保证模型数据和指标形式化数据的一致性, 降低因数据转换带来的不确定性。

(2) 建立面向 MBSE 环境的指标验证语言规范

本文提出一种面向 MBSE 环境的指标验证语言规范。本文在统一建模语言 KARMA 的基础上, 结合基于可满足性模理论的语言 SMT-LIB, 定义 KARMA-指标验证模块的语法和语义。并完成基于 ANTLR 语法分析器的指标验证语言编译器, 实现指标验证功能。该语言规范实现异构模型构建、指标形式化表达和验证的统一, 突破 MBSE 领域内通用建模语言不具备形式化基础的困境, 降低因模型管理语言和工具杂糅带来的验证结果不确定性, 以及因使用多种建模语言、模型管理语言和其他验证工具所带来的学习成本。

工作展望

本文围绕指标的形式化表达和验证展开研究, 仍然有一些不足。本文的工作展望如下:

本文指标验证语言在面向大规模非线性约束时求解速度较慢, 需继续研究如何提升包含非线性约束的指标验证效率。非线性约束存在于许多计算复杂的数值型指标,

如惯量指标、重心指标等。本文使用的求解方法是一种基于定理证明的形式化验证方法，使用的求解器是综合能力最强的基于可满足性模理论的求解器 Z3 求解器，主要面向融合多种理论的一阶逻辑表达式。该求解器虽然支持最多的数学理论，但是在求解非线性约束的时，其求解时间随着变量数目、非线性约束的数量迅速提高。除 Z3 求解器外，专门用于解决非线性约束的可满足性求解器是存在的，然而其综合能力较差。因此，后续讨论可以针对指标验证语言解决非线性问题进行讨论。

本文设计的指标验证语言获取模型元素的方法较为繁琐，不利于用户使用，需继续研究如何简化验证步骤。指标验证语言获取模型元素时需要依次定义其语言名、图名、模型实例名，在获取点、角色和属性时还需要定义其关联的模型实例名称。该方法较为繁琐，代码构成也较为繁琐。在遇到较长名称时，还可能出现无法验证的情况。需要进一步针对获取模型元素的方法进行研究。

本文设计的指标验证语言不能支持定义模型内部关系的约束构建，也不支持策略求解器的求解，需继续研究如何验证模型自身的规则。本文的指标验证语言主要面向属性构建的约束，尚未完全支持其他模型元素之间的约束描述，如：点和对象之间的关系、某关系实例的角色类型等。需要进一步研究此类语言的语法和语义构建。

参考文献

- [1] Ladyman J, Lambert J, Wiesner K. What is a complex system? [J]. European Journal for Philosophy of Science, 2013, 3(1): 33-67.
- [2] Verhoef M, Pierce K, Gamble C, et al. Collaborative Development of Embedded Systems[M]. Springer Berlin Heidelberg, 2014: 3-14.
- [3] Hölttä-Otto K, Niutanen V, Eppinger S, et al. Design sprint for complex system architecture analysis[C]. International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (IDETC/CIE). American Society of Mechanical Engineers, 2018: 1-9.
- [4] Sampson M, Friedenthal S. Model-based Systems Engineering (MBSE) Initiative[C]. INCOSE International Workshop. 2011:1-10.
- [5] Huldt T, Stenius I. State - of - practice survey of model - based systems engineering[J]. Systems Engineering, 2019, 22(2): 134-145.
- [6] IEEE Standards Coordinating Committee. IEEE Standard Glossary of Software Engineering Terminology (IEEE Std 610.12-1990). Los Alamitos[J]. CA: IEEE Computer Society, 1990, 169.
- [7] Hobday M. Product complexity, innovation and industrial organization[J]. Research policy, 1998, 26(6): 689-710.
- [8] Rechtin E, Maier M W. The art of systems architecting[M]. CRC Press, 2010: 66-69.
- [9] Northrop L, Feiler P, Gabriel R P, et al. Ultra-large-scale systems: The software challenge of the future[R]. Carnegie-Mellon Univ Pittsburgh Pa Software Engineering Inst, 2006.
- [10] Prencipe A. Breadth and depth of technological capabilities in CoPS: the case of the aircraft engine control system[J]. Research Policy, 2000, 29(7-8): 895-911.
- [11] 徐路宁, 张和明, 张永康. 复杂产品协同设计中关键问题的研究[J]. 现代制造工程, 2005(01): 10-13.
- [12] 付振通. 复杂产品创新知识获取系统研究[D]. 哈尔滨: 哈尔滨工程大学, 2013.
- [13] 关锋, 葛平, 周国栋, 等. MBSE 发展趋势与中国探月工程并行协同论证[J]. 空间科学学报, 2022, 42(02): 183-190.
- [14] 陈竹梅. 复杂电子信息系统 R&D 数字化转型综述[J]. 中国电子科学研究院学报, 2020 ,15(12): 1219-1231.

- [15] Feiler P H, Gluch D P. Model-based engineering with AADL: an introduction to the SAE architecture analysis & design language[M]. Addison-Wesley, 2012: 19-37.
- [16] Cardoso J , Diniz P C . Embedded Computing for High Performance[M]. Elsevier. 2017: 17-56.
- [17] Betari O, Filali S, Azzaoui A, et al. Applying a Model Driven Architecture Approach: Transforming CIM to PIM Using UML[J]. International Journal of Online and Biomedical Engineering (iJOE), 2018, 14(09): 170-181.
- [18] Fisher A, Nolan M, Friedenthal S, et al. 3.1. 1 model lifecycle management for MBSE[C]. INCOSE International Symposium. 2014, 24(1): 207-229.
- [19] Walden D D, Roedler G J, Forsberg K J, et al. Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities[M]. Engineering I. INCOSE 2015: 46-75.
- [20] 朱静,杨晖,高亚辉,等.基于模型的系统工程概述[J].航空发动机,2016,42(4):12-16.
- [21] Ozlem Demirci. Development of Mbse/Uml maturity model[D]. Tekniska Jönköping School of Engineering, 2010.
- [22] Chami M, Aleksandraviciene A, Morkevicius A, et al. Towards solving MBSE adoption challenges: the D3 MBSE adoption toolbox[C]. INCOSE International Symposium. 2018, 28(1): 1463-1477.
- [23] Moura L D. Z3: An Efficient SMT Solver Tools, and Algorithms for the Construction and Analysis of Systems. 2008.
- [24] Sandeep Jain. Non-functional Requirements in Software Engineering[EB/OL]. 2020[2022-5-1]. <https://www.geeksforgeeks.org/non-functional-requirements-in-software-engineering/#:~:text=Non-Functional%20Requirements%20are%20the%20constraints%20or%20the%20requirements,maintainability%2C%20performance%2C%20portability%2C%20security%2C%20reliability%2C%20and%20many%20more.>
- [25] Brittany, Friedland, John, et al. Conducting a Model Based Systems Engineering Tool Trade Study Using a Systems Engineering Approach[C]. 2017:1087-1099.
- [26] Hoffmann H P. Deploying model-based systems engineering with IBM® rational® solutions for systems and software engineering[C]. Digital Avionics Systems Conference. IEEE, 2012: 1-8.
- [27] 谭琪璘, 王旭昊. 基于 Harmony SE 方法的嵌入式软件设计综合过程研究[J]. 信息通信, 2019 (4): 39.
- [28] Friedenthal, Sanford. A Practical Guide to SysML: The Systems Modeling Language[M]. America: Morgan Kaufmann, 2015: 1-2.

- [29] Authors S . A Survey of Model-Based Systems Engineering (MBSE) Methodologies[J]. 2013: 1-10.
- [30] Waseem M, Sadiq M U. Application of model-based systems engineering in small satellite conceptual design-A SysML approach[J]. IEEE Aerospace and Electronic Systems Magazine, 2018, 33(4): 24-34.
- [31] de Athayde Prata B, Pitombeira-Neto A R, de Moraes Sales C J. An integer linear programming model for the multiperiod production planning of precast concrete beams[J]. Journal of Construction Engineering and Management, 2015, 141(10): 04015029.
- [32] 魏笑. 基于元数据的指标遴选方法研究[D]. 北京: 军事科学院, 2021.
- [33] Roques P. Systems Architecture Modeling with the Arcadia Method: A Practical Guide to Capella[M]. Elsevier, 2017: 86-96.
- [34] Gogolla M , Hilken F , Doan K H . Achieving Model Quality through Model Validation, Verification and Exploration[J]. Computer Languages Systems & Structures, 2017, 54(DEC.):474-511.
- [35] EAST-ADL Association . About EAST-ADL[EB/OL]. [2022-4-1]. <http://east-adl.info/>.
- [36] 门星火, 李晓利, 刘波. 复杂系统仿真概念模型验证方法研究[J]. 软件导刊, 2016, 15(10):3.
- [37] 杨伯军, 苏明岳, 万鹏, 王玥, 刘芳. 基于压力-状态-响应框架的产品可持续设计指标体系研究[J]. 机械设计, 2022, 39(03): 130-137.
- [38] 贾启蒙, 孔祥华, 张鹏, 田小兵, 孙业兵, 李庆龙. 基于正向研发的汽车设计指标体系研究[J]. 重型汽车, 2020, (01): 5-6
- [39] Hause M. The SysML modelling language[C]. Fifteenth European Systems Engineering Conference. 2006, 9: 1-12.
- [40] Booch G, Rumbaugh J, Jacobson I. UML: Unified Modeling Language[J]. Versão, 1997.
- [41] 刘征, 武晓春. 联锁系统 UML 模型的建立与形式化验证[J]. 铁道标准设计, 2018, 62(6): 164-170.
- [42] 罗奇鸣. 一种利用 Kodkod 约束求解器验证 UML-OCL 类图的方法[J]. 小型微型计算机系统, 2014, 35(2): 205-209.
- [43] Herzig S J I, Qamar A, Reichwein A, et al. A conceptual framework for consistency management in model-based systems engineering[C]. International Design Engineering Technical Conferences and Computers and Information in Engineering Conference. 2011, 54792: 1329-1339.
- [44] Lu J, Wang G, Ma J, et al. General Modeling Language to Support Model - based Systems Engineering Formalisms (Part 1)[C]. INCOSE International Symposium. 2020, 30(1): 323-338.

- [45] Li Q, Wei H, Yu C, et al. Data and model-based triple V product development framework and methodology[J]. Enterprise Information Systems, 2021: 1-33.
- [46] Spangelo S C, Cutler J, Anderson L, et al. [C]//2013 IEEE Aerospace Conference. IEEE, 2013: 1-18.
- [47] Kaslow D, Ayres B, Cahill P T, et al. A Model-Based Systems Engineering (MBSE) approach for defining the behaviors of CubeSats[C]. 2017 IEEE Aerospace Conference. IEEE, 2017: 1-14.
- [48] Ryan M. On the Relationship Between Functions and Measures in System Design[C]. Systems Engineering. 2014: 1-19.
- [49] Kaslow D, Madni A M. Validation and verification of MBSE-compliant CubeSat reference model[M]. Disciplinary Convergence in Systems Engineering Research. Springer, Cham, 2018: 381-393.
- [50] SEBok authors. NASA Systems Engineering Handbook[J]. 2013:1-1.
- [51] 付广胜. 基于 xUML 可执行模型的 CISR 系统需求开发与验证方法研究[D]. 湖南: 国防科学技术大学, 2007.
- [52] Manuele Leonelli. Simulation and Modelling to Understand Change[EB/OL]. 2021-4-19[2022-5-1]. https://bookdown.org/manuele_leonelli/SimBook/.
- [53] Aloui K, Hammadi M, Soriano T, et al. On the continuity of the swarm robot design using MBSE method and simulation. MOSIM 20[C]. Morocco: HAL, 2020: 1-10.
- [54] 郝斐, 董庆超, 曾广军. 一种基于描述逻辑的 UML 模型验证方法[J]. 计算机与数字工程, 2011, 39(11):5.
- [55] 单卓为. 基于 SPIN 的 UML 模型验证技术的研究[D]. 西北大学, 2008
- [56] Wang Dandan, Liang H, Cui J W. Modeling and Simulation of Top-level Design Based on MBSE[J]. Journal of Physics Conference Series, 2020, 1486:072061.
- [57] 刘伟. 基于 Modelica 的载人航天器空气环境控制系统建模与仿真[D]. 武汉: 华中科技大学, 2016.
- [58] 张鹏翼, 黄百乔, 鞠鸿彬. MBSE: 系统工程的发展方向[J]. 科技导报, 2020, 38(21):21-26.
- [59] Pitchumani V, Stabler E P. A formal method for computer design verification[C]. 19th Design Automation Conference. IEEE, 1982: 809-814.
- [60] Hasan O, Tahar S. Formal verification methods[M]. Encyclopedia of Information Science and Technology, Third Edition. IGI Global, 2015: 7162-7170.
- [61] Bjesse P. What is formal verification?[J]. ACM SIGDA Newsletter, 2005, 35(24): 1-es.
- [62] Hierons R M, Bogdanov K, Bowen J P, et al. Using formal specifications to support testing[J]. ACM

- Computing Surveys (CSUR), 2009, 41(2): 1-76.
- [63] Gaudel M C. Formal specification techniques[C]. Proceedings of 16th International Conference on Software Engineering. IEEE, 1994: 223-227.
- [64] Kern C, Greenstreet M R. Formal verification in hardware design: a survey[J]. ACM Transactions on Design Automation of Electronic Systems (TODAES), 1999, 4(2): 123-193.
- [65] Rushby J. Theorem proving for verification[C]. Summer School on Modeling and Verification of Parallel Processes. Springer, Berlin, Heidelberg, 2000: 39-57.
- [66] Robinson A P, Froese R E. Model validation using equivalence tests[J]. Ecological Modelling, 2004, 176(3-4): 349-358.
- [67] 拉森杨保华. 实用空间系统工程[M]. 中国宇航出版社, 2013: 105-110.
- [68] 刘峰华. 基于本体知识的计算机辅助卫星概念设计研究[D]. 哈尔滨工业大学, 2011.
- [69] Desai A, Dreossi T, Seshia S A. Combining model checking and runtime verification for safe robotics[C]. International Conference on Runtime Verification. Springer, Cham, 2017: 172-189.
- [70] Kerzhner A A, Paredis C J J. Model-based system verification: a formal framework for relating analyses, requirements, and tests[C]. International Conference on Model Driven Engineering Languages and Systems. Springer, Berlin, Heidelberg, 2010: 279-292.
- [71] Kolovos D S, Paige R F, Polack F A C. Eclipse development tools for epsilon[C]. Eclipse Summit Europe, Eclipse Modeling Symposium. 2006, 20062: 200.
- [72] Gogolla M, Büttner F, Richters M. USE: A UML-based specification environment for validating UML and OCL[J]. Science of Computer Programming, 2007, 69(1-3): 27-34.
- [73] Richters M, Gogolla M. OCL: Syntax, semantics, and tools[M]. Object Modeling with the OCL. Springer, Berlin, Heidelberg, 2002: 42-68.
- [74] Brucker A D, Wolff B. HOL-OCL: a formal proof environment for UML/OCL[C]. International Conference on Fundamental Approaches to Software Engineering. Springer, Berlin, Heidelberg, 2008: 97-100.
- [75] Pérez B, Porres I. Reasoning about UML/OCL class diagrams using constraint logic programming and formula[J]. Information Systems, 2019, 81: 152-177.
- [76] Sillitto H G. 1.3.1 Design principles for Ultra-Large-Scale (ULS) Systems[C]. INCOSE International Symposium. John Wiley & Sons, Ltd, 2010:1-8.
- [77] Kolovos D S, Paige R F, Polack F A C. On the evolution of OCL for capturing structural constraints

- in modelling languages[M]. Rigorous Methods for Software Construction and Analysis. Springer, Berlin, Heidelberg, 2009: 204-218.
- [78] Madani S, Kolovos D S, Paige R F. Parallel Model Validation with Epsilon[C]. European Conference on Modelling Foundations and Applications. Springer, Cham, 2018: 115-131.
- [79] 刘丹. 装配式建筑设计标准体系构建研究[D]. 黑龙江:东北林业大学,2018.
- [80] 孙东川, 朱桂龙. 变通的逻辑[M]. 北京: 科学出版社, 2010: 71-104.
- [81] The SMT-LIB Initiative. SMT-LIB Theory[EB/OL]. [2022-5-3]. <http://smtlib.cs.uiowa.edu/theories.shtml>.
- [82] Ch C . [SMT-LIB] Bitvectors[EB/OL]. [2022-5-3]. <https://www21.in.tum.de/teaching/sar/SS20/7.pdf>
- [83] Kern, Heiko & Hummel, Axel & Kühne, Stefan. Towards a Comparative Analysis of Meta-Metamodels[C]. 11th Workshop in Domain-Specific Modeling, 2011: 1-10.
- [84] Lu J, Wang G, Wang H, et al. General Modeling Language for Supporting Model-based Systems Engineering Formalisms (Part 1)[J]. John Wiley & Sons, Ltd, 2020:1-11.
- [85] Wang H , Wang G , Lu J , et al. Ontology Supporting Model-Based Systems Engineering Based on a GOPPRR Approach[M]. 2019: 63-72.
- [86] De Moura L, Bjørner N. Satisfiability modulo theories: An appetizer[C]. Brazilian Symposium on Formal Methods. Springer, Berlin, Heidelberg, 2009: 23-36.
- [87] Nieuwenhuis R, Oliveras A. On SAT modulo theories and optimization problems[C]. International conference on theory and applications of satisfiability testing. Springer, Berlin, Heidelberg, 2006: 156-169
- [88] Clarke, Edmund M, Henzinger, Thomas A, Veith, Helmut, et al. Handbook of Model Checking[M]. Springer, Cham, 2018: 1-26.
- [89] Oliveras A. Survey of satisfiability modulo theories (SMT)[C]. Banff International Research Station for Mathematical Innovation and Discovery (BIRS) Workshop Lecture Videos. Banff International Research Station for Mathematical Innovation and Discovery. 2014: 34-45.
- [90] de Moura L, Dutertre B, Shankar N. A tutorial on satisfiability modulo theories[C]. International Conference on Computer Aided Verification. Springer, Berlin, Heidelberg, 2007: 20-36.
- [91] 李婧. SMT 求解器理论组合技术研究[J]. 计算机工程与科学, 2011:1-13.
- [92] 李婧. SMT 求解器技术对比分析及其能力扩展研究[D]. 国防科学技术大学, 2010.

- [93] 王翀, 吕荫润, 陈力,等. SMT 求解技术的发展及最新应用研究综述[J]. 计算机研究与发展, 2017, 54(7): 1405-1425.
- [94] Vizel Y, Weissenbacher G, Malik S. Boolean satisfiability solvers and their applications in model checking[J]. Proceedings of the IEEE, 2015, 103(11): 2021-2035.
- [95] Ge Y, Barrett C, Tinelli C. Solving quantified verification conditions using satisfiability modulo theories[C]. International Conference on Automated Deduction. Springer, Berlin, Heidelberg, 2007: 167-182.
- [96] Ansótegui C, Bofill M, Palahí M, et al. Solving weighted CSPs with meta-constraints by reformulation into satisfiability modulo theories[J]. Constraints, 2013, 18(2): 236-268.
- [97] 熊伟. 可满足性问题 DPLL 算法研究[D]. 上海: 复旦大学, 2007.
- [98] 郝彬彬, 鲁金直, 李军, 等.基于特定域建模方法的航空发动机 MBSE 实施方法[J]. 科技导报, 2019(7): 8.
- [99] Kai C, Sztipánovits J, Ab De Lwalled S, et al. Semantic Anchoring with Model Transformations[C]. Proceedings of the First European conference on Model Driven Architecture: foundations and Applications. Springer Berlin Heidelberg, 2005:1-9.
- [100] The SMT-LIB Initiative. SMT-LIB [EB/OL]. [2022-5-3]. <http://smtlib.cs.uiowa.edu.shtml>.
- [101] Healy A, Monahan R, Power J F . Predicting SMT Solver Performance for Software Verification[J]. Food Microbiology, 2017: 254.
- [102] Clark A, Evans A, Kent S, et al. The MMF approach to engineering object-oriented design languages.[C]. 2001:1-15.

附录 A 语法规范（部分）

（1）操作符及其表达式

■ 字符串类型的内置操作

String 类型有一些内置操作，以下及将分别进行介绍：

- length(): 返回字符串包含的字符数；
- concat(String s): 相当于“+”，将 s 添加到已有字符串后面；

```
1. String test = "Hello";
2. Int b = test.length();//
3. test.concat(" world!");//此时 test 变为"Hello world!"
4. test = test + " world!";//同理
```

- substring(Integer index1, Integer index2): 返回由 index1 开始到 index2 结束的子字符串

```
1. String test = "Hello";
2. String a = test.substring(1,3);//a 为 "ell"
```

- toInteger(): 转换成 Integer 类型；
- toReal(): 转换成 Real 类型；

```
1. String test = "10";
2. Int a = test.toInteger();
3. Real b = test.toReal();
```

- toUpper()和 toLower(): 转换成大写或者小写形式；
- trim(): 去掉首尾空格；
- indexOf(String s)和 lastIndexOf(String s) : 分别返回 s 的第一个索引和最后一个索引；

```
1. String test = "experiment";
2. Int a = test.indexOf(e);//返回 0
3. Int b = test.indexOf(e);//返回 7
```

- 比较运算符：“==”相当于 java 中的 equals

```

1. String test = "experiment";
2. String test2 = "experments";
3. Boolean condition = true;
4. if(test.equals(test2))
5.   condition = false;
6. end

```

■ 算数类型的操作符及其运算表达式

该小结说明算数类型的表达式可以使用的操作符，以及对应的运算表达式。下列说明中的“expression”特指返回数据类型为算数类型的表达式。

- 加法运算符“+”：通过加法运算符连接算数表达式，返回的仍是算数表达式。在 SMTAnalysis 模块中，加法运算符通常用于表达约束情况、进行计算等。

```

1. Int y;
2. Int x;
3. Int y = 1+3; // 进行计算，构建运算表达式

```

SolverA.add(x == y+4); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中

- 减法运算符“-”：通过减法运算符连接算数表达式，返回的仍是算数表达式。在 SMTAnalysis 模块中，减法运算符通常用于表达约束情况、进行计算等。

```

1. Int y;
2. Int x = y-4; // 进行计算，构建运算表达式

```

SolverA.add(x > y-4); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中

- 乘法运算符“*”：通过乘法运算符连接算数表达式，返回的仍是算数表达式。在 SMTAnalysis 模块中，乘法运算符通常用于表达约束情况、进行计算等。

```

1. Int y;
2. Int x = 2+4*y; // 进行计算，构建运算表达式

```

SolverA.add(x > y-4*3); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中

- 除法运算符“/”：通过除法运算符连接算数表达式进行除法运算，返回的仍是

算数表达式。

在 SMTAnalysis 模块中，除法运算符通常用于表达约束情况、进行计算等。

```
1. Int y;
2. Int x = y/2+4; // 进行计算，构建运算表达式
3. SolverA.add( x > y/3); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中
```

- 乘方运算符 “**”: 通过乘方运算符连接算数表达式进行乘方运算，返回的仍是算数表达式。

在 SMTAnalysis 模块中，乘方运算符通常用于表达约束情况、进行计算等。

```
1. Int y;
2. Int x = y**2+4; // 进行计算，构建运算表达式
3. SolverA.add( x > y**2); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中
```

- 括号运算符 “(expression)”: 通过 “()” 优先返回括号内的数值表达式，返回的仍是算数表达式。

在 SMTAnalysis 模块中，括号运算符通常用于辅助表达约束情况、进行计算等以突出运算优先级，带括号的运算表达优先级最高。

```
1. Int y;
2. Int x = y**(2+4); // 进行计算，构建运算表达式
3. SolverA.add( (x+3)*5 > y); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中
```

- 绝对值运算符 “|expression|”: 通过 “| |” 优先计算表达式的绝对值，返回的仍是算数表达式。

在 SMTAnalysis 模块中，绝对值运算符通常用于表达约束情况、进行计算等。

```
1. Int y;
2. Int x = y*2+|4-9|; // 进行计算，构建运算表达式
3. SolverA.add( |x-3|*4 > y); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中
```

- 比较运算符: 比较运算符包含全等运算符 “==”、不等运算符 “!=”、大于运算符 “>”、大于等于运算符 “>=”、小于运算符 “<”、小于等于运算符 “<=”。

比较运算符用于比较两个算数表达式的值。当用运算符比较两个值时，结果返回的是一个布尔逻辑值。

在 SMTAnalysis 模块中，比较运算符通常用于表达各类约束情况。

```
1. Int y;
2. Int x = y*2+|4-9|; // 进行计算，构建运算表达式
3. SolverA.add( |x-3|*4 >y);
4. SolverA.add( x-3*4 <=y); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，并将约束添加在求解器中
```

- ***简化运算符**: 简化运算符只用于 SMTAnalysis 模块，可用于简化数值表达式。

在 SMTAnalysis 模块中，简化运算符通常用于读取简化后的表达式。

```
1. Int y;
2. Int x = y*2+|4-9|+y; // 进行计算，构建运算表达式
3. Int xy = x.simplify(); //简化 x 表达式
4. print(xy);
5. => 3*y-5
```

■ 布尔类型的操作

该小结说明布尔类型的表达式可以使用的操作符，以及对应的运算表达式。下列说明中的“expression”特指返回数据类型为布尔类型的表达式。

- **逻辑与运算符“&&”**: 通过与运算符连接布尔表达式，返回的仍是布尔表达式。

在 SMTAnalysis 模块中，逻辑与运算符通常用于进行布尔计算、表达约束情况等。

```
1. Int y;
2. Int x;
3. Boolean b;
4. Boolean a;
5. Int y = 1+3; // 进行计算，构建算数运算表达式
6. Boolean a = b && (x<y); // 构建布尔运算表达式
7. SolverA.add( (x == y+4)&&a); // 假设有一求解器 SolverA，构建约束表达 x、y 的关系、布尔表达式 a、b 的关系，并将约束添加在求解器中
```

- **逻辑或运算符“||”**: 通过或运算符连接布尔表达式，返回的仍是布尔表达式。

在 SMTAnalysis 模块中，逻辑或运算符通常用于进行布尔计算、表达约束情况等。

```

1. Int y;
2. Int x;
3. Boolean b;
4. Boolean a;
5. Int y = 1+3; // 进行计算, 构建算数运算表达式
6. Boolean a = b && (x<y); // 构建布尔运算表达式
7. SolverA.add( (x == y+4)||a); // 假设有一求解器 SolverA, 构建约束表达 x、
  y 的关系、布尔表达式 a、b 的关系, 并将约束添加在求解器中

```

- 逻辑非运算符“!”: 通过非运算符构建布尔表达式, 返回的仍是布尔表达式的非运算表达式。

在 SMTAnalysis 模块中, 逻辑非运算符通常用于进行布尔计算、表达约束情况等。

```

1. Int y;
2. Int x;
3. Boolean b;
4. Boolean a;
5. Int y = 1+3; // 进行计算, 构建算数运算表达式
6. Boolean a = (!b) && (x<y); // 构建布尔运算表达式
7. SolverA.add( (x == y+4)&&a); // 假设有一求解器 SolverA, 构建约束表达 x、
  y 的关系、布尔表达式 a、b 的关系, 并将约束添加在求解器中

```

- 逻辑异或运算符“^”: 通过异或运算符连接布尔表达式, 返回的仍是布尔表达式。

在 SMTAnalysis 模块中, 逻辑异或运算符通常用于进行布尔计算、表达约束情况等。

```

8. Int y;
9. Int x;

1. Boolean b;
2. Boolean a;
3. Int y = 1+3; // 进行计算, 构建算数运算表达式
4. Boolean a = b && (x<y); // 构建布尔运算表达式
5. SolverA.add( (x == y+4)&&b); // 假设有一求解器 SolverA, 构建约束表达 x、
  y 的关系、布尔表达式 a、b 的关系, 并将约束添加在求解器中

```

- 逻辑全等运算符“==”: 通过全等运算符连接布尔表达式, 获取两布尔表达式是否相等的结果。

在 SMTAnalysis 模块中，逻辑全等运算符通常用于进行布尔计算、表达约束情况等。

```
1. Int y;
2. Int x;
3. Boolean b;
4. Boolean a;
5. Int y = 1+3; // 进行计算，构建算数运算表达式
6. Boolean a = b && (x<y); // 构建布尔运算表达式
7. SolverA.add( (x == y+4)==b); // 假设有一求解器 SolverA，构建约束判断 x =
   = y+4 和 b 两布尔表达式是否全等，并将约束添加在求解器中。
```

- 优先运算符 “()”：通过全等运算符连接布尔表达式，获取两布尔表达式是否相等的结果。

在 SMTAnalysis 模块中，逻辑全等运算符通常用于进行布尔计算、表达约束情况等。

```
1. Int x;
2. Boolean b;
3. Boolean a;
4. Boolean a = x<7; // 构建布尔运算表达式
5. SolverA.add( ((!b) ||a)); // 假设有一求解器 SolverA，构建关于 a 和 b 关系的
   约束判断。
```

- 蕴含运算符 “ $\Rightarrow$ ”：通过蕴含运算符表达一种逻辑关系。复合命题“如果 p，则 q”称为 p 与 q 的蕴含式，记作 $p \Rightarrow q$ 。p 与 q 的蕴含式的实质是 q 是 p 的必要条件，或 p 是 q 的充分条件。返回的仍是逻辑表达式。

在 SMTAnalysis 模块中，蕴含运算符通常用于辅助表达约束情况。

```
1. Boolean b = true;
2. Boolean a = false;
3. Boolean c = a>b; //构建 a 和 b 的蕴含表达式，c 值为 true
4. Boolean d = b>a; // 构建 b 和 a 的蕴含表达式，d 值为 false
5. SolverA.add( c>d); // 假设有一求解器 SolverA，构建约束表达 x 和 y 的关系，
   并将约束添加在求解器中
6.
```

■ 数组类型的操作符及其运算表达式

该小结说明数组类型的表达式可以使用的操作符，以及对应的运算表达式。下列说明中的“**expression**”特指返回数据类型为数组类型的表达式。

● 数组存储运算方法 “**store**”:

通过“**store**”调用数组存储方法，用于在数组的特定位置存储数据，返回的是完成存储特定位置数据的数组。方法形式一般为“**store(int i, primitive p)**”，方法需要输入两个值，**int i** 表示数组中的位置，为一整型数据，**primitive p** 表示被储存的值。**p** 的数据类型 **primitive**，根据定义数组存储数据的类型决定。

在 **SMTAnalysis** 模块中，数组存储运算方法通常用于进行数组计算、表达约束情况等。

```
1. Array(Int, Int) a1;
2. Array(Int, Int) a2;
3. Array(Int, Boolean) b1;
4. Int x;
5. Int y;
6. Boolean z;
7. a2 = a1.store(x,y); // 进行数组运算，将数组 a1 的第 x 个值替换为数值 y，将替换后的数组为数组 a2 赋值。
8. SolverA.add(b1==b1.store(y,z)); // 假设有一求解器 SolverA，构建约束表达，判断数组 b1 和将 b1 的第 y 个值替换为布尔值 z 后的数组是否相等。
```

● 数组选择运算方法 “**select**”:

通过“**select**”调用选择数组中数据方法，用于选择数组中特定位置的数据，返回的是数组中的数据，返回的类型和数组存储的数据类型一样。方法形式一般为“**select(int i)**”，方法需要输入一个值，**int i** 表示数组中要选择的数据的位置，为一整型数据，返回该数组 **i** 位置的数据。

在 **SMTAnalysis** 模块中，数组选择方法通常用于进行数组计算、表达约束情况等。

```
1. Array(Int, Int) a1;
2. Int x;
3. Int y;
4. y= a1.select(x); //将 a1 数组的第 x 个值提取出，并赋值给 y。
5. SolverA.add(x==a1.select(x)); // 假设有一求解器 SolverA，构建约束表达，判断数值 x 和将 a1 的第 x 个值是否相等。
```

● 数组添加数据运算方法 “**arrayAdd**”:

通过“arrayAdd”调用添加数据运算方法，用于在数组末尾添加指定的数据，返回的是完成添加数据的数组，完成添加后，数组长度加一，返回的类型和原数组类型相同。方法形式一般为“arrayAdd(primitive p)”，方法需要输入一个值，primitive p 表示被储存的值，数据类型 primitive 必须和数组原来存储的数据类型一致。

在 SMTAnalysis 模块中，数组添加数据方法通常用于进行数组计算、表达约束情况等。

```
1. Array(Int, Int) a1;
2. Array(Int, Int) a2;
3. Int x;
4. a2= a1.arrayAdd(x); //在 a1 数组的末尾添加一个新的整数 x，并赋值给数组 a2。
```

● 数组减少数据方法“arrayDel”:

通过“arrayDel”调用减少数据运算方法，用于在数组中删除指定的数据，返回的是完成减少数据的数组，完成减少数据后，数组长度减一，返回的类型和原数组类型相同。方法形式一般为“arrayDel(primitive p)”，方法需要输入一个值，primitive p 表示被删除的值，数据类型 primitive 必须和数组原来存储的数据类型一致。

在 SMTAnalysis 模块中，数组减少数据方法通常用于进行数组计算、表达约束情况等。

```
1. Array(Int, Int) a1;
2. Array(Int, Int) a2;
3. Int x;
4. a2= a1.arrayDel(x); //在 a1 数组中减少一个特定整数 x，并赋值给数组 a2。
```

● 计算数组长度方法“range()”:

通过“range()”调用计算数组长度的方法，返回的是数组的长度，类型一般为整型。方法形式一般为“range()”。

在 SMTAnalysis 模块中，计算数组长度方法通常用于进行数组计算、表达约束情况等。

```
1. Array(Int, Int) a1;
2. Array(Int, Int) a2;
3. Int x;
4. a2= a1.arrayAdd(x); //在 a1 数组的末尾添加一个新的整数 x，并赋值给数组 a2。
```

```
5. SolverA.add(a2.range()>a1.range());
```

假设有一求解器 SolverA，构建约束表达，判断数组 a2 的长度是否大于数组 a1 的长度。

● 数组全等运算符 “==”:

通过数组全等运算符连接数组表达式，获取两数组表达式是否相等的结果。数组相等指的是两个长度一样的数组，数组内每个值都一一对应地相等。

在 SMTAnalysis 模块中，数组全等运算符通常用于进行数组计算、表达约束情况等。

```
1. Array(Int, Boolean) b1;
2. Int y;
3. Boolean z;
4. SolverA.add(b1==b1.store(y,z));
```

// 假设有一求解器 SolverA，构建约束表达，判断数组 b1 和将 b1 的第 y 个值替换为布尔值 z 后的数组是否相等。

6.7 矩阵类型的操作符及其运算表达式

该小结说明矩阵类型的表达式可以使用的操作符，以及对应的运算表达式。下列说明中的“expression”特指返回数据类型为矩阵类型的表达式。

● 加法运算符 “+”:

通过加法运算符连接矩阵表达式，计算形式遵守矩阵地加法规则，两矩阵的行数和列数相等，返回的仍是矩阵表达式，行数与列数与原矩阵地行列数相等。

在 SMTAnalysis 模块中，加法运算符通常用于表达矩阵地约束情况、进行计算等。

```
1. Matrix m = IntergerMatrix.initial(2,3);
```

// 声明变量 m 为存储整型的矩阵

```
2. m =[1,2,3;1,2,3];
```

// 为变量 m 赋值

```
3. Matrix M = [2,3,4;5,6,7];
```

```
4. Matrix P = M+m;
```

//进行计算，构建运算表达式

● 减法运算符 “-”:

通过减法运算符连接矩阵表达式，计算形式遵守矩阵地减法规则，两矩阵的行数和列数相等，返回的仍是矩阵表达式，行数与列数与原矩阵地行列数相等。

在 SMTAnalysis 模块中，减法运算符通常用于表达矩阵的约束情况、进行计算等。

```
1. Matrix m = IntergerMatrix.initial (2,3);
```

// 声明变量 m 为存储整型的矩阵

```
2. m =[1,2,3;1,2,3];
```

// 为变量 m 赋值

```
3. Matrix M = [2,3,4;5,6,7];
```

```
4. Matrix P = M-m;
```

//进行计算，构建运算表达式

- 乘法运算符 “*”:

通过乘法运算符连接矩阵表达式，计算形式遵守矩阵地乘法规则。若矩阵的乘法形式为 $M_{m \times n} * N_{n \times b}$ ，则第一个矩阵的列数 n 和第二个矩阵的行数 a 需要相等，返回的仍是矩阵表达式，行数为第一个矩阵的行数 m 和第二个矩阵的列数 b 。

在 SMTAnalysis 模块中，乘法运算符通常用于表达矩阵的约束情况、进行计算等。

```
1. Matrix m = IntergerMatrix.initial (2,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3]; // 为变量 m 赋值
3. Matrix M = [2,3;4,5;6,7]; //新建一个矩阵 M
4. Matrix P =m*M; //进行计算，构建运算表达式,得到 2×2 的矩阵 P
```

- 转置运算符 “T()”:

转置运算符 T()能够调用矩阵的转置方法，即，将矩阵 $M_{m \times n}$ 的第 i 行第 j 列的数值放在新矩阵的第 j 行第 i 列上。返回的仍是矩阵表达式，新矩阵的行数为原矩阵的列数，新矩阵的列数为原矩阵的行数。

在 SMTAnalysis 模块中，转置运算符通常用于表达矩阵计算和约束。

```
1. Matrix m = IntergerMatrix.initial (2,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3]; // 为变量 m 赋值
3. Matrix n = m.T();
4. Matrix M = [2,3;4,5;6,7];
5. SolverA.add(n==M); //假设有一求解器 SolverA，构建约束表达，判断 m 的转置矩阵和矩阵 M 是否相等。
```

- 逆矩阵运算符 “inv()”:

逆矩阵运算符 inv()能够调用矩阵的逆方法，设 A 是一个 n 阶矩阵，若存在另一个 n 阶矩阵 B ，使得： $AB=BA=E$ ， E 为 n 阶单位矩阵，则称方阵 A 可逆，并称方阵 B 是 A 的逆矩阵。逆矩阵运算符用来求矩阵的逆。

在 SMTAnalysis 模块中，转置运算符通常用于表达矩阵计算和约束。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Matrix n = m.inv();
4. SolverA.add(n==m); //假设有一求解器 SolverA，构建约束表达，判断 m 的逆矩阵和矩阵 m 是否相等。
```

- 获取矩阵的行方法 “getRow”:

通过“getRow”调用获取矩阵的行方法,用于得到矩阵中的指定行,若原矩阵为 $Mm \times n$, 则返回的 $1 \times n$ 的行向量,也就是行数为 1 列数和原矩阵列数相等的矩阵。方法形式一般为“getRow(int i)”,方法需要输入一个值, int i 表示指定矩阵的第 i 行, i 的数据类型为整型,获取的就是该行的行向量。

在 SMTAnalysis 模块中,获取矩阵的行方法通常用于进行矩阵计算、表达约束情况等。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Matrix n = [3,2,1];
4. SolverA.add(n==m.getRow(2)); //假设有一求解器 SolverA, 构建约束表达, 判断 m 的第 2 行和矩阵 n 是否相等。
```

- 获取矩阵的列方法“getCol”:

通过“getCol”调用获取矩阵的列方法,用于得到矩阵中的指定列,若原矩阵为 $Mm \times n$, 则返回的 $m \times 1$ 的列向量,也就是列数为 1 列数和原矩阵行数相等的矩阵。方法形式一般为“getCol(int j)”,方法需要输入一个值, int j 表示指定矩阵的第 j 列, j 的数据类型为整型,获取的就是该列的列向量。

在 SMTAnalysis 模块中,获取矩阵的列方法通常用于进行矩阵计算、表达约束情况等。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Matrix n = [3,2,1];
4. SolverA.add(n==m.getCol(3)); //假设有一求解器 SolverA, 构建约束表达, 判断 m 的第 3 列和矩阵 n 是否相等。
```

- 矩阵的求和方法“sum()”: 通过“sum()”调用获取矩阵的和的方法,用于得到矩阵中各数值的和,返回的为矩阵各项之和。若矩阵为整型矩阵,则求和后返回的类型为整型,若矩阵为实数型矩阵,则求和后返回的类型为实数型。

在 SMTAnalysis 模块中,获取矩阵的和的方法通常用于进行矩阵计算、表达约束情况等。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Int x = 15;
```

```
4. SolverA.add(m.sum()==x); //假设有一求解器 SolverA, 构建约束表达, 判断 m 矩阵的各项之和是否和 x 相等。
```

● 矩阵行的求和方法 “sum(int i)”:

通过 “sum(int i)”调用获取矩阵某行的和的方法。方法形式一般为 “sum(int i)”, 方法需要输入一个值, int i 表示指定矩阵的第 i 行, i 的数据类型为整型, 获取的就是该行的各项之和。若矩阵为整型矩阵, 则求和后返回的类型为整型, 若矩阵为实数型矩阵, 则求和后返回的类型为实数型。

在 SMTAnalysis 模块中, 矩阵行的求和方法通常用于进行矩阵计算、表达约束情况等。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Int x = 15;
4. SolverA.add(m.sum(2)==x); //假设有一求解器 SolverA, 构建约束表达, 判断 m 矩阵第二行的各项之和是否和 x 相等。
```

● 矩阵列的求和方法 “sum(; , int j)”:

通过 “sum(; , int j)”调用获取矩阵某列的和的方法。方法形式一般为 “sum(; , int j)”, 方法需要输入一个值, int j 表示指定矩阵的第 j 列, j 的数据类型为整型, 获取的就是该列的各项之和。若矩阵为整型矩阵, 则求和后返回的类型为整型, 若矩阵为实数型矩阵, 则求和后返回的类型为实数型。

在 SMTAnalysis 模块中, 矩阵列的求和方法通常用于进行矩阵计算、表达约束情况等。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Int x = 15;
4. SolverA.add(m.sum(: , 2)==x); //假设有一求解器 SolverA, 构建约束表达, 判断 m 矩阵第二列的各项之和是否和 x 相等。
```

● 矩阵全等运算符 “==”:

通过矩阵全等运算符连接矩阵表达式, 获取两矩阵表达式是否相等的结果。矩阵相等指的是两个行数和列数一样的矩阵, 矩阵内每个值都一一对应地相等。

在 SMTAnalysis 模块中, 数组全等运算符通常用于进行矩阵计算、表达约束情况

等。

```
1. Matrix m = IntergerMatrix.initial (3,3); // 声明变量 m 为存储整型的矩阵
2. m =[1,2,3;1,2,3;3,2,1]; // 为变量 m 赋值
3. Matrix n = m.inv();
4. SolverA.add(n==m); //假设有一求解器 SolverA, 构建约束表达, 判断 m 的逆矩阵
    和矩阵 m 是否相等。
```

(2) 执行语句

■ 打印语句

`print` 打印语句可以将其内容输出到控制台, `print` 语句可以打印 KARMA 中所有基本数据类型的数据, 例如 `String` 类型、`Number`、`Int`、`Boolean` 等。可以打印的内容如下:

- 元素属性: 将属性信息以字符串的形式输出;
- 字符串: 将字符串原样输出, 特殊字符需转义 (使用 \ 斜杆进行转义), 例如单引号 ' 需转义成 \';
- 自定义变量: 输出时会把所有的变量转换为字符串形式;
- 转义字符: 关键字 `newline` 或者 `/n` 形式输出换行符;

`print` 打印语句的具体语法如下所示: 其中 `var1` 以及 `var2` 代表变量名或者字符串, 中间以逗号形式隔开, 分号 “;” 后的为控制标志, 是可选项, 下例中的 `end= “\n”` 即输出一个换行符。

```
1. print (var1(,var2)*; [end=“\n”]?);
```

■ For 循环

`for` 循环的功能非常强大, 它使用计数器实现循环。`for` 循环会先初始化计数器, 然后, 在每次循环前检测循环条件, 在每次循环后更新计数器。计数器变量通常命名为 `i`。`for` 循环的基础语法如下, 其中循环后更新计数器部分如果默认为每次加 1 则可省略。

```
5. for(初始条件:结束条件:循环后更新计数器)
6. statements
7. end
8. //简单示例
9. Int sum = 0;
```

```

10.for(Int a=1:a<=5:a++)
11.sum = sum + a;
12.print(sum);
13.end

```

同时，for 循环的计数器可以自定义，例如 $i = i+2$ ，如下例所示：

//计算 10 以内所有奇数的和

```

14.Int sum = 0;
15.for(Int a=1:a<=10:a=a+2)
16.sum = sum + a;
17.end
18.print(sum);

```

for 循环可以嵌套，示例如下：

```

19.Int sum = 0;
20.for(Int i=1:i<=10:i++)
21.for(Int j=1:j<=10:j++)
22. sum = sum + i*j;
23.end
24.end

```

在指标验证中，可以使用 for 语句执行循环结构，简化约束的添加。

```

25.Matrix X1 = IntegerMatrix. Initial(3,3);
26.//初始化一个 3X3 的矩阵
27.for(Int a=0:a<3:a=a+1)
28.for(Int b=0:b<3:b=b+1)
29.SolverA.add(X1.get(a,b)>2)
30.//添加约束，使得矩阵中的每一个值的大小都大于二
31.end
32.end

```

■ If 语句

在 KARMA 程序中，如果要根据条件来决定是否执行某一段代码，就需要 if 语句。根据 if 的计算结果（true 还是 false），决定是否执行 if 语句块。if 语句的基本语法是

```

1. if (condition)    // 条件满足时执行，condition 是返回为布尔类型的表达式

```

```
2. <执行>
3. end
4. 在 SMTAnalysis 的模块中, if 语句常根据求解器求解的结果, 判断执行哪一段代码。
   如:
5. if (SolverA.check==SAT)
6.   print("Satisfiable");
7. end
8. //假设有一通用求解器 SolverA, 求解器内已有约束, 判断求解器求解后的结果是否为
   SAT, 如果是 SAT, 则打印 "Satisfiable"
```

if 语句还可以编写一个 else, 当条件判断为 false 时, 将执行 else 的语句块: if-else 语句的基本语法为:

```
9. if (condition) // 条件满足时执行, condition 是返回为布尔类型的表达式
10. <执行 1>
11. else // 条件不满足时
12. <执行 2>
13. end
```

在 SMTAnalysis 的模块中, if 语句常根据求解器求解的结果, 判断执行哪一段代码。如:

```
14. if (SolverA.check==SAT)
15.   print("Satisfiable");
16. else
17.   print("Unsatisfiable");
18. end
```

//假设有一通用求解器 SolverA, 求解器内已有约束, 判断求解器求解后的结果是否为 SAT, 如果是 SAT, 则打印 "Satisfiable" 否则打印 "unsatisfiable"

还可以用多个 if ... elseif ... 串联, if-elseif-else 语句的基本语法为:

```
19. if (condition1)
20.   <执行 1>
21. elseif (condition2)
22.   <执行 2>
23. else
24.   <执行 4>
25. end
```

if 嵌套语句的基本语法如下：

```
26.if (condition1)
27.if (condition2)
28. if (condition3)
29.  if (condition4)
30.    <执行 1>          else
31.    <执行 2>
32.  end
33. else
34.  <执行 3>
35. end
36.else
37. <执行 4>
38.end
39.else
40.<执行 5>
41.end
```

在 SMTAnalysis 的模块中，if 语句常根据求解器求解的结果，判断执行哪一段代码。如：

```
42.if (SolverA.check==SAT)
43.SolverA.solution();
44.elseif (SolverA.check==UNSAT)
45.print("Unsatisfiable");
46.else
47. print("Unknown");
48.end
49.//假设有一通用求解器 SolverA，求解器内已有约束，判断求解器求解后的结果是否为 SAT，如果是 SAT，则输出一组满足全局约束的解，如果求解结果为 UNSAT，则打印“Unsatisfiable”，如果求解结果两者都不是，则打印“unknown”。
```

攻读学位期间发表论文与研究成果清单

发表论文：

[1] Model-based system engineering supporting production scheduling based on satisfiability modulo theory. Journal of Industrial Information Integration[J]. 2022 (第一作者, SCI I 区 IF: 10.063)

[2] A Semantic Modeling Approach Supporting Property Verification based on Satisfiability Modulo Theories[C]. 2022 IEEE System Engineering Conference (第一作者, EI 检索)

科研项目

- (1) 基础加强计划重点基础研究项目课题
- (2) 科技部网络协同制造和智能工厂项目
- (3) 201 重点科研项目

相关专利

[1] 一种支持模型属性量化评估的指标验证技术 专利号：2022103292260 (第一作者)

在投论文

[1] Model-based Systems Engineering Supporting Manufacturing System Planning through Semantic Modeling[J]. Journal of Manufacture Producing. (第二作者, SCI II 区)

相关竞赛

- (1) 2021 年 9 月, 北京理工大学第十七届“世纪杯”学生创业竞赛银奖
- (2) 2021 年 9 月, 第六届中国国际“互联网+”大学生创新创业大赛国赛入围
- (3) 2021 年 5 月, 北京优秀创业团队一等奖
- (4) 2020 年 9 月, 第五届中国国际“互联网+”大学生创新创业大赛(北京赛区复赛)一等奖

致谢

我终于拖到了最后一天，因为我知道，致谢的结尾就是研究生的句点。

首先，感谢我的导师，王国新老师。三年前，我有幸进入知识工程组，开始全新领域的研究。感谢王老师三年来对我们的悉心培育。王老师无疑是一位治学严谨的老师，他勤恳治学的作风、踏实肯干的态度春风化雨般影响了我们的科研态度和工作习惯。其次，感谢鲁金直博士，鲁博士虽然身在国外，但时刻关系我们的科研。每当我科研上遇到技术问题，他总会把问题放在心上，帮我解决。感谢吴晗老师和李雪原老师在本科阶段给我提供的帮助，感谢郝佳老师在研究生阶段对我研究的指点。感谢所有老师对我的耐心指导、鼓励帮助。同时，感谢我们的大师兄马君达博士，无论是学术问题还是项目瓶颈，他都会帮助我们闯过难关。最后，感谢课题组可爱的同学们：栗寒、丁洁、吴绶玄、李子航、董梦如、陈锐，以及曾经的师兄师姐们：王宏伟、郭江敏、余士严、靳益利，是他们和我一起走在科研道路上披荆斩棘。

四年本科、三年研究生，我的北理工生活也将画上句号。当离别的日子真的来临时，内心最多的还是不舍，不舍在这里结识的良师益友。感谢张慧、郅天琪、赵凌萱、张宇航、曾洺锴、郑群锋的七年相助，感谢杨梓红、刘吉威、韦睿川、信子林、李琦、杨子传的三年同进。感谢你们用温暖消解我的别扭、用昂扬驱散我的郁闷、用相信激励我的信心，也感谢你们在百忙中还会帮我检查论文、在苦闷中不忘用欢笑驱融压力，我们互相说着生活工作的如意和不容易，也不忘展望更好的明天。感谢室友刘聪、欧洋佳欣的三年陪伴和包容，有幸成为你们的室友，让我在集体宿舍也能找到倚靠的港湾。我本想一一写下对你们的感谢，却又提笔忘言，只能希冀来日方长。

感谢父母的支持与理解。你们无论在什么时候都能够支持我，鼓励我、包容我也理解我。我困顿时，你们听我疏泄、给我意见，我烦闷时，你们听我诉说、给我支持。我性格急躁，非黑即白，感谢你们用包容融化我的棱角，让我知道世界也不太差。感谢小姨、外公、外婆等家人的关心和爱护，你们对我的爱从没随时间消解半分。

感谢这七年的北理工生活。七年间，我不止一次的想，如果我没有选择北理工，会怎么样。可惜生活没有重启键，我也没办法想象另一个维度的自己过的如何。回首七年，坦诚地说，是北理工的帮助我看到我的浮躁和浅显，是北理工的环境和指导让我一步一步看到自己的初心和志趣，是在北理工结识的朋友让我走出自己的小世界、拥抱真实不确定又残酷的世界。

最后，感谢疫情坚守在工作岗位的各位老师、职工和同学。没有你们，就没有我面前这安静的书桌，没有你们就没有我们有条不紊的生活。希望我们都能够找到平静。

千言万语汇成一句句感谢。虽然知道天下没有不散的宴席，也知道昨天今天和明天不过是一切的假设，但还是放任自己住在此时此刻的感怀，衷心地希望海内知己，天涯若邻。